DSAA 4011 · LECTURE 9 · THURSDAY 5 NOVEMBER 2026

Graph Neural Networks

Learned message passing: when the factors are unknown, a network learns the messages from examples

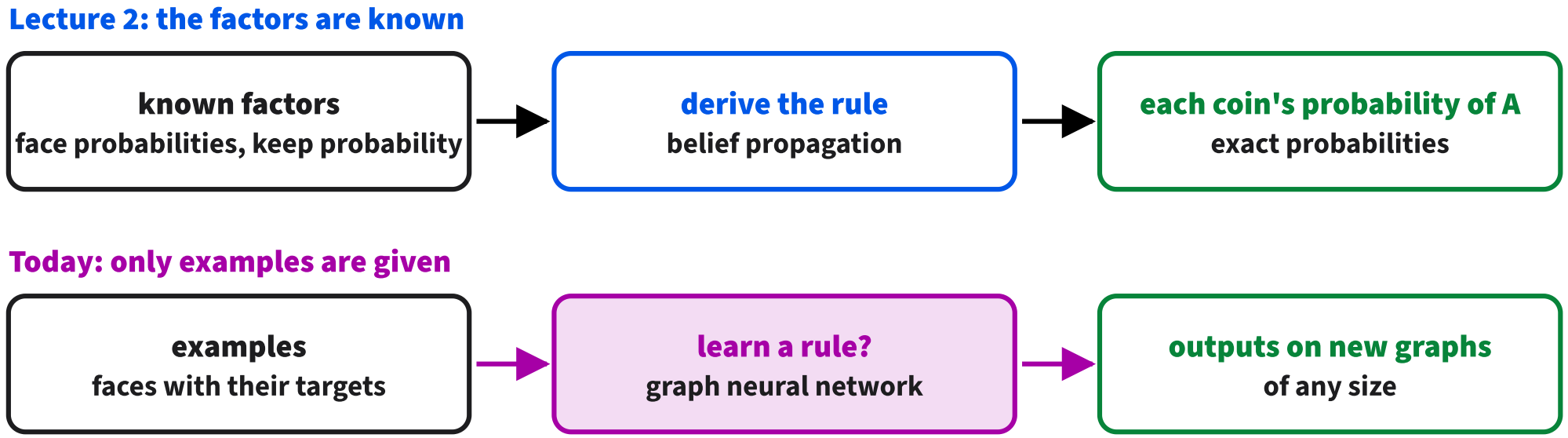
Builds on Lecture 2 (belief propagation) and Lecture 4 (Markov chains)

Yuwen Huang
Advanced Machine Learning and Deep Learning · HKUST (Guangzhou)

THE MAIN QUESTION

Can a network learn the message rule from examples?

In Lecture 2, belief propagation **derived** its messages from **known** factors.



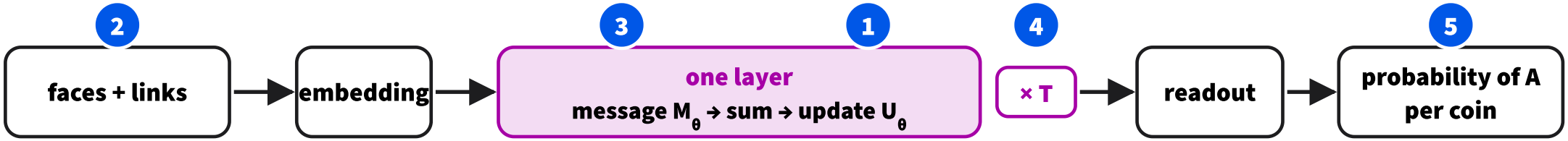
Today: **graph neural networks**, which learn the rule from examples. What can they do, and what can they not do?

Derived rules need factors; learned rules need examples.

TODAY'S PLAN

One network, five questions about it

↳ **First:** finishing Lecture 7, step by step (19 slides).



1	**What is one layer, and how do we learn it?**	**because belief propagation needs the factors; we have only examples**
2	**Does one trained network fit any numbering and size?**	**because we train on short chains, then use it on new graphs**
3	**Which message and sum should a layer use?**	**because real nodes have 3 or 300 neighbours, not always 2**
4	**How many layers does a coin need?**	**because its output depends on faces several coins away**
5	**Which graphs can it never tell apart?**	**because then we know when not to trust it**

then 6: all five results on one small network · 7: one application, Sudoku, and the Transformer

Each part answers one question about the same network.

 M_θ = message, ours $w \cdot s_j$ · U_θ = update, ours $u \cdot x_i$ + sum · θ = parameters, ours (u, w)

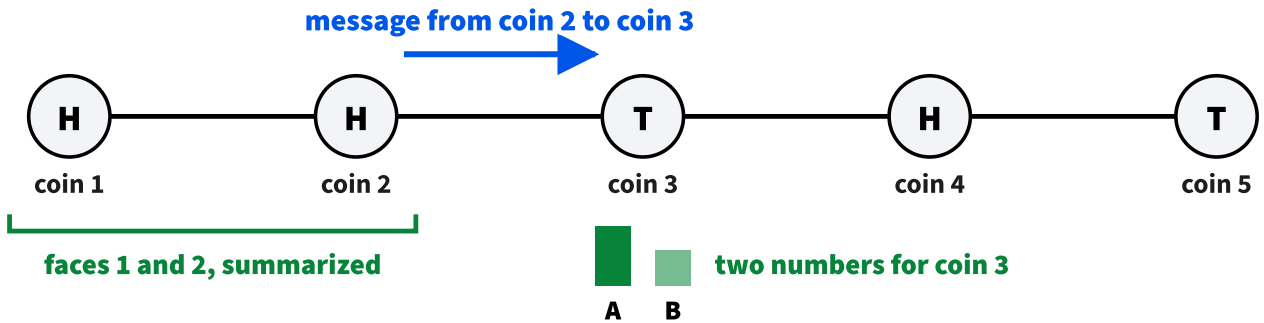
WARM-UP

Two ideas from earlier lectures that return today

1. On the coin chain, what does coin 2's message to coin 3 carry (Lecture 2)?

coin 2's face only | coin 3's final output | what faces 1 and 2 say about coin 3

Answer. What faces 1 and 2 say about coin 3, as two numbers. **Returns in Part 1**, as a learned message.



2. A Markov chain runs for many steps (Lecture 4). What happens to its start?

it is remembered | it is forgotten | it repeats

Answer. Forgotten: the chain settles at its stationary distribution. **Returns in Part 4**: deep networks forget their input.

OBJECTIVES

By the end of today, you can do five things

1. **write** belief propagation as a learnable layer — **message passing**
2. **explain** why one network fits every graph — **symmetry**
3. **build** three classic graph neural networks from one template — **three networks**
4. **predict** how far a network sees, and when depth fails — **depth**
5. **name** graphs no message-passing network tells apart — **limits**

One coin chain, one learned rule, five questions about it.

TODAY'S SYMBOLS

Each symbol today, with its meaning in words

symbol	meaning
$G = (V, E), n$	**a graph: nodes** V**, edges** E**;** $n = \lvert V \rvert$ **nodes**
$N(i), \deg i$	**the neighbours of node** i**; their number** $\deg i = \lvert N(i) \rvert$
x_i	**the observation (input) at node** i**; for a coin, its face:** $+1$ **for H,** -1 **for T**
$h_i^{(t)} \in \mathbb{R}^d$	**node** i**'s vector of** d **numbers after layer** t**;** $h_i^{(0)}$ **is computed from** x_i
$m_{j \to i}^{(t)}, a_i^{(t)}$	**the message from** j **to** i **in layer** t**; their sum** $a_i = \sum_{j \in N(i)} m_{j \to i}$
θ **(theta)**	**the parameters, the numbers the network learns; ours:** $\theta = (u, w)$**, two parameters**
s_i, u, w	**coin** i**'s score in our network;** u**: weight on its own face,** w**: weight on each neighbour**
M_θ, U_θ	**message and update; ours:** $M_\theta(s_i, s_j) = w \cdot s_j, U_\theta(s_i, a_i) = u \cdot x_i + a_i$
$T, \hat{y}_i, \sigma$	**the number of layers (rounds); the output for node** i**;** $\sigma(u) = 1/(1 + e^{-u})$
$A, \hat{A}, P$	**the adjacency matrix; its normalised form; a permutation matrix**
α_{ij}	**an attention weight**
$z_i,\ \varphi_i,\ \psi,\ \kappa$	**coin model: hidden coin (A or B); face factor** $\varphi_i(z_i) = p(x_i \mid z_i)$**; link factor; keep probability**

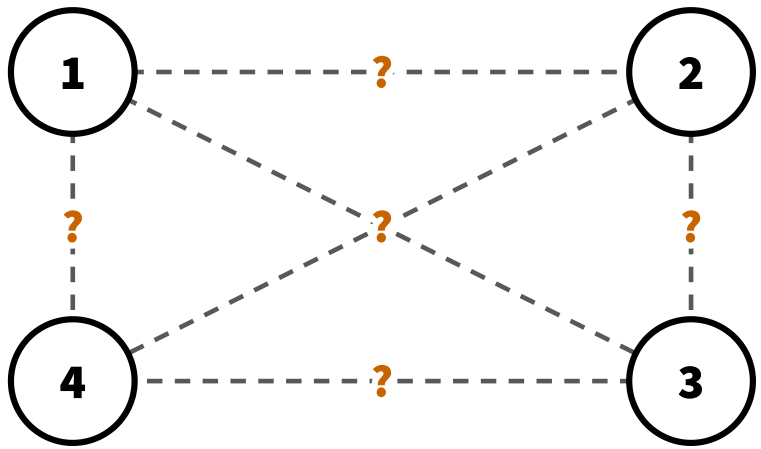
Subscript i**: a node; subscript** θ**: learned; superscript** (t)**: a layer.**

M$_{\boldsymbol{\theta}}$ = message, ours w·s_j · **U**$_{\boldsymbol{\theta}}$ = update, ours u·x_i + sum · **θ** = parameters, ours (u, w)

First, we finish Lecture 7: who copies whom?

Four friends toss a coin every evening, and some of them copy a friend. We only see their notebook. Can we find who copies whom?

some friends copy another friend's face



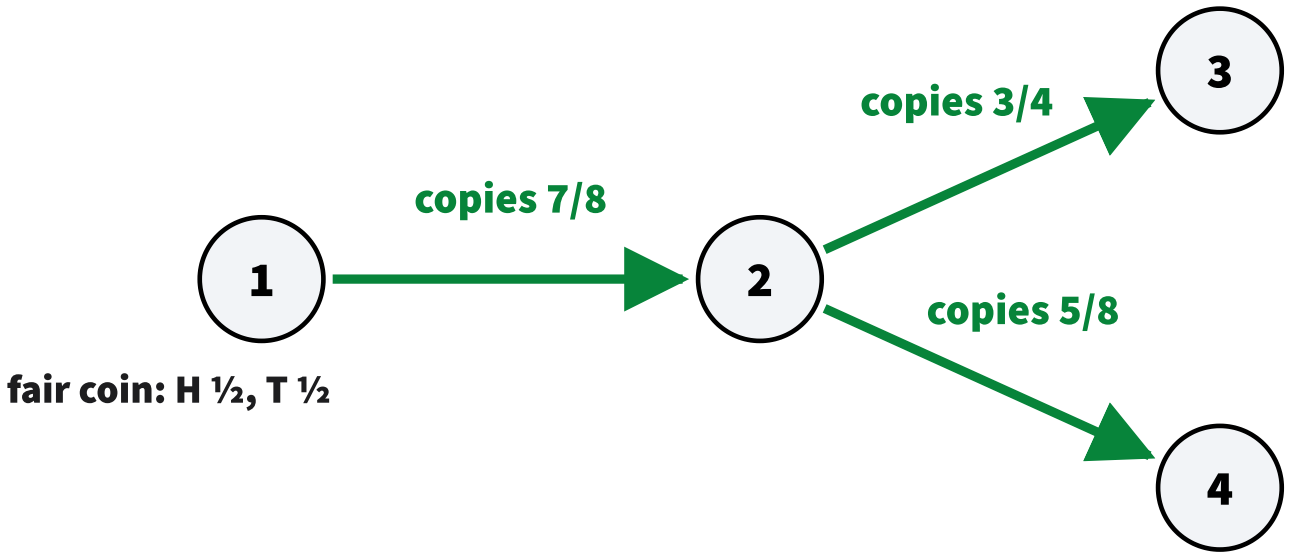
each evening, each friend writes one face: H or T

SO **We finish this example, then see where the idea is used today.**

NEXT What are the friends' real habits?

The answer we hope to find: three copy links

Friend 1 tosses a fair coin. Friend 2 copies friend 1 with chance 7/8; friends 3 and 4 copy friend 2 with chances 3/4 and 5/8. The method never sees this picture.

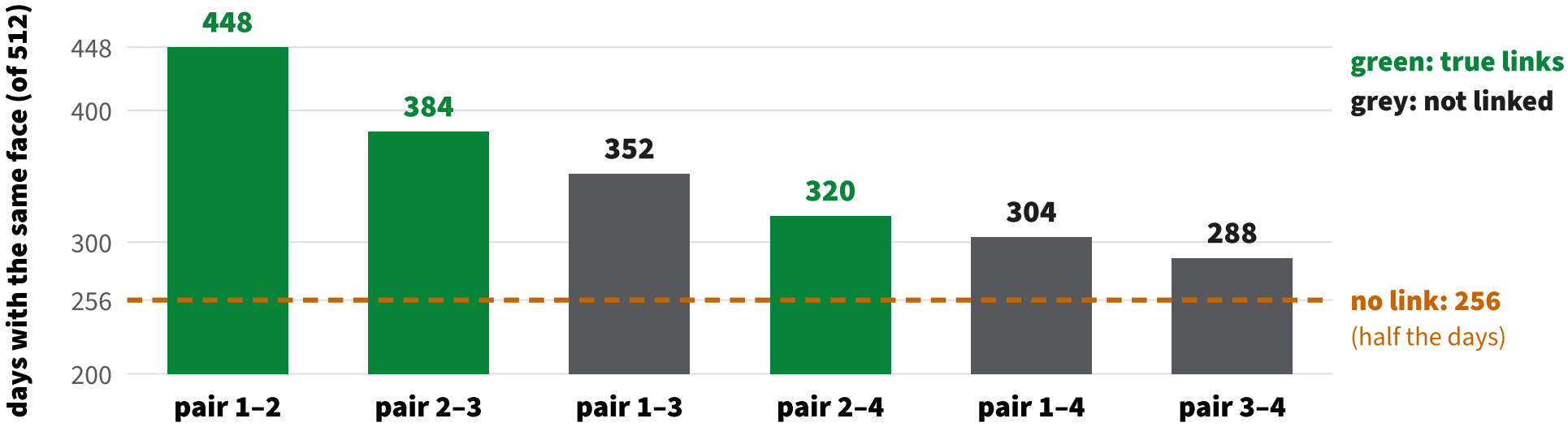


SO **A strong, a medium and a weak link.**

NEXT What does the notebook show?

Linked friends write the same face more often

In 512 evenings, count how often each pair wrote the same face. Two friends who never copy agree on half the evenings: 256.

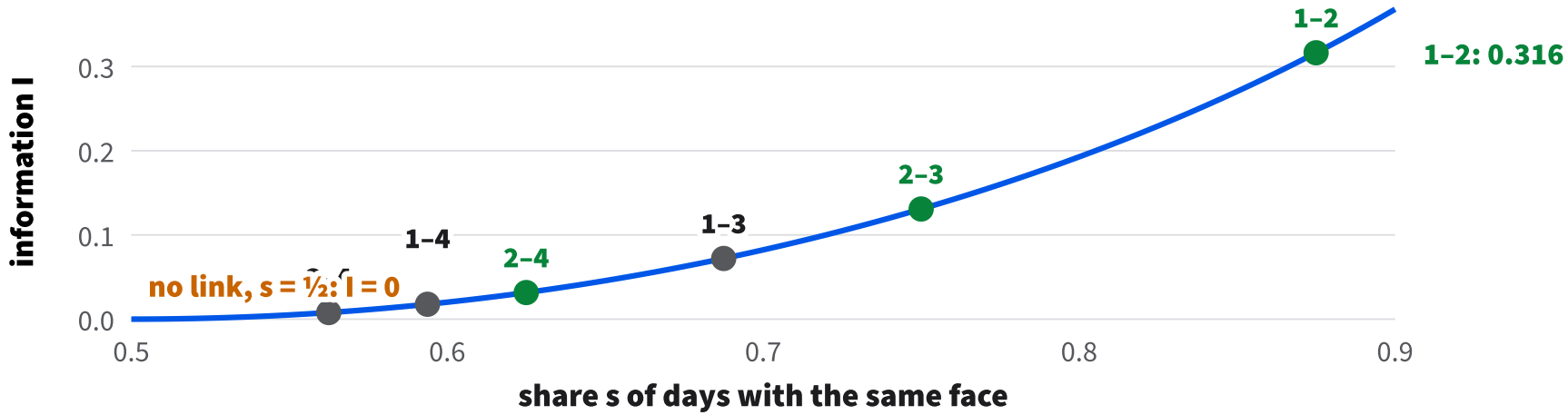


SO **Every pair agrees more than 256, even the three pairs with no link.**

NEXT How do we turn "how often the same" into one number?

One score per pair: zero at half the days, larger above

Lecture 7 turned each pair's agreement into one score, its **information**: 0 when the pair agrees on half the days, and larger the more often the pair agrees.

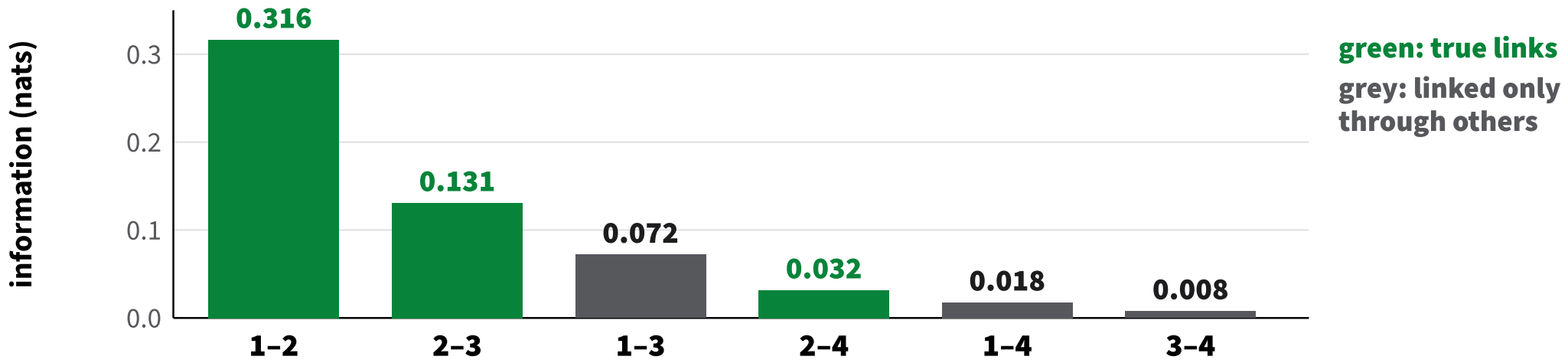


SO **Pair 1–2 agrees most and scores highest; no link scores 0.**

NEXT How do the six pairs rank?

The indirect pair 1–3 scores above the true link 2–4

The six scores, strongest first. Green bars are true links; grey bars are not.



SO **Keeping the three strongest pairs would pick 1–3 and miss 2–4.**

NEXT Why does pair 1–3 score so high?

Two copies in a row make friends 1 and 3 agree

Q: Neither friend 1 nor friend 3 copies the other. Why do they agree on 352 of 512 days? **A:** Through friend 2: friend 3 copies friend 2, who copies friend 1.



friend 3 copies friend 2, and friend 2 copies friend 1,
so friend 3 often writes friend 1's face too: on 352 of 512 days

more than the weak true link 2–4: 320 of 512 days

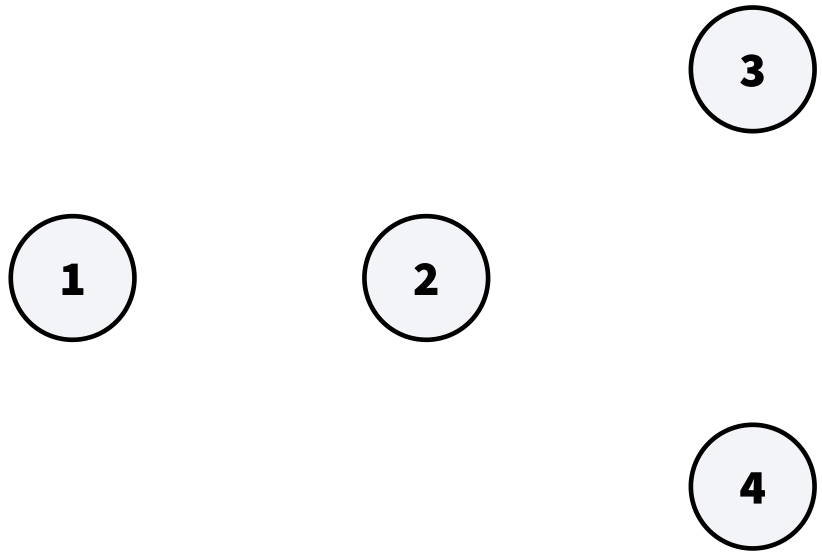
SO **A chain of strong copies can beat one weak direct copy.**

NEXT Which rule avoids the indirect pair?

Take the strongest pair, unless it closes a loop

Lecture 7's greedy rule (the Chow–Liu tree): go down the list of pairs; link a pair unless its two friends are already connected; stop at three links, because four friends need three links to form a tree.

	pairs, strongest first
1–2	0.316
2–3	0.131
1–3	0.072
2–4	0.032
1–4	0.018
3–4	0.008



Q: Which pair will the rule skip first? **A:** Pair 1–3: once 1–2 and 2–3 are linked, it would close a loop.

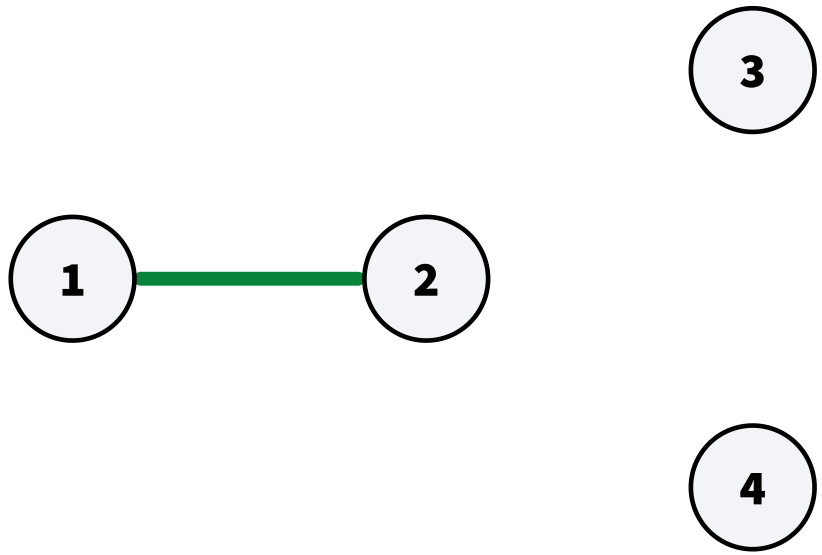
SO **One pass down the sorted list.**

NEXT Which pair does the rule take first?

Friends 1 and 2 score highest, so we link them

Pair 1–2 has the largest score, 0.316: link it.

pairs, strongest first		
1–2	0.316	✓ link
2–3	0.131	
1–3	0.072	
2–4	0.032	
1–4	0.018	
3–4	0.008	



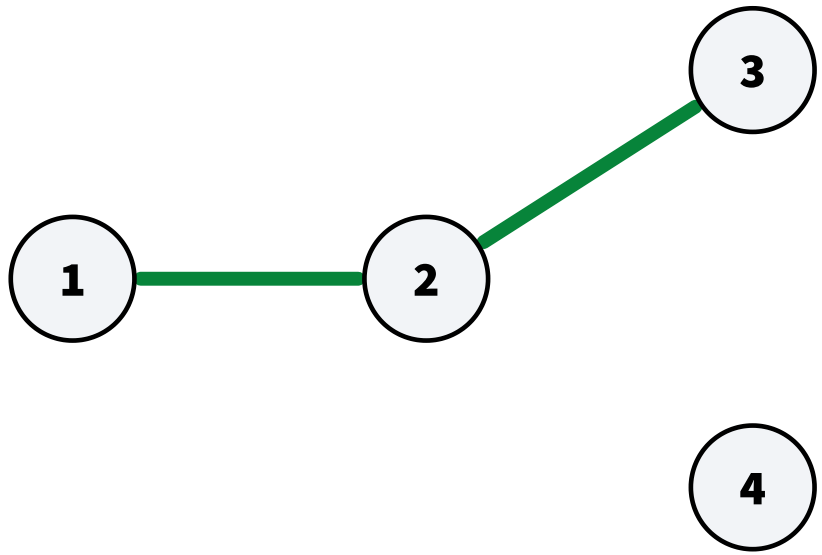
SO **One link: 1–2.**

NEXT Which pair comes next?

Friends 2 and 3 come next, and no loop forms

Pair 2–3 (0.131) joins friend 3 to friend 2.

	pairs, strongest first	
1–2	0.316	✓ link
2–3	0.131	✓ link
1–3	0.072	
2–4	0.032	
1–4	0.018	
3–4	0.008	



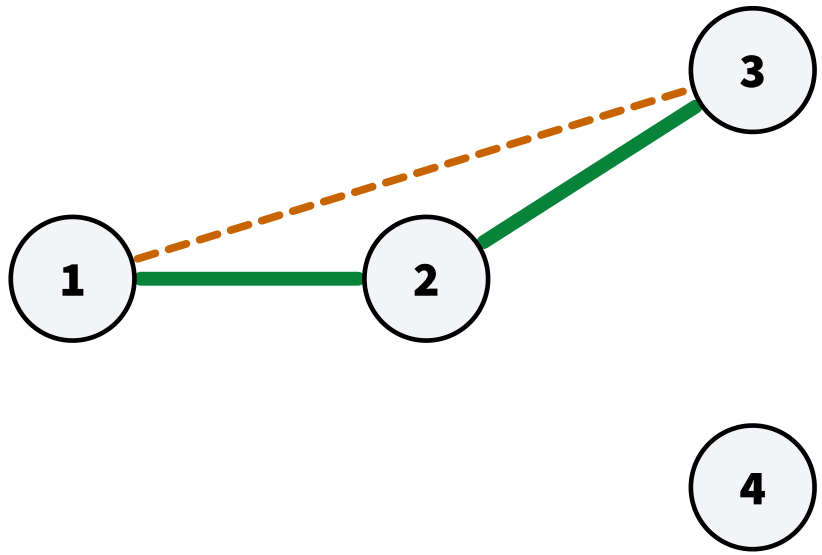
SO **Two links: 1–2, 2–3.**

NEXT Can the rule take pair 1–3?

Pair 1–3 would close a loop, so we skip it

Friends 1 and 3 are already connected through friend 2.

	pairs, strongest first	
1–2	0.316	✓ link
2–3	0.131	✓ link
1–3	0.072	✗ closes a loop
2–4	0.032	
1–4	0.018	
3–4	0.008	



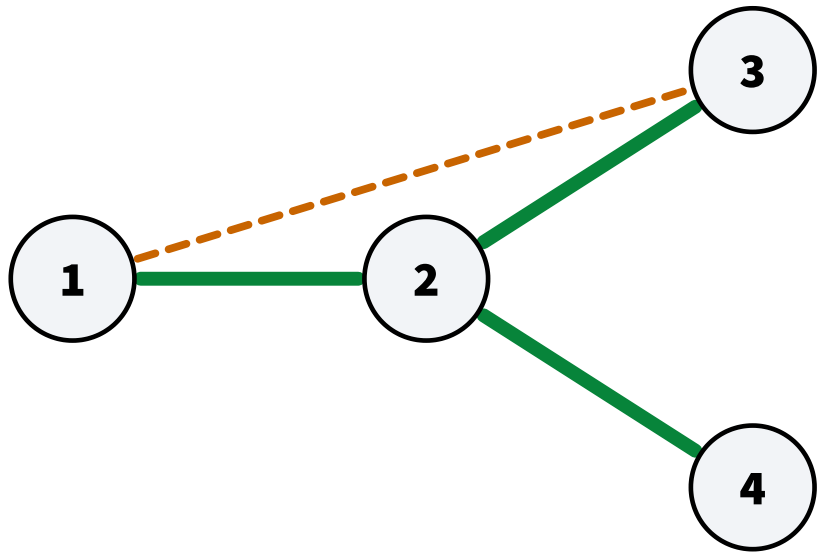
SO **The indirect pair is skipped.**

NEXT Which pair joins friend 4?

Pair 2–4 joins friend 4: three links, done

Pair 2–4 (0.032) connects friend 4. Three links: stop.

	pairs, strongest first	
1–2	0.316	✓ link
2–3	0.131	✓ link
1–3	0.072	✗ closes a loop
2–4	0.032	✓ link
1–4	0.018	not needed: three links made
3–4	0.008	not needed: three links made



SO **The rule finds the true tree: 1–2, 2–3, 2–4.**

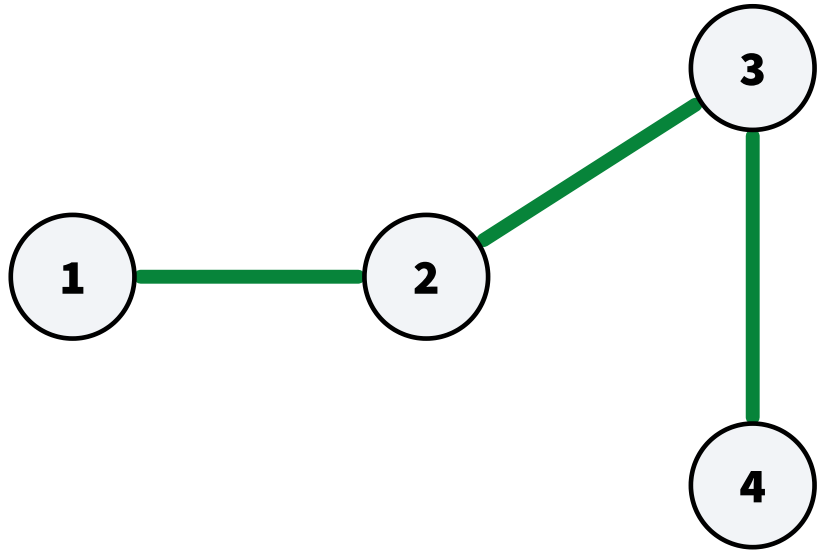
NEXT Does this work on a short notebook?

With only 20 evenings, friend 4 is linked to friend 3

In 20 random evenings, pair 3–4 (0.046) came out above the true link 2–4 (0.020) by chance.

pairs in the 20-day notebook, strongest first

2–3	**0.275**	**✓ link**
1–2	**0.205**	**✓ link**
3–4	**0.046**	**✓ link**
1–3	**0.042**	not needed: three links made
2–4	**0.020**	not needed: three links made
1–4	**0.000**	not needed: three links made



SO **The weakest link is the first to go wrong.**

NEXT How many evenings are enough?

Longer notebooks place friend 4 correctly

The same rule on notebooks of different lengths:

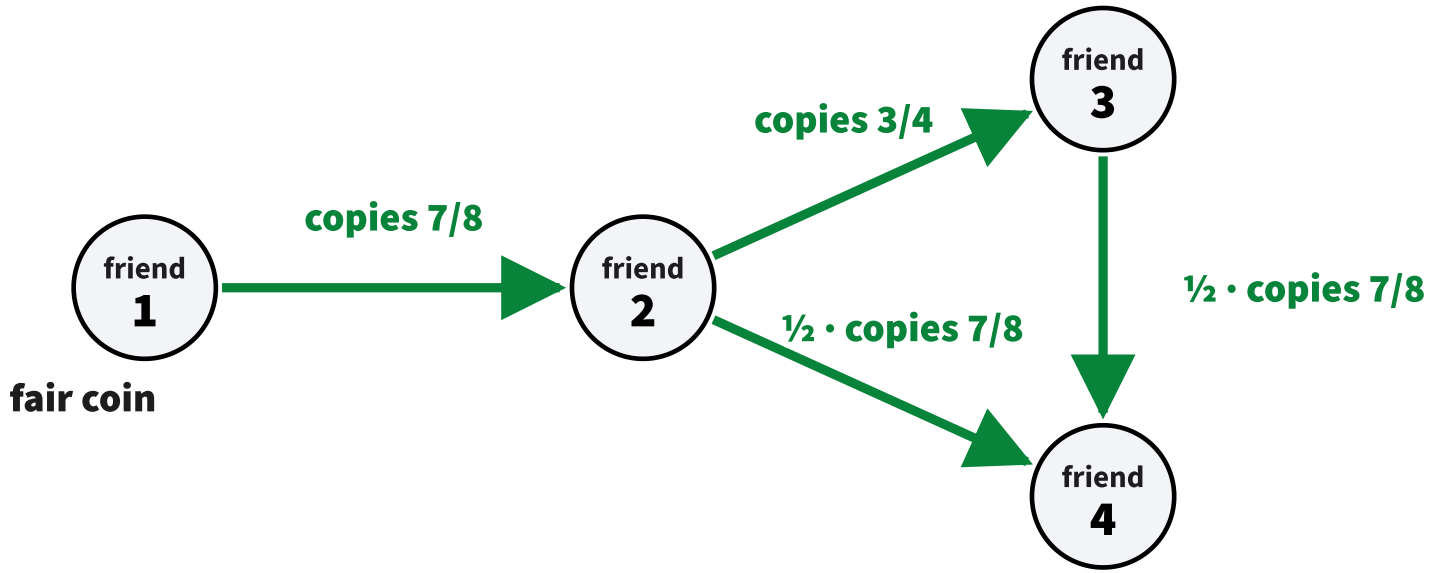
notebook	links found	friend 4
20 evenings	**1–2, 2–3, 3–4**	**wrong**
320 evenings, 1000 notebooks	**the true tree in 93%**	**most errors misplace friend 4**
512, perfect	**1–2, 2–3, 2–4**	**right**

SO **Long notebooks give the truth; short ones need checks.**

NEXT What if the true links form a loop?

If friend 4 copies friend 2 or friend 3, the links form a loop

Change one habit: a fair toss tells friend 4 whom to copy, friend 2 or friend 3; it copies with chance 7/8.

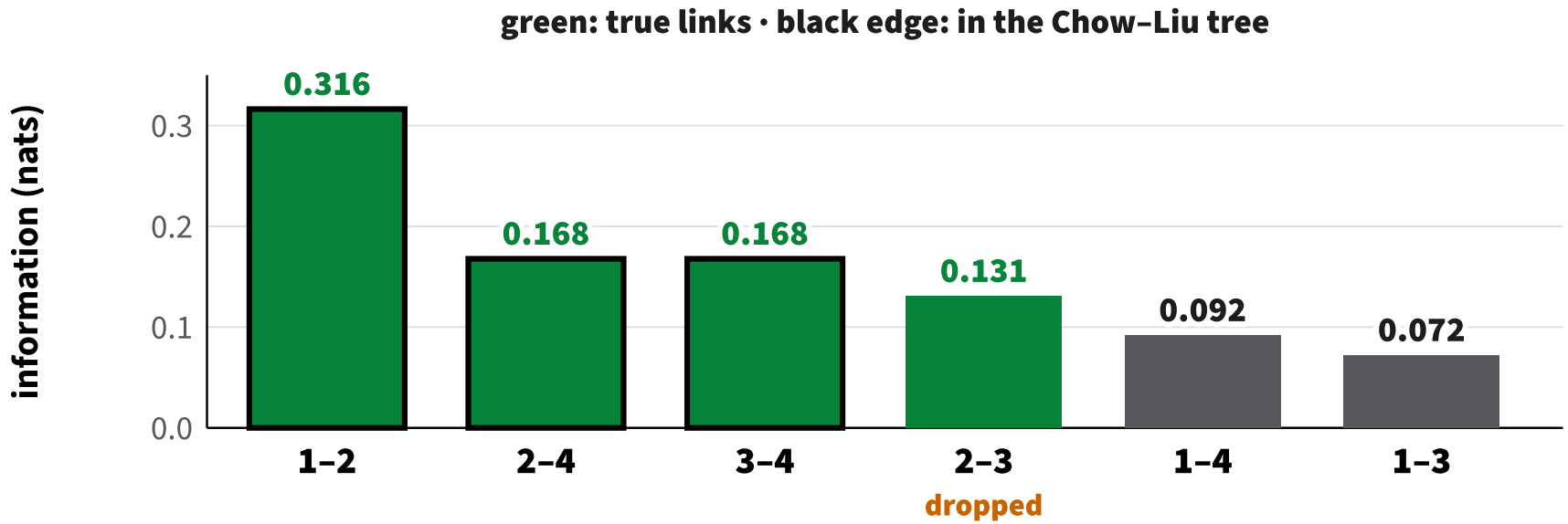


SO **Four true links: 1–2, 2–3, 2–4, 3–4.**

NEXT What does the greedy rule do now?

The tree keeps three links and drops the true link 2–3

Pairs 2–4 and 3–4 (0.168 each) enter right after 1–2; then three links are done.

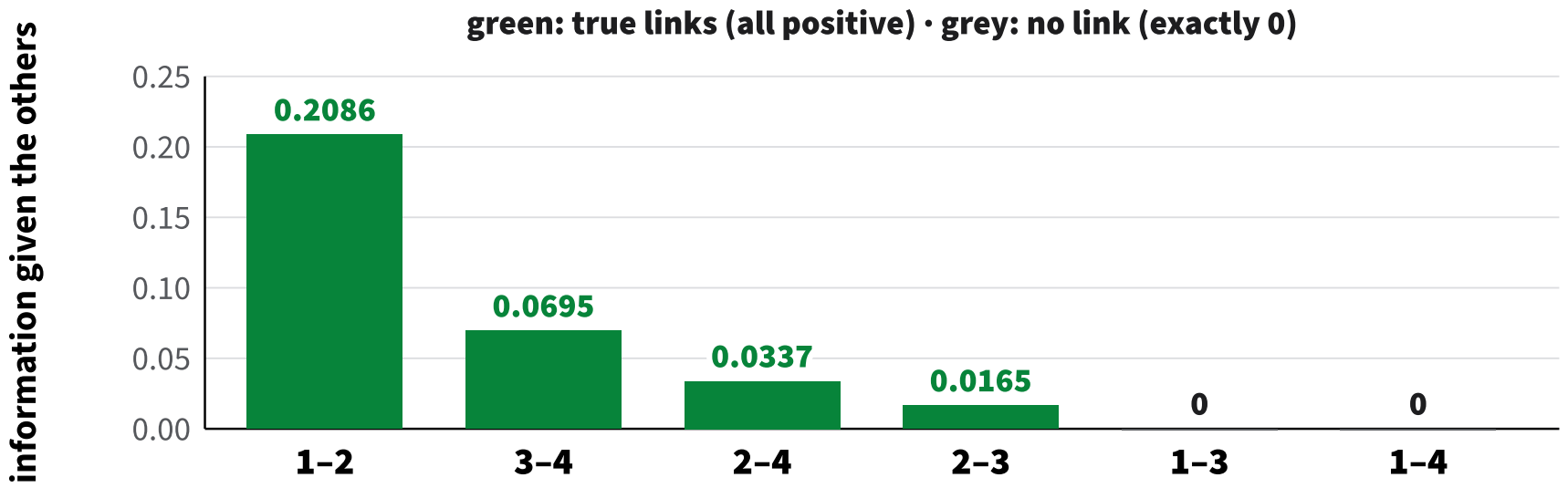


SO **A tree has three links; this loop needs four.**

NEXT Can another score find all four links?

Scoring each pair given the others finds the four links

Lecture 7's fix: score each pair **given the other friends' faces** (conditional information). Unlinked pairs drop to 0.

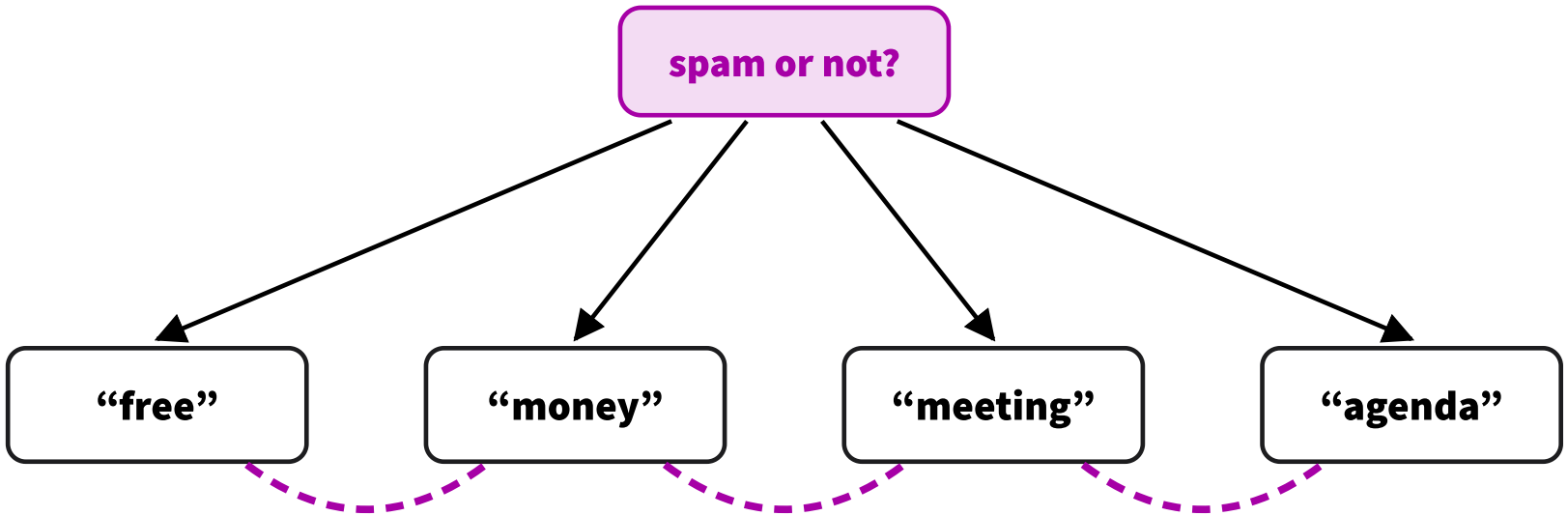


SO **Exactly the four true links remain.**

NEXT Where is learning links used today?

A spam filter can link words that appear together

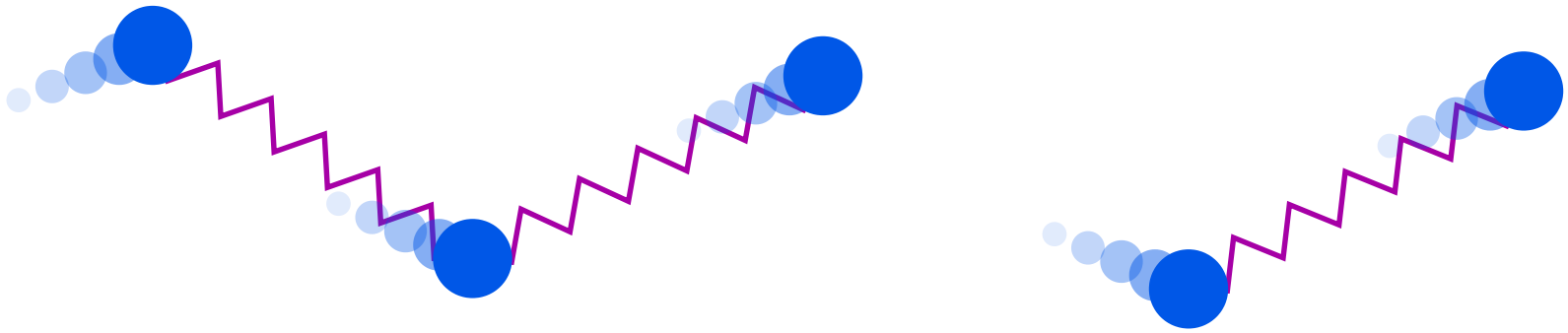
1. Friends = **words** of an email; notebook = emails marked spam or not.
2. **Naive Bayes**: words unlinked given the class, so "free" and "money" count **twice**.
3. **Tree-augmented naive Bayes** adds a Chow–Liu tree on the words, using information **given the class**.



One extra tree of links, and evidence is no longer counted twice.

A neural network learns which balls are tied by springs

1. Friends = **moving balls**; notebook = their trails.
2. Links = **hidden springs**. One network guesses them; a second predicts the motion.
3. Both train on Lecture 6's **evidence lower bound**, with links as hidden variables.

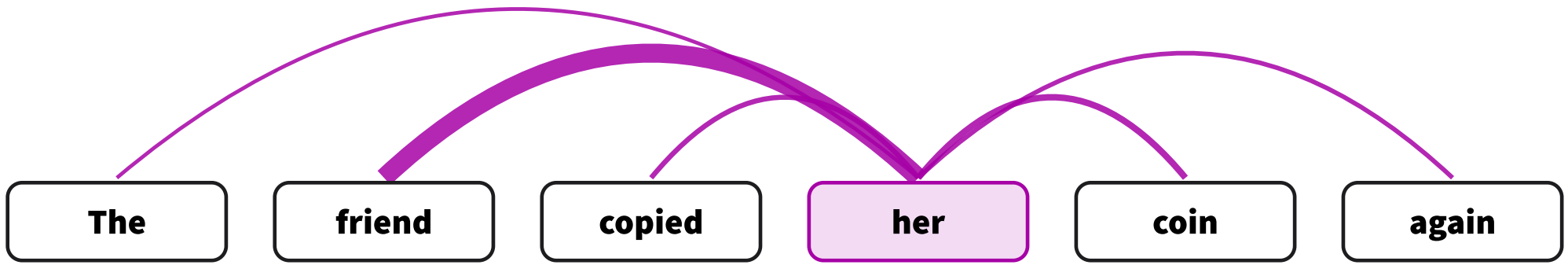


we see the trails; the springs (magenta) are hidden links a neural network learns to guess

Neural relational inference learns the links with Lecture 6's score.

Attention chooses new links for every sentence

1. Friends = **words** of one sentence.
2. Each word weights every (earlier) word: a soft map, **computed anew for each sentence**.
3. Lecture 7: one map per notebook. Attention learns **how to draw a map** for each input.



"her" links most strongly to "friend": a new map of links for each sentence (schematic)

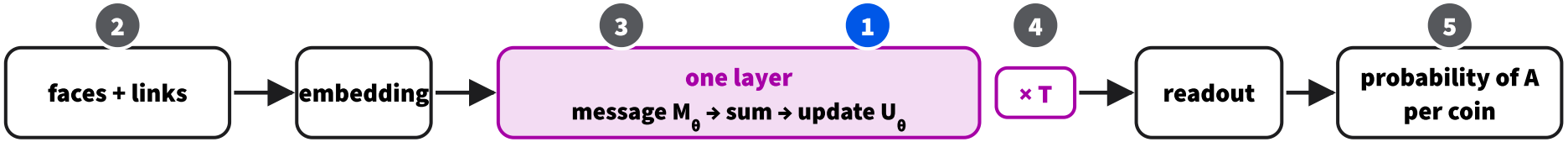
Today's topic: networks that also learn what to send along the links.

Part 1 builds a graph neural network from belief propagation

PROBLEM Belief propagation needs the factors: face probabilities and κ, the match probability (4 in 5). Often we have **only examples**.

WE DO Keep its three parts, start, message and update; let small networks **learn** them.

WHY This is the network that Parts 2 to 5 question.



1	**What is one layer, and how do we learn it?**	**because belief propagation needs the factors; we have only examples**
2	**Does one trained network fit any numbering and size?**	**because we train on short chains, then use it on new graphs**
3	**Which message and sum should a layer use?**	**because real nodes have 3 or 300 neighbours, not always 2**
4	**How many layers does a coin need?**	**because its output depends on faces several coins away**
5	**Which graphs can it never tell apart?**	**because then we know when not to trust it**

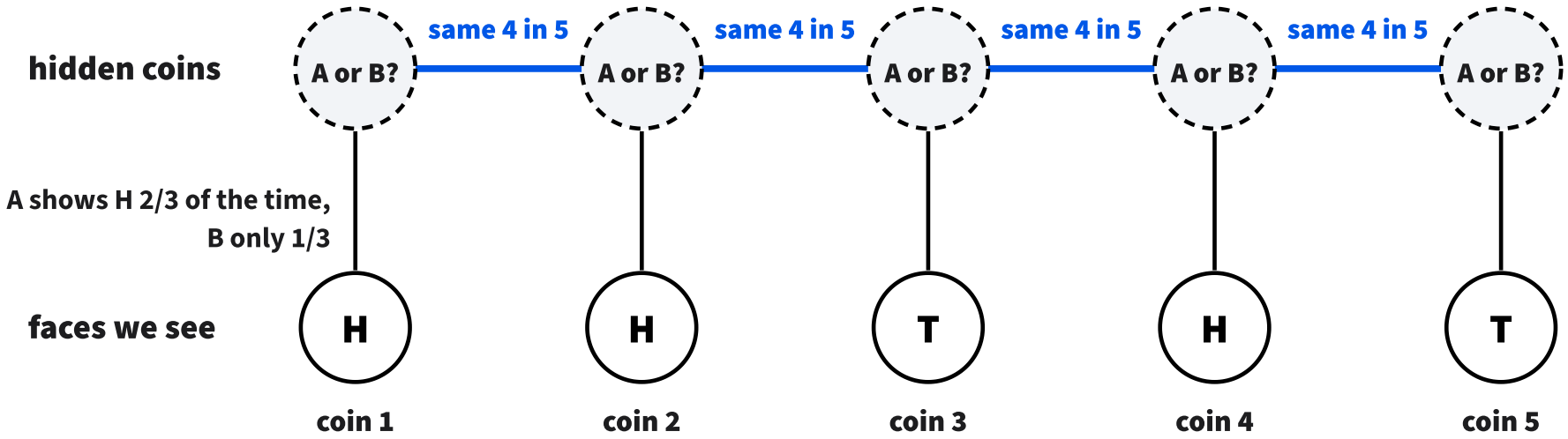
then 6: all five results on one small network · 7: one application, Sudoku, and the Transformer

SO **The result is a graph neural network; we train it and test it on our chain.**

NEXT What is our coin model?

Our model: hidden coins in a row, each showing a face

Each coin has a hidden type, A or B (z_i), and shows one **face**, H or T: the observation x_i, the only thing we see. Neighbouring coins are usually the same type, so a coin's face is also a clue about its **neighbours**.



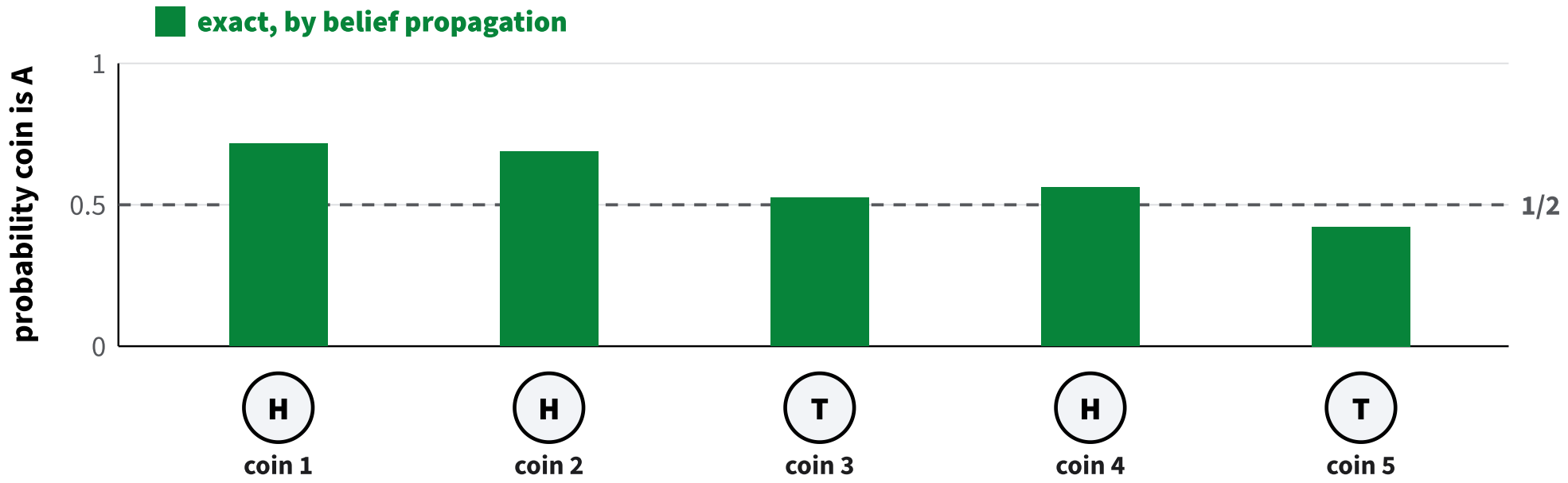
SO **To guess one coin, combine its own face with the faces around it.**

NEXT What does the best combination give on H H T H T?

Belief propagation gives each coin its probability of A

Belief propagation (Lecture 2, p. 169) combines all five faces and gives each coin its exact probability of A.

↳ Coin 1 shows H, and so does its neighbour coin 2: its probability of A, 0.715, is above the 0.667 its own face alone gives.



SO **Given only examples, can a network learn this rule?**

NEXT Where does a coin's first guess come from?

Round 0: a coin's own face gives its first guess

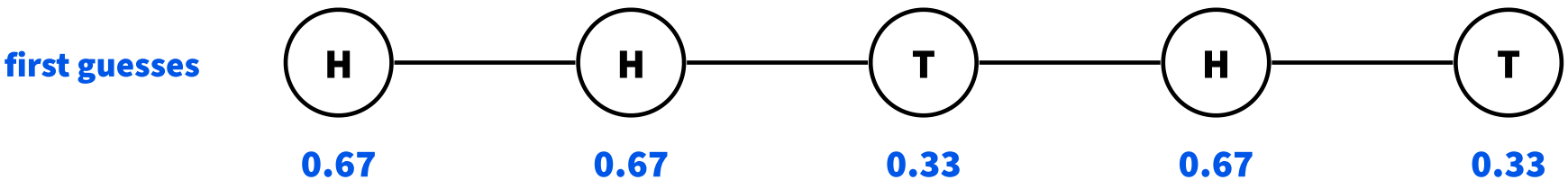
Before any message, a coin knows only its own face. A shows H twice as often as B, so an H makes A **twice as likely** as B.

H

if the coin is A: shows H with probability 2/3
if the coin is B: shows H with probability 1/3
A : B = 2 : 1, so probability of A = 2/3

T

if the coin is A: shows T with probability 1/3
if the coin is B: shows T with probability 2/3
A : B = 1 : 2, so probability of A = 1/3



each coin's probability of A, from its own face only

SO **H starts at 0.67, T at 0.33.**

NEXT How does a neighbour's face change the guess?

Round 1: a neighbour's face is a second, weaker clue

Coin 1 now hears coin 2's face. Coin 2 matches coin 1 only 4 times in 5, so its H is a **weaker** clue; the two clues multiply.



if coin 1 is A: coin 2 is A 4 times in 5,
so coin 2 shows H with probability 0.6

if coin 1 is B: coin 2 shows H with probability 0.4

(4/5 · 2/3 + 1/5 · 1/3 = 0.6; 1/5 · 2/3 + 4/5 · 1/3 = 0.4)

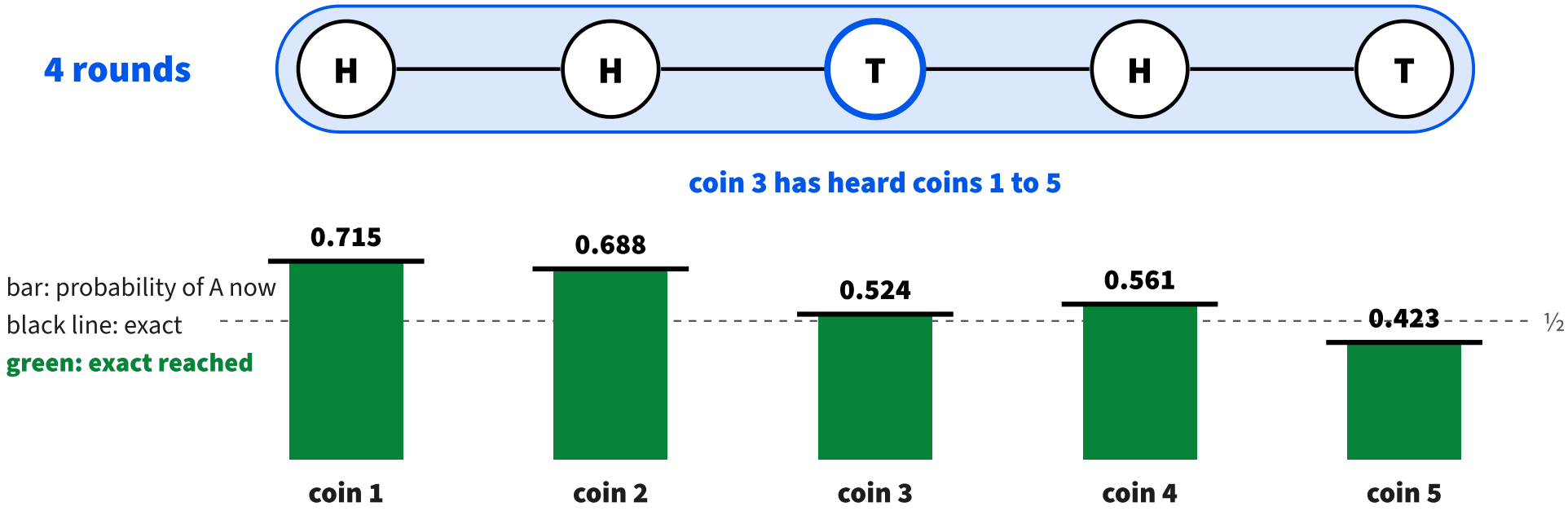
clue 1, coin 1's own face H	**A : B = 2 : 1**
clue 2, coin 2's face H	**A : B = 0.6 : 0.4 = 1.5 : 1**
both clues together	**A : B = 2 · 1.5 = 3 : 1**
coin 1's probability of A after one round	**3/4 = 0.75**

SO **After one round, coin 1's probability is 0.75.**

NEXT What happens in later rounds?

Each round, every coin hears one coin further away

Each round repeats round 1: a neighbour's message also carries what that neighbour heard the round before, so every coin hears **one more coin on each side**.

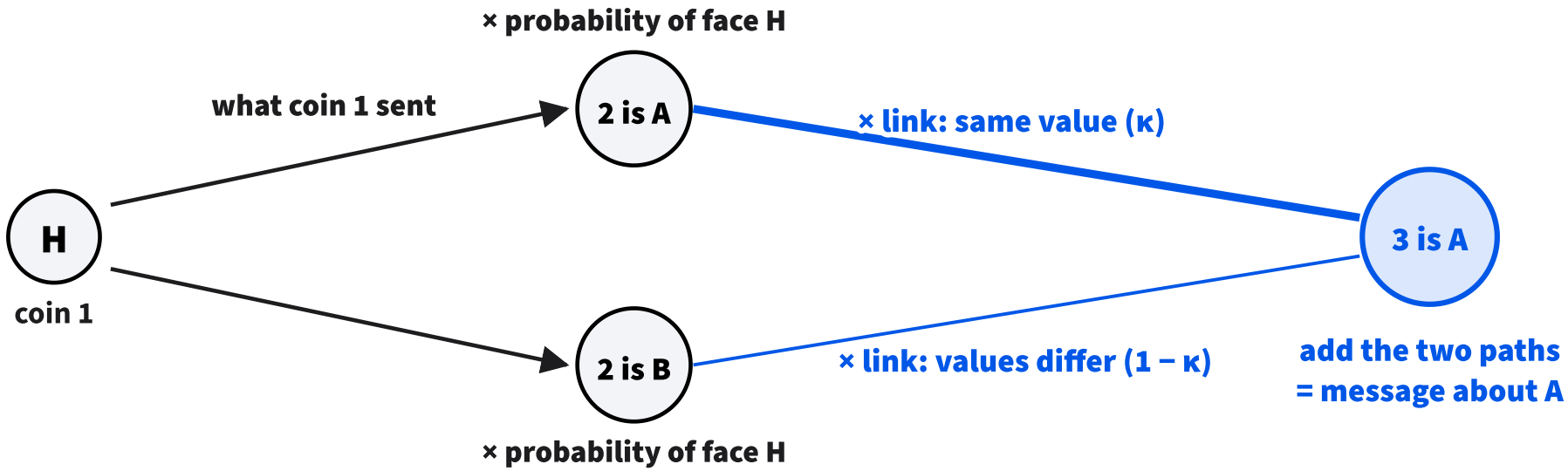


SO **Coin 3 hears every coin, and is exact, after two rounds; every coin after four.**

NEXT What does one message compute?

Coin 2's message to coin 3 adds two paths

Coin 2 may be A or B, so its message about “coin 3 is A” adds two paths.



$$\mu_{2\to 3}(\mathrm{A}) = \sum_{z_2 \in \{\mathrm{A},\mathrm{B}\}} \psi(\mathrm{A}, z_2) \cdot \varphi_2(z_2) \cdot \mu_{1\to 2}(z_2)$$

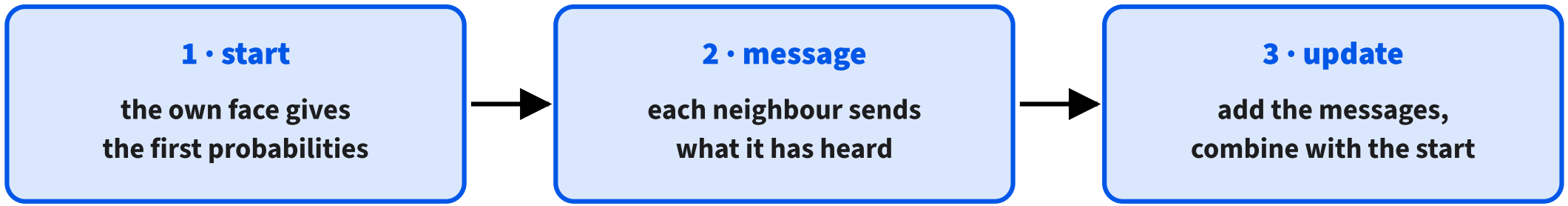
z_2: coin 2's hidden value; ψ: κ if equal, $1-\kappa$ if not; φ_2: probability of coin 2's face; $\mu_{1\to 2}$: what coin 1 sent.

SO **A message sums over the sender's values.**

NEXT Which parts make up one round?

Belief propagation has three parts: start, message, update

Look back at rounds 0 and 1 (p. 29): every round on any graph uses the same three parts.



on the chain H H T H T:

coin 3 shows T:
A 0.33, B 0.67

coins 2 and 4 tell coin 3
about A and about B

coin 3 gets its
new probability of A

SO **All three parts are formulas fixed by the face probabilities and κ (p. 138).**

NEXT If we do not know κ, which parts can a network learn?

A graph neural network learns each part instead

Keep the rounds and the links. Replace each **fixed formula** by a simple part with **parameters** to learn. Ours has only two parameters.

	belief propagation: fixed by the model		graph neural network: two parameters
start	probabilities from the face: H gives A 0.67	→	**embedding: score $u \cdot x_i$ (x_i = +1 for H, −1 for T)**
a coin holds	2 numbers: its probability of A and of B	→	**one score s_i: above 0 leans to A, below 0 to B**
message from j	a formula from κ and the face probabilities	→	**$M_\theta(s_i, s_j) = w \cdot s_j$**
update	add the messages, combine with the start	→	**$U_\theta(s_i, a_i) = u \cdot x_i + a_i$ (a_i: sum of messages)**
output	the probability of A itself	→	**readout: probability of A = 1 / (1 + e^(−s_i))**

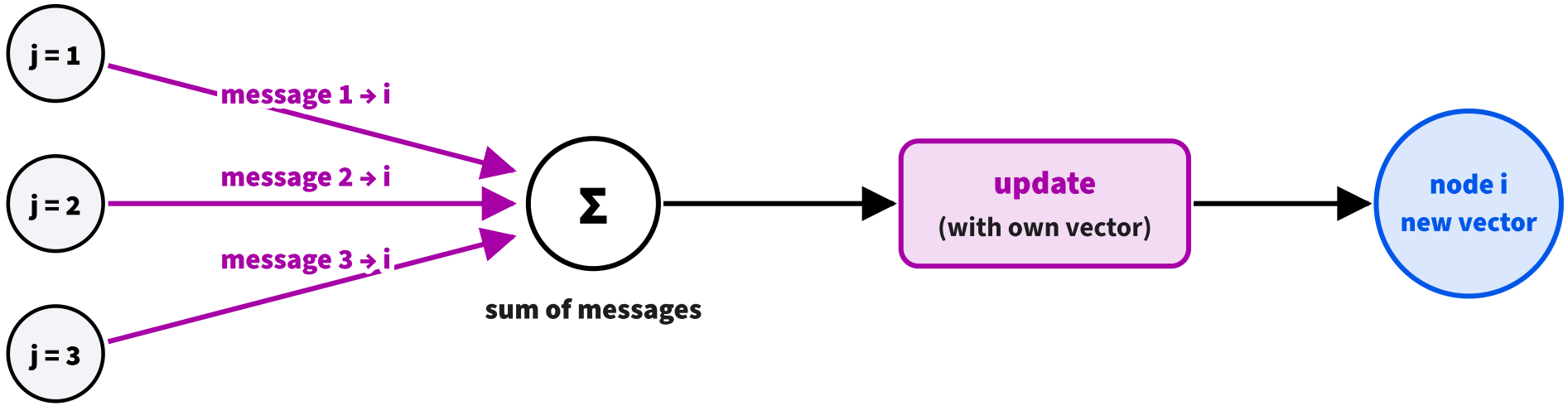
parameters, learned from examples: θ = (u, w), just two

SO **M_θ and U_θ are simple formulas; only $\theta = (u, w)$ is learned. u: how much a coin trusts its own face; w: how much it trusts each neighbour. The score (textbook name: logit) is turned into a probability by $1/(1 + e^{-s})$; score 0 means ½.**

NEXT How does one round look as a general formula?

One round of the network is a message-passing layer

In general a coin holds a vector h_i and M_θ, U_θ are small networks; ours: $h_i = s_i, M_\theta = w \cdot s_j, U_\theta = u \cdot x_i + a_i$.



Message-passing layer: $h_i' = U_\theta\Big(h_i,\ \underbrace{\textstyle\sum_{j \in N(i)} M_\theta(h_i, h_j)}_{a_i,\ \text{the sum}}\Big)$

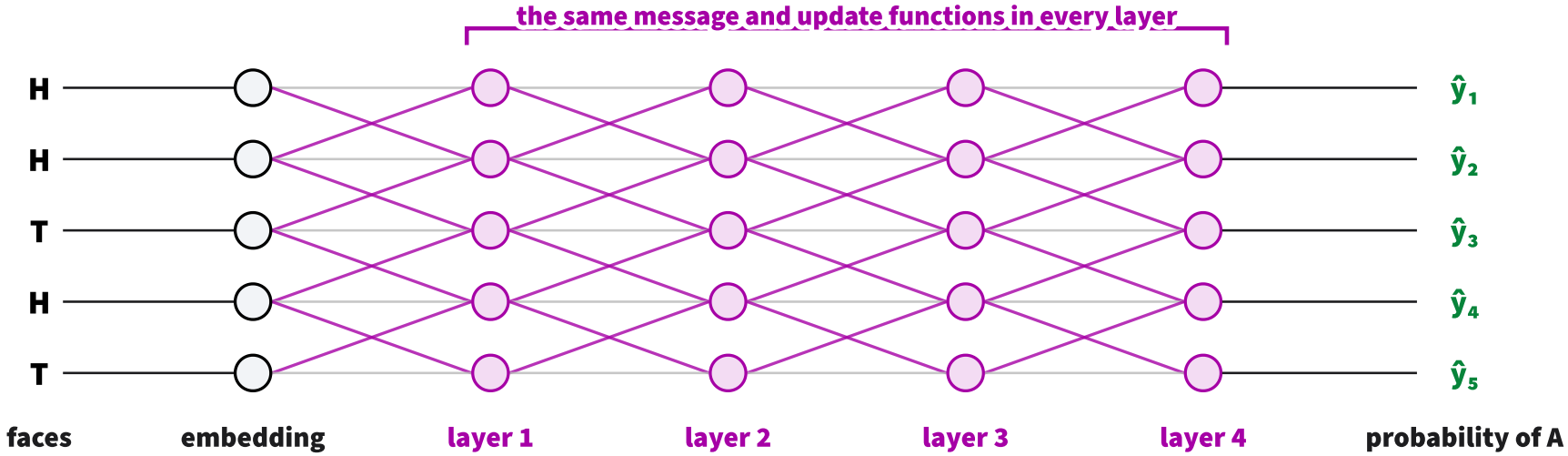
h_i: node i's vector; h_i': its new one; $N(i)$: its neighbours; M_θ, U_θ: small networks with parameters θ.

SO **Stacked layers = a graph neural network (message-passing neural network, MPNN). Belief propagation: both parts fixed by the coin model (p. 123).**

NEXT How do start, rounds and readout fit together?

A graph neural network: start, T layers, readout

Input: the faces **and the links**. Embedding: each coin's vector from its face; T layers pass messages along the links; readout: $\hat{y}_i$, the probability that coin i is A.



Graph neural network: $x_i \rightarrow h_i^{(0)} \rightarrow T \text{ layers} \rightarrow h_i^{(T)} \rightarrow \hat{y}_i$

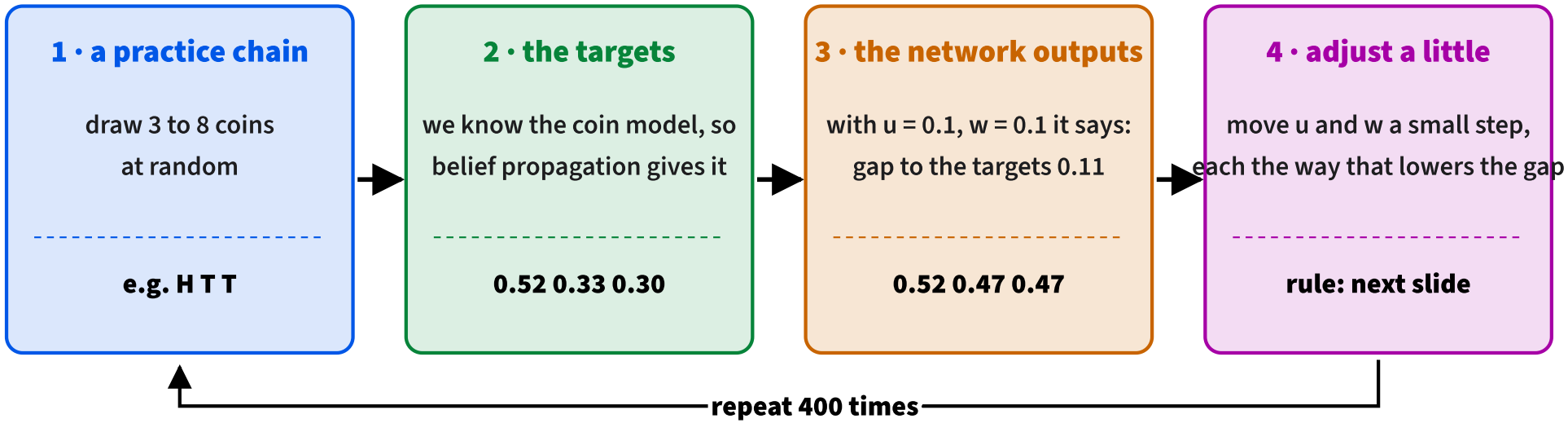
x_i: coin i's face; $h_i^{(t)}$: its vector after layer t; $\hat{y}_i$: the network's probability of A.

SO **Ours: 3 layers, $\theta = (u, w)$; the links are given, not learned. A bigger network: 16-number vectors, 1 649 parameters (p. 114).**

NEXT How do we choose u and w from examples?

Training: practise on chains whose targets we know

Practice chains we make ourselves: belief propagation writes the **targets** y_i, the exact probabilities. The **gap** is the mean absolute error between outputs $\hat{y}_i$ and targets.



$\text{gap}(u, w) = \frac{1}{n}\sum_{i=1}^{n} |\hat{y}_i - y_i|$, averaged over the practice chains

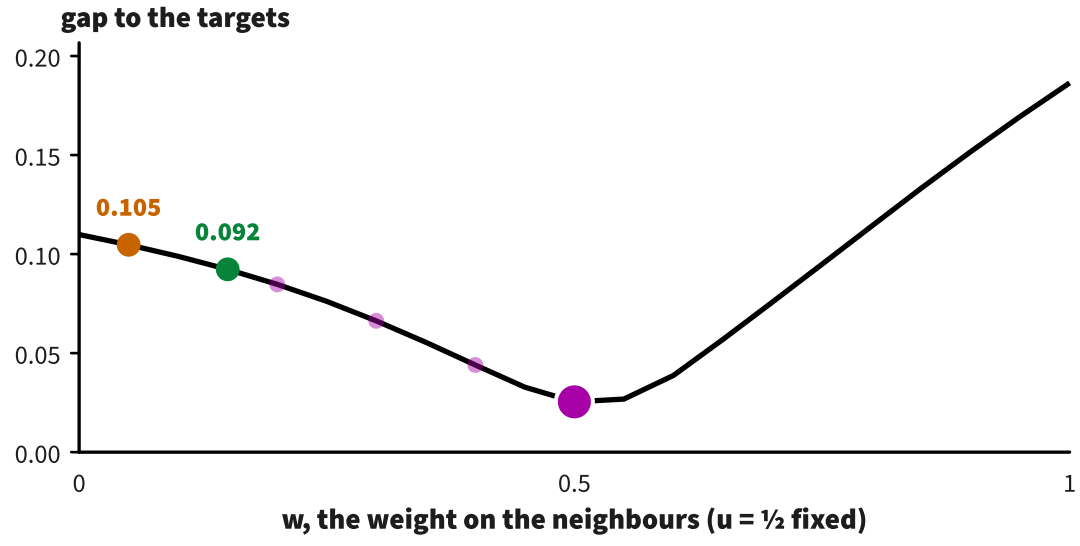
H T T with $u = w = 0.1$: $\frac{1}{3}\left(|0.52 - 0.52| + |0.48 - 0.33| + |0.47 - 0.30|\right) = 0.11$.

SO **Training = find u, w with the smallest gap; then the network runs on new chains without targets.**

NEXT How does the network know which way to move u and w?

The rule: step the way that lowers the gap

Fix $u = \frac{1}{2}$, move only w; the curve is the gap on 200 practice chains. The network never sees the curve: it **tries** w a little up and a little down.



repeat until the bottom:

w = 0.5, gap 0.025

training does this for

u and w together

$$w \leftarrow w - \eta \cdot \frac{\text{gap}(w+\delta) - \text{gap}(w-\delta)}{2\delta}, \quad \text{and the same for } u$$

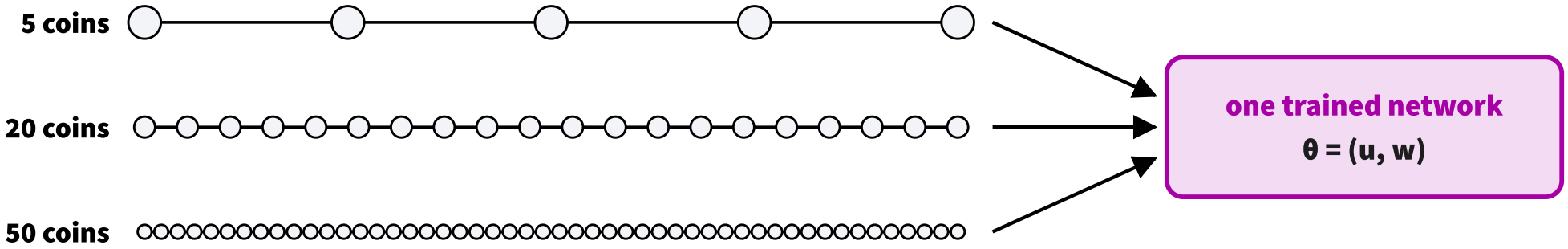
δ: trial step (0.0001); the fraction is the **slope**; η: learning rate (0.5). Uphill, w moves down; downhill, up.

SO **Each parameter walks downhill; 400 steps on u and w together are the whole training.**

NEXT Can one trained network work on chains of any length?

Next: one trained network works on chains of any length

The network trained on 3 to 8 coins has **two parameters** $\theta = (u, w)$. Can we feed it 50 coins without changing them? Look back at one layer (p. 35): every coin uses the **same** u and w, and the sum takes any number of neighbours.



A chain of 50 coins needs how many sets of numbers θ?

50, one per coin | one, shared by all coins | it cannot run

Answer. One: each coin applies the same u and w, so θ never depends on the length. Whether the outputs stay **accurate** is a separate question (p. 88).

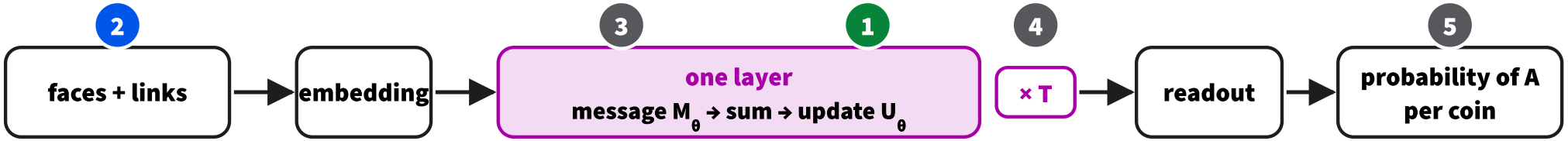
Next: sharing the same functions also gives symmetry (Part 2).

Part 2 asks: does one trained network fit every graph?

PROBLEM We trained on chains of 3 to 8 coins, numbered left to right.

WE DO Renumber the coins and change the size; prove what must happen.

WHY Otherwise every new graph would need **new training**.



1	What is one layer, and how do we learn it?	✓ answered
2	**Does one trained network fit any numbering and size?**	**because we train on short chains, then use it on new graphs**
3	Which message and sum should a layer use?	because real nodes have 3 or 300 neighbours, not always 2
4	How many layers does a coin need?	because its output depends on faces several coins away
5	Which graphs can it never tell apart?	because then we know when not to trust it

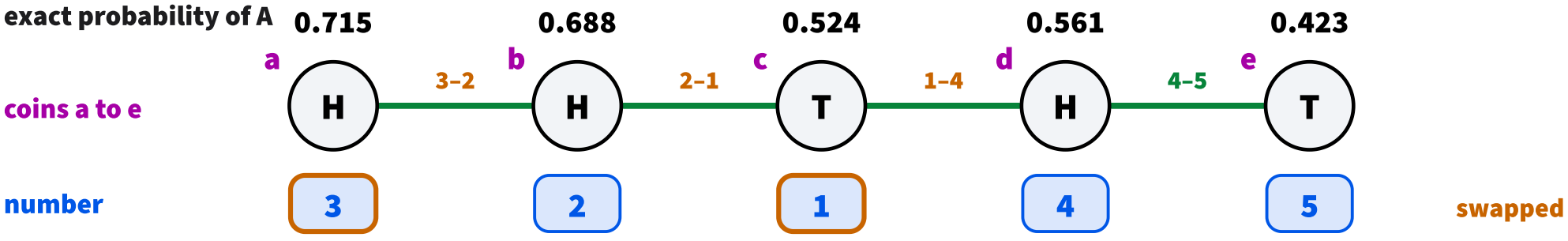
then 6: all five results on one small network · 7: one application, Sudoku, and the Transformer

SO **A layer built from a sum lets one θ serve every numbering and size.**

NEXT What happens when we renumber the coins?

Renumbering the coins must not change their outputs

No retraining. Name the coins a to e. **Swap the numbers** of a and c (1 and 3) and run the trained network again: coins, faces and links stay, so each probability must stay.



the network is given: **faces by number T H H H T, links 3–2, 2–1, 1–4, 4–5: the same links, renamed**

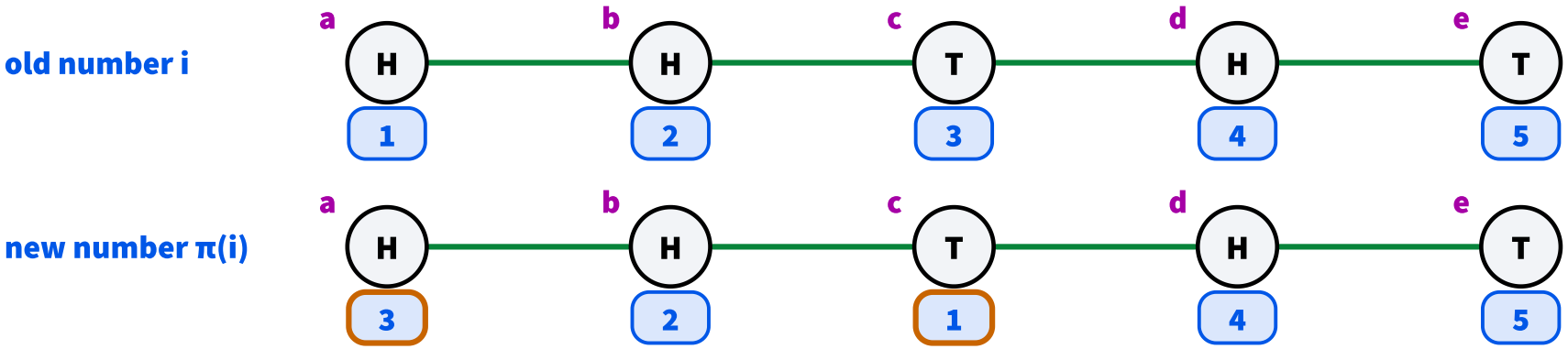
After the swap, which coins get a different probability of A?

no coin | coins a and c | every coin

Answer. No coin. Each update reads the **linked** coins, and the links are renamed with the coins (3–4 is now 1–4). The theorem ahead proves this for every graph.

A relabelling gives nodes new numbers, nothing else

Our swap (p. 41), written as a rule: each coin keeps its face and its links, only its number changes.



π(1) = 3, π(2) = 2, π(3) = 1, π(4) = 4, π(5) = 5: coin a gets 3, coin c gets 1

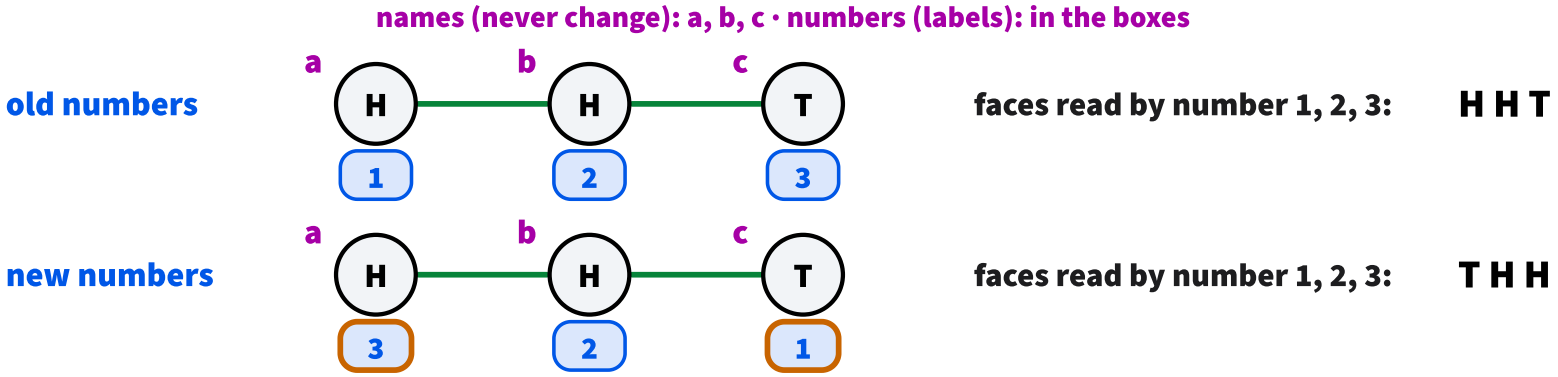
Definition. A **relabelling** π gives old node i the new number $\pi(i)$.

SO **Renumbering moves each node's face and its links together; matrix form (p. 139).**

NEXT Which outputs should stay, and which should move?

Invariant outputs stay; equivariant outputs follow their coins

Renumber the seats in a class: the **class average** stays; each student **keeps their own mark**. Below: coins a, b, c; swap the numbers 1 and 3.



one output for the whole chain:	**number of heads = 2, before and after**	**invariant**
one output per coin:	**number 3 now names coin a, so it reads H**	**equivariant**
	coins a, b, c still show H, H, T	

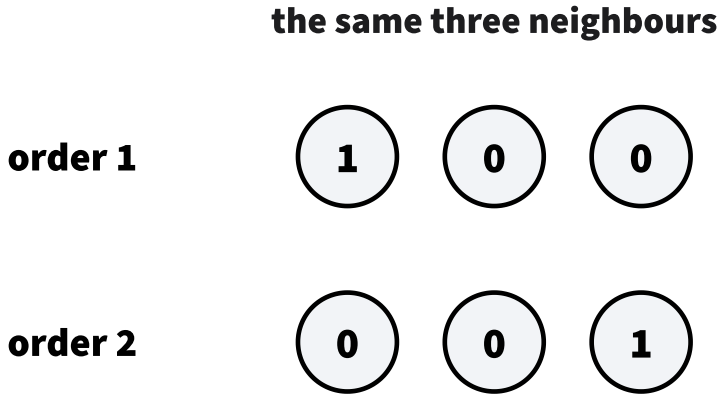
Definition. A graph-level output is **invariant** if renumbering leaves it unchanged. Node outputs are **equivariant** if each node keeps its output under its new number: $\hat{y}'_{\pi(i)} = \hat{y}_i$ (coin a: $\hat{y}'_3 = \hat{y}_1$).

SO **Our probabilities of A, one per coin, should be equivariant, like the faces.**

NEXT Which ways of combining neighbours ignore their order?

Sum, mean and maximum ignore the order of neighbours

Neighbours come in **no natural order**: combine the same three values in two orders.



combine by	order 1	order 2	same?
sum	1	1	yes
mean	1/3	1/3	yes
max	1	1	yes
the first one	1	0	no
stack in order	(1, 0, 0)	(0, 0, 1)	no

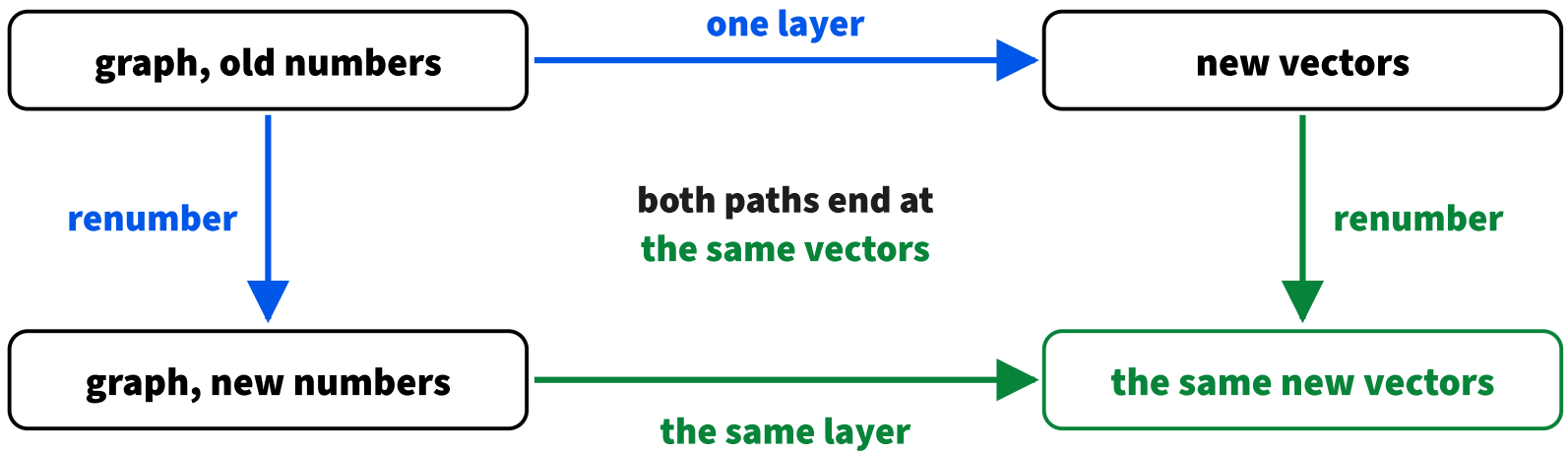
Lemma. The sum, mean and maximum do not depend on the order.

SO **A new order changes neither the values nor their count.**

NEXT Does a whole layer then ignore the numbering?

Theorem: a message-passing layer moves with the nodes

Renumber, then apply a layer (p. 35), or the reverse.



Theorem (equivariance). For every message and update network, both paths give the same vectors (for us: the same scores).

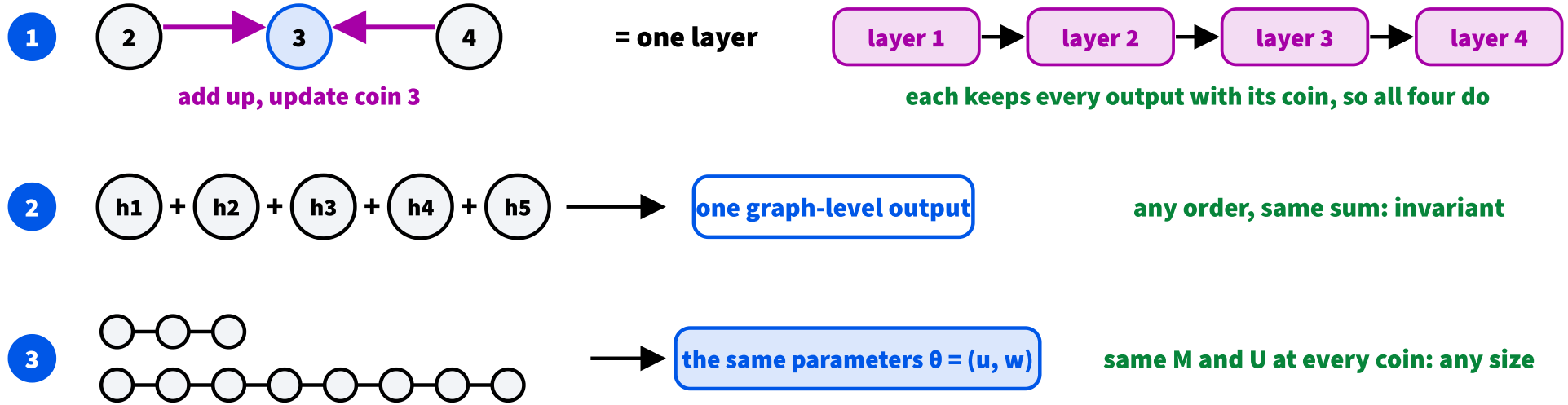
In words: a renumbered node keeps its neighbours, so its sum (p. 140).

SO **This comes from the layer's shape, not training.**

NEXT What follows for many layers and graph-level outputs?

Three consequences: many layers, one graph-level output, any size

1. **Many layers.** Each layer keeps every output with its coin (p. 45), so the whole network is **equivariant**.
2. **One graph-level output.** Add up all final node values; a sum ignores order (p. 44), so it is **invariant**.
3. **Any size.** The same M_θ, U_θ at every coin, so one θ fits every length (p. 39).

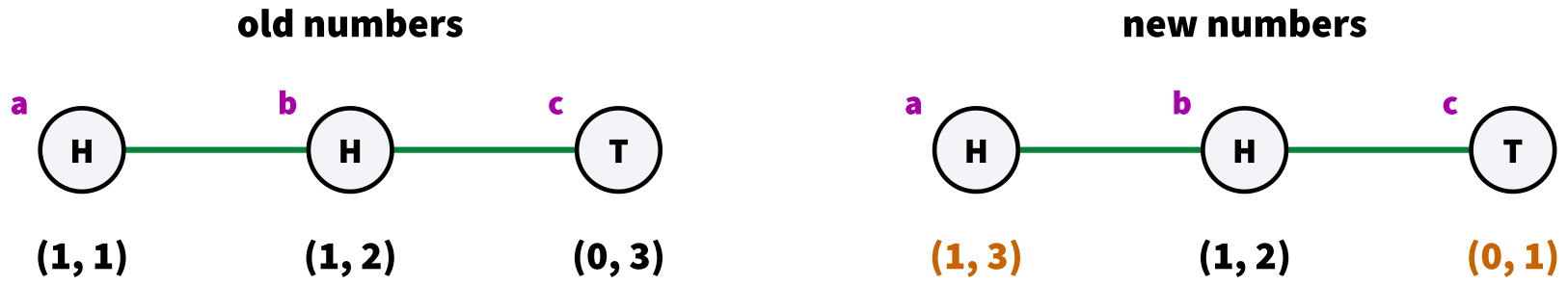


SO **One $\theta = (u, w)$, trained once, serves every graph and every numbering.**

NEXT Can extra node features break this symmetry?

Add each node's number as a feature: what happens?

Some tasks need a node's place. Give each coin the input (face, its number) and a third parameter v: start $s_i = u \cdot x_i + v \cdot (\text{number})$. Then swap the numbers 1 and 3 again.



feature of a coin: (face, number)

coins a and c: the same coin, another feature

With the feature (face, i), do the outputs still move with the nodes?

yes, the layer is unchanged | no | only on chains

Answer. No. Coin a's input changes from $(+1, 1)$ to $(+1, 3)$, so its start moves from $u + v$ to $u + 3v$. With the face alone, its input is $+1$ under both numberings (p. 41).

SO **Use numbers as features only when order matters (word positions, Lecture 12).**

NEXT Which message and update functions do people use?

Next: which message and update functions to choose?

Symmetry fixes the **shape**; it leaves the message and update **open**.

One node has 3 neighbours, another 300. Which order-free combination keeps both on one scale?

their sum | their mean | the sum of their squares

Answer. Their mean: it divides by the count, while a sum of 300 similar messages is about 100 times a sum of 3.

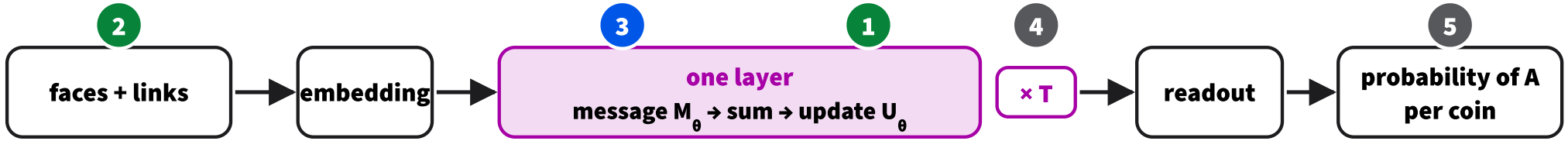
Next: three classic ways to weigh neighbours.

Part 3 changes the message: how to weigh each neighbour

PROBLEM A coin has at most 2 neighbours; real nodes have 3 or 300, not all useful.

WE DO Compare three classic choices on Lecture 7's friends' tree.

WHY Our message was $w \cdot s_j$, the same weight for every neighbour (p. 34).



1	What is one layer, and how do we learn it?	✓ answered
2	Does one trained network fit any numbering and size?	✓ answered
3	**Which message and sum should a layer use?**	because real nodes have 3 or 300 neighbours, not always 2
4	How many layers does a coin need?	because its output depends on faces several coins away
5	Which graphs can it never tell apart?	because then we know when not to trust it

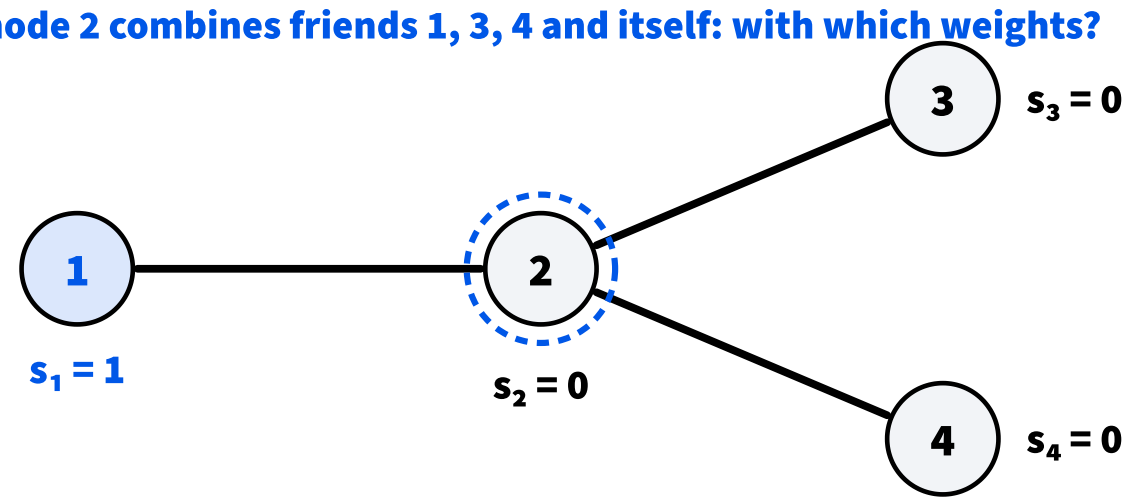
then 6: all five results on one small network · 7: one application, Sudoku, and the Transformer

SO **Each classic network is one way to fill the same layer.**

NEXT How should node 2 weigh its three friends?

How should node 2 weigh its three friends?

Lecture 7's friends' tree (Lecture 7, p. 18): node 2 has friends 1, 3 and 4; only friend 1 carries a signal. Each node holds one score s_i (the h_i of (p. 35)); s_i' is its score after one layer.

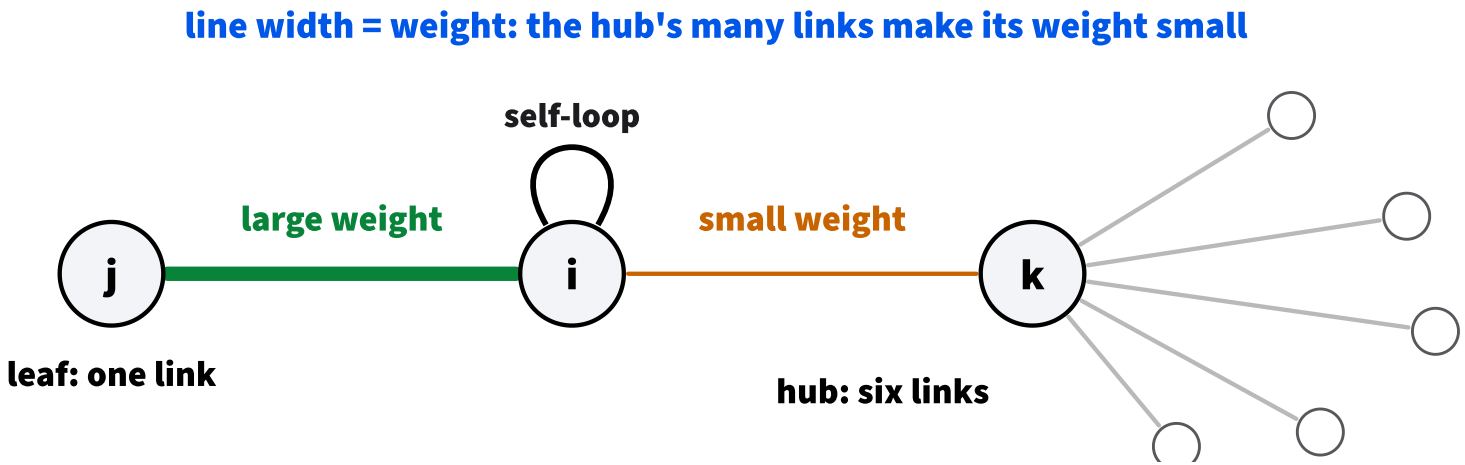


SO **Three classic rules: average by degree, sample and average, learn the weights.**

NEXT How should an average treat a busy neighbour?

Weigh each neighbour by the degrees at both ends

A plain sum **grows with the links**, so busy neighbours should count less.



$$s'_i = \mathrm{relu}\Big(w \cdot \sum_{j \in N(i) \cup \{i\}} \frac{s_j}{\sqrt{\tilde{d}_i \cdot \tilde{d}_j}}\Big)$$

s_j: scores now; s'_i: new score; $\tilde{d}_i$: links of i plus one. **Parameter:** w. $\mathrm{relu}(u) = \max(u, 0)$. Message $w \cdot s_j / \sqrt{\tilde{d}_i \tilde{d}_j}$, node i included; update: relu of the sum (p. 35).

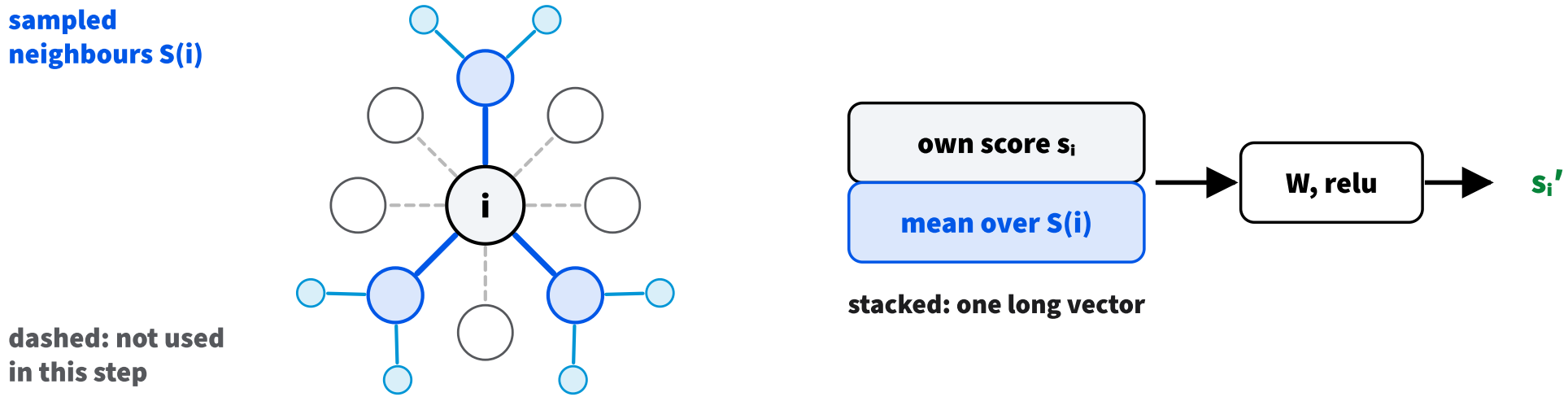
Node 2 of the tree ($\tilde{d}_2 = 4, \tilde{d}_1 = \tilde{d}_3 = \tilde{d}_4 = 2$): $s'_2 = \mathrm{relu}\big(w \cdot (\frac{s_1}{\sqrt{8}} + \frac{s_3}{\sqrt{8}} + \frac{s_4}{\sqrt{8}} + \frac{s_2}{\sqrt{16}})\big)$.

SO **This is a graph convolutional network (GCN); why this division (p. 141).**

NEXT What if a node has millions of neighbours?

Sample a few neighbours and keep the own vector apart

A popular account has millions of neighbours; average only a **random few**, $S(i)$.



$$s_i' = \mathrm{relu}\Big(w_1 \cdot s_i + w_2 \cdot \mathrm{mean}_{j \in S(i)}\, s_j\Big)$$

$S(i)$: a random few of the neighbours. **Parameters:** w_1 (own score), w_2 (neighbours). Message $s_j/|S(i)|$, so the sum is the mean; update $\mathrm{relu}(w_1 s_i + w_2 a_i)$.

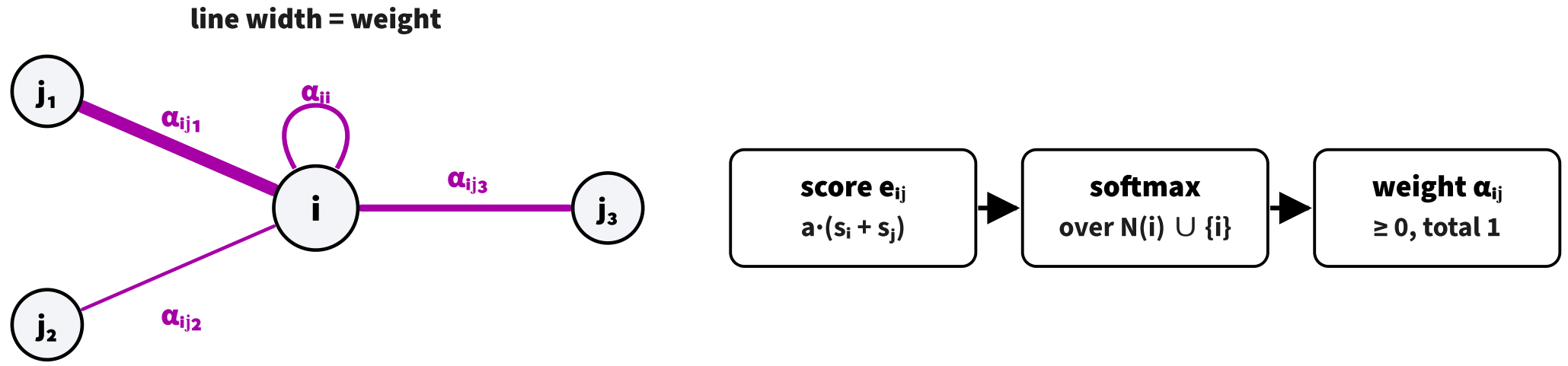
Node 2, sample = all three friends: $s_2' = \mathrm{relu}\big(w_1 \cdot s_2 + w_2 \cdot \frac{s_1+s_3+s_4}{3}\big)$.

SO **This is GraphSAGE (sample and aggregate): the work per node stays small.**

NEXT Can a node learn which neighbours matter more?

Learn a weight for each neighbour from both scores

Score each neighbour from both scores; scores become weights adding to 1. Simplified from the original, which also bends the score with a leaky relu (p. 142).



$$e_{ij} = a \cdot (s_i + s_j), \qquad \alpha_{ij} = \frac{e^{e_{ij}}}{\sum_{k \in N(i) \cup \{i\}} e^{e_{ik}}}, \qquad s'_i = \mathrm{relu}\Big(w \cdot \sum_{j \in N(i) \cup \{i\}} \alpha_{ij} \cdot s_j\Big)$$

Parameters: a (scoring) and w. The weights α_{ij} are positive and add to 1. Message $\alpha_{ij} \cdot w \cdot s_j$, node i included; update: relu of the sum.

Node 2: $s'_2 = \mathrm{relu}\big(w \cdot (\alpha_{21} s_1 + \alpha_{23} s_3 + \alpha_{24} s_4 + \alpha_{22} s_2)\big)$; with $a = 1$ and $s_1 = 1$, the rest 0: $\alpha_{21} = e/(e+3) = 0.475$.

SO **This is a graph attention network (GAT); on words it becomes attention (Lecture 12).**

NEXT Are all three the same template?

All three fill the same template in different ways

All three keep one score per node and have one or two parameters. With a vector per node, each parameter becomes a matrix W (p. 143).

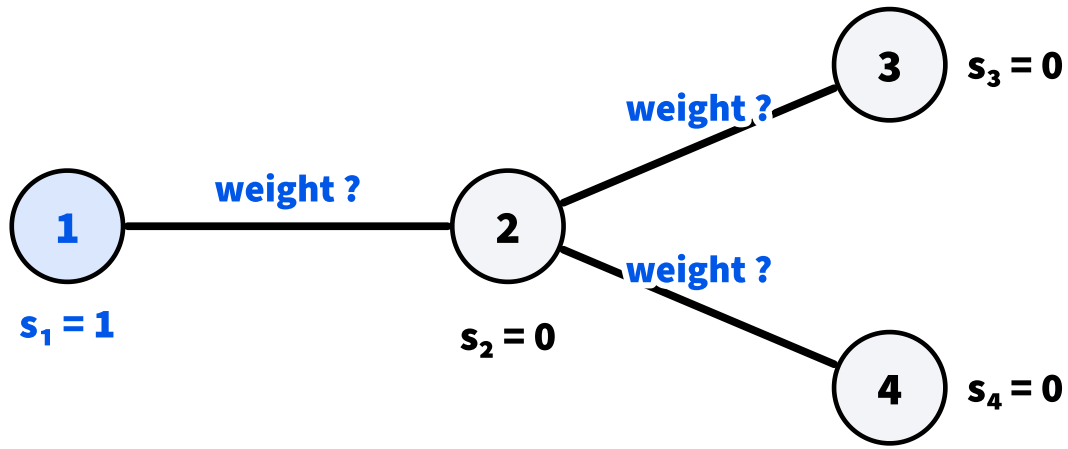
network	message from a neighbour	combination
graph convolutional	its vector, scaled by both degrees	sum, itself included
GraphSAGE	its vector	mean of a sample
graph attention	its vector, times a learned weight	sum, itself included

SO **Each combination ignores order, so all three move with the nodes (p. 45).**

NEXT Which of them can prefer one friend?

Which network can prefer friend 1 over friends 3 and 4?

Friends 1, 3 and 4 each have one link; only friend 1 carries a signal.



Which network can give friend 1 the larger weight?

graph convolutional | GraphSAGE | graph attention

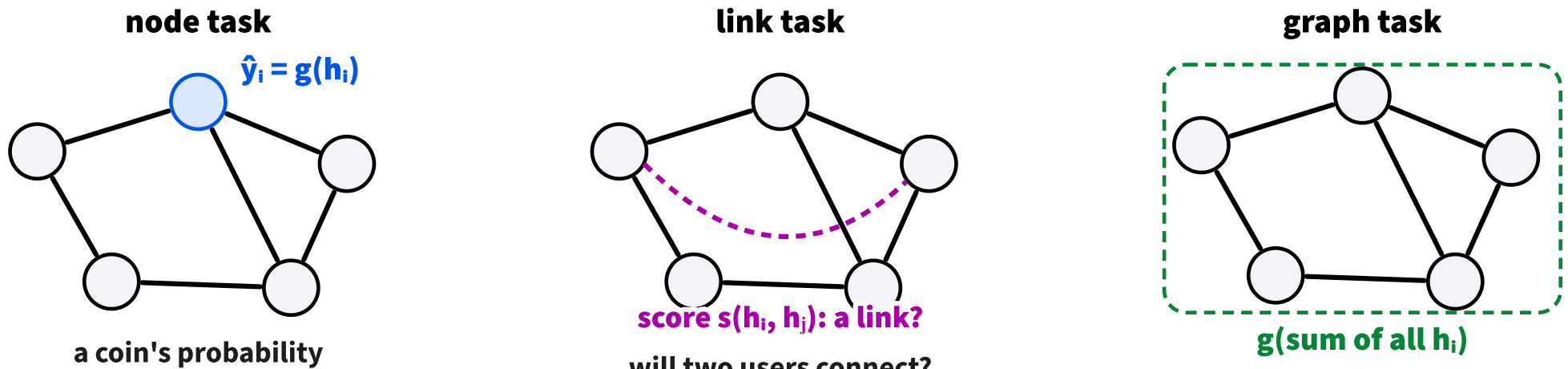
Answer. Graph attention: α_{21} comes from $a \cdot (s_2 + s_1)$, so friend 1's signal raises it. Degrees give all three friends $1/\sqrt{8}$; sampling gives all three $1/3$.

SO **Only attention's learned weights read the scores; degrees and sample size cannot see the signal.**

NEXT What outputs can these node scores give?

One network gives outputs for nodes, links or graphs

Same layers; only the last step reads the final vectors h_i differently.



g: a small network; $\hat{y}_i$: node i's output.

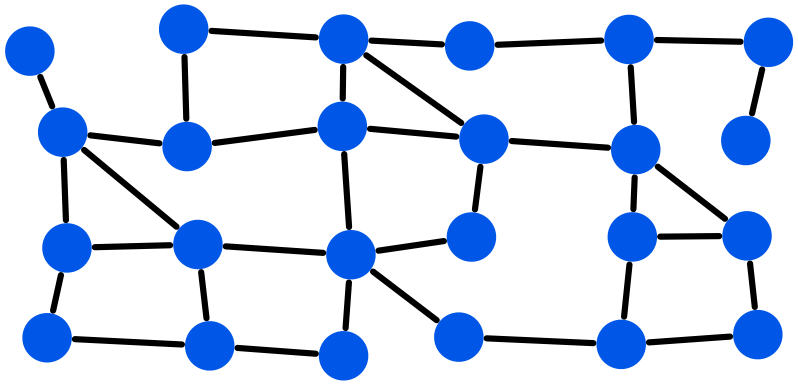
SO **Node and link outputs move with the nodes; the graph-level output never changes (p. 46).**

NEXT How do we train them on graphs too large for memory?

Train on sampled neighbourhoods when the graph is large

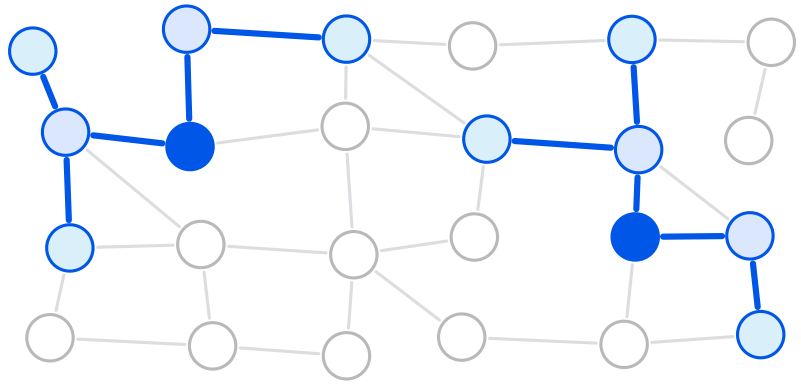
A large graph does **not fit in memory**: train on a few target nodes and their sampled neighbours.

full graph: every node in every step



memory grows with the number of nodes

mini-batch: two target nodes



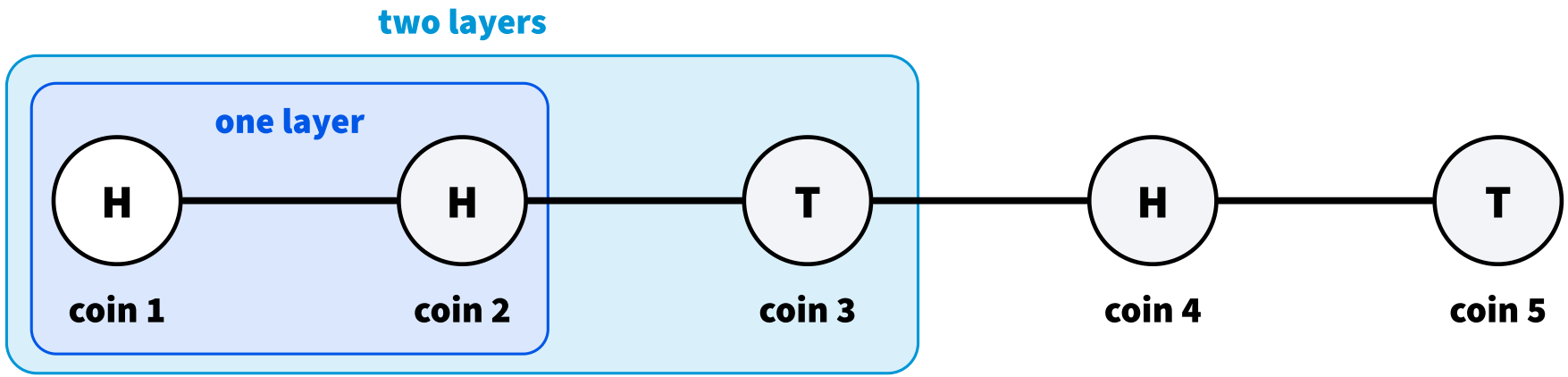
dark: target nodes; light: sampled neighbours, one and two links away

SO **After T layers a node uses only nodes within T links: memory does not grow with the graph.**

NEXT How many layers does a node need?

Next: each layer looks one link further; how many?

One layer passes messages along **one link**.



On the chain H H T H T, a two-layer network runs. Which coins can change coin 1's output?

coins 1 and 2 | coins 1, 2 and 3 | all five coins

Answer. Coins 1, 2, 3: layer 2 passes on what coin 2 heard from coin 3.

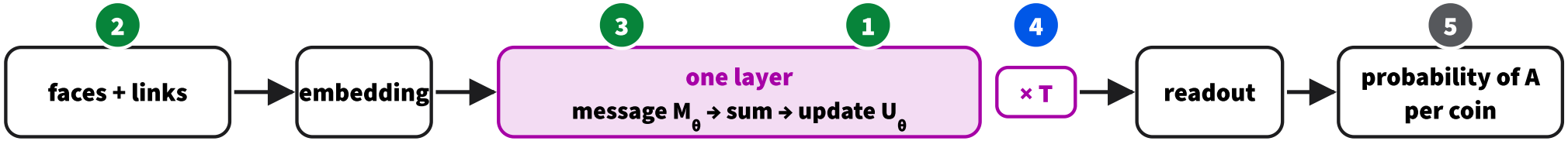
Next: how far a graph neural network sees, and what too many layers cost.

Part 4 asks: how many layers does a coin need?

PROBLEM A coin's output depends on faces several coins away.

WE DO Count how far T layers reach, and what too many cost.

WHY Parts 1 to 3 built one layer; the network stacks T of them.



1	What is one layer, and how do we learn it?	✓ answered
2	Does one trained network fit any numbering and size?	✓ answered
3	Which message and sum should a layer use?	✓ answered
4	**How many layers does a coin need?**	because its output depends on faces several coins away
5	Which graphs can it never tell apart?	because then we know when not to trust it

then 6: all five results on one small network · 7: one application, Sudoku, and the Transformer

SO **Depth buys reach; too much depth makes all nodes alike.**

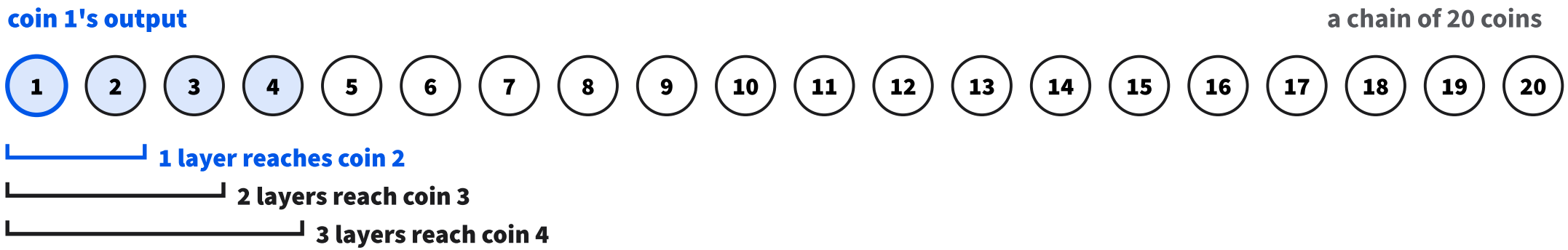
NEXT Can one layer carry coin 3's face to coin 1?

One layer cannot carry coin 3's face to coin 1

PROBLEM Each layer passes messages along **one link**.

WE DO Follow coin 1's output on a chain of 20 coins.

WHY Which faces can its output use?



SO **One layer: coin 1 hears only coin 2. Each layer adds one coin.**

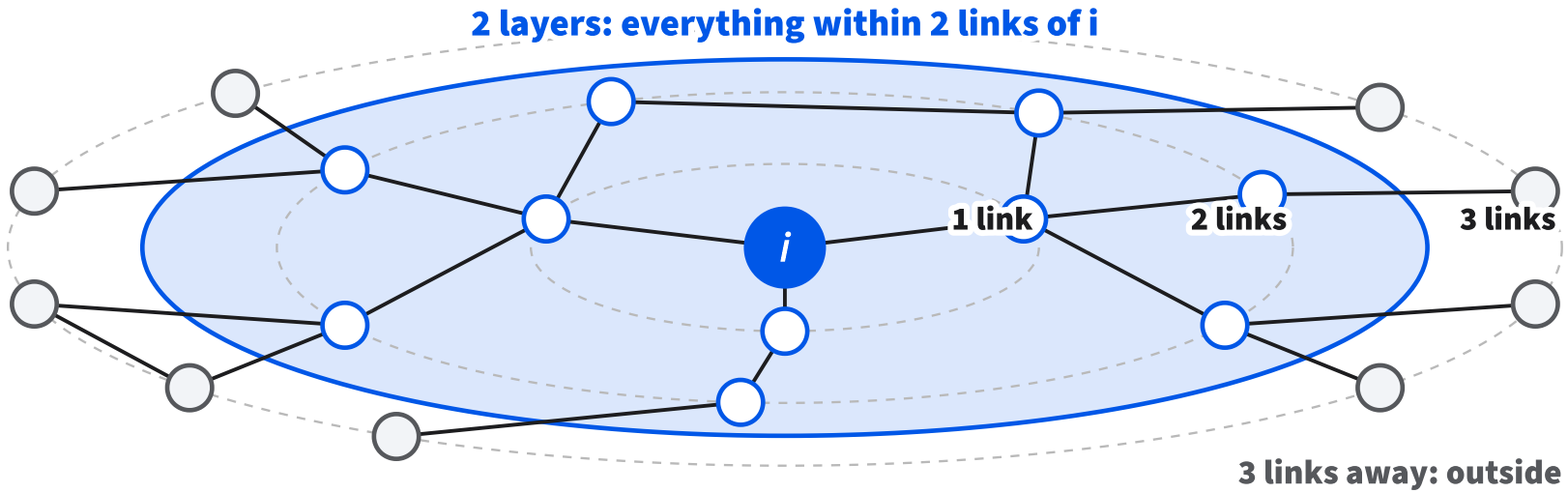
NEXT Which nodes can T layers reach?

The nodes within T links form the receptive field

PROBLEM "Reach" needs a meaning on any graph.

WE DO Measure distance in links.

WHY Distance decides what layers use.



Receptive field $N_T(i) = \{j : \ \mathrm{dist}(i,j) \le T\}$: nodes within T links of i; dist: links on a shortest path.

SO **On the chain:** $N_1(1) = \{1, 2\}$.

NEXT Do T layers use only this field?

After T layers, a node sees only T links far

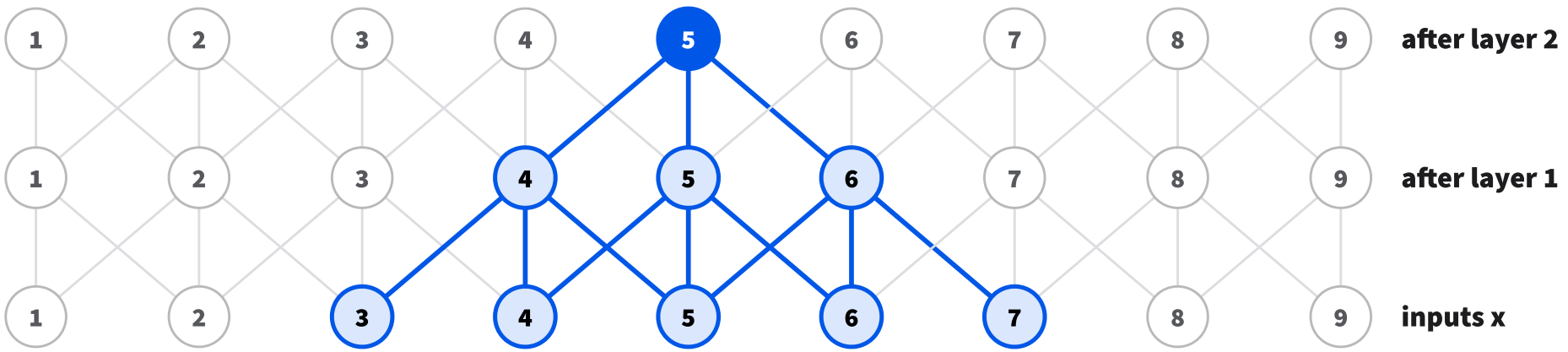
PROBLEM Does it hold on every graph?

WE DO State it for all networks.

WHY Depth then limits reach.

Theorem (receptive field). In every message-passing neural network (MPNN), node i after T layers uses only inputs within T links (p. 146).

In words: one layer, one link.



SO **Coin 5, two layers: coins 3 to 7.**

NEXT Which output is best?

On a tree, the best T-layer output is T rounds of BP

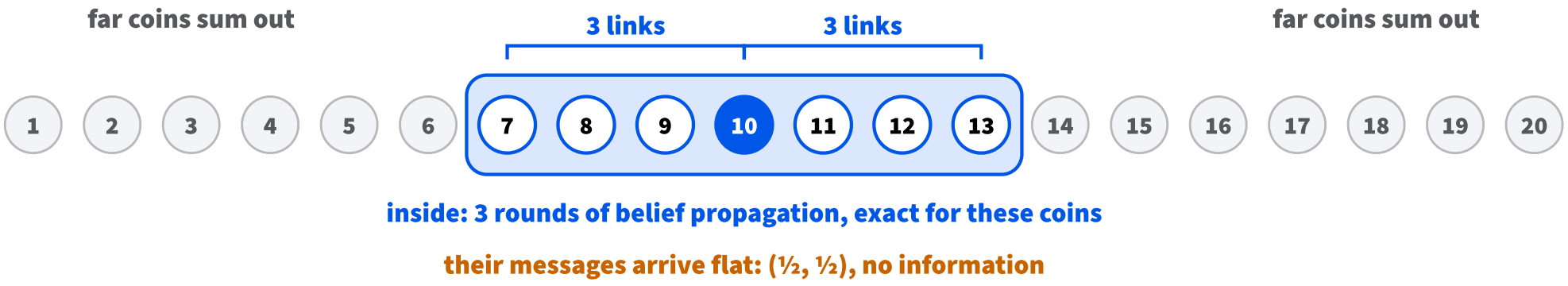
PROBLEM T layers see only nearby coins.

WE DO Find the best output from them.

WHY No training can beat it.

Corollary (tree). On a **tree** (no loops; our chain is one), let y_i be the exact probability of coin i given the faces within T links, which T rounds of belief propagation compute. Any T-layer message-passing network, whatever its parameters, has average cross-entropy loss $\geq$ that of y_i. Proof: (p. 124). With loops the tree step fails and belief propagation itself can fall above the floor (p. 76).

In words: use the T links perfectly; the faces beyond them are out of reach.



SO **Its error is the floor.**

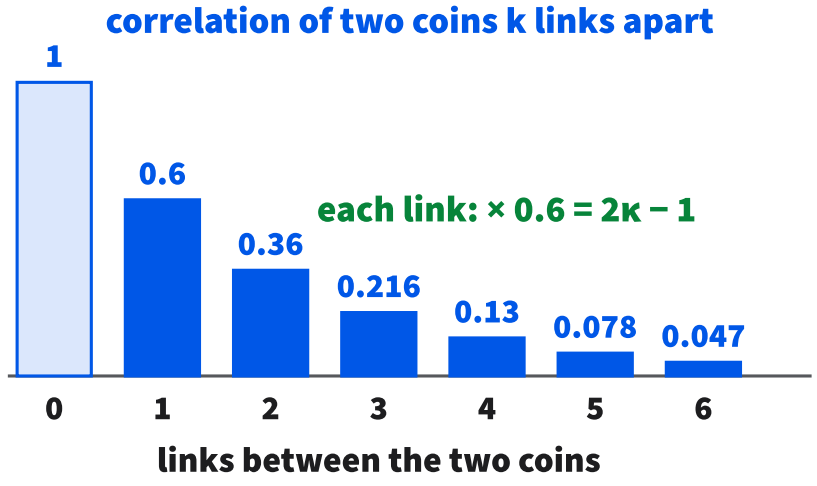
NEXT How large is this floor?

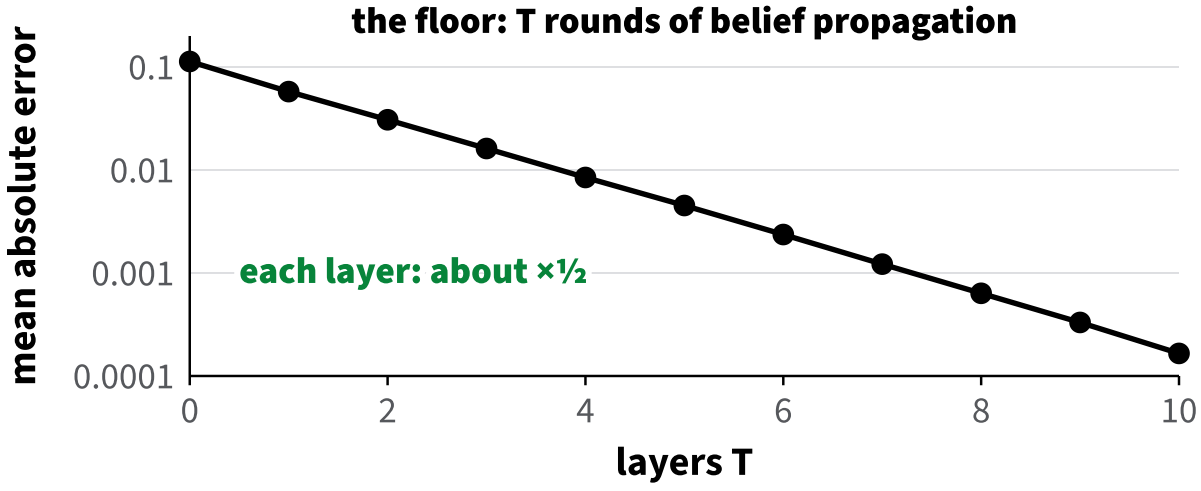
Each extra layer about halves the error floor

PROBLEM How much do the coins beyond T links still matter?

WE DO Measure the gap (p. 37) of T rounds of belief propagation on 400 chains of 20 coins.

WHY **Proposition.** If coin j leans to A by δ (probability $\frac{1}{2}+\delta$), coin $j+k$ leans by $(2\kappa-1)^k \cdot \delta$ (p. 147).





Left: one link, both paths counted: $\kappa(\frac{1}{2}+\delta)+(1-\kappa)(\frac{1}{2}-\delta)=\frac{1}{2}+(2\kappa-1)\delta$; with $\kappa=0.8$ the lean keeps 0.6, and the next link acts on that new lean, so k links keep 0.6^k. **Right:** 400 chains of 20 coins; T rounds of belief propagation against the exact probability, mean absolute difference.

SO **Each link keeps 0.6 of the lean, so each extra layer reaches coins that matter 0.6 times less; measured, the floor falls by about ×0.5 per layer.**

NEXT Should we just stack more layers?

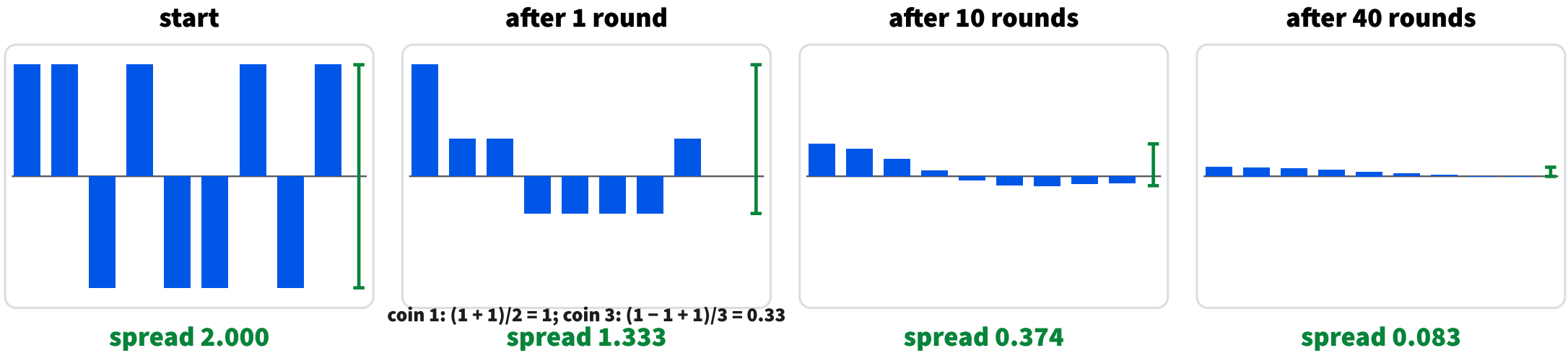
Repeated averaging makes all nodes nearly equal

PROBLEM More layers see further (p. 62). Why not 100 layers?

WE DO Watch the simplest layer, score ← mean of itself and its neighbours, applied 40 times to 9 coins.

WHY Each averaging mixes a coin with its neighbours; after many, all coins hold nearly the **same** score, and the outputs can no longer tell them apart.

↳ 9 coins in a chain, H = +1, T = −1, one bar each.



SO **The spread (range: tallest minus lowest bar) falls from 2 to 0.083: oversmoothing. This is the first price of too many layers; the second is on (p. 67).**

NEXT How fast does the spread shrink?

The spread shrinks by one fixed factor every round

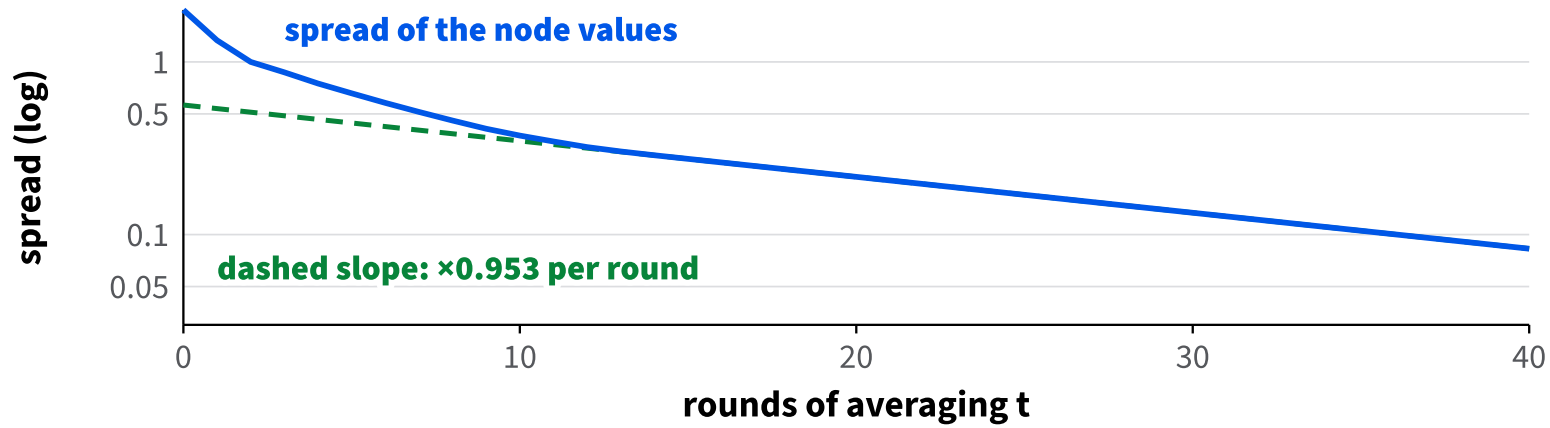
PROBLEM How fast do differences vanish?

WE DO Plot it on a log scale.

WHY A fixed factor gives a straight line.

Theorem (oversmoothing). On a connected graph with self-loops, the spread after t rounds is at most $C \cdot \lambda_2^t$, with $\lambda_2 < 1$ fixed by the graph: after the first rounds it shrinks by about λ_2 each round (p. 148).

In words: differences fade steadily.



SO **Here it is 0.953, as in Markov-chain mixing (Lecture 4, p. 62).**

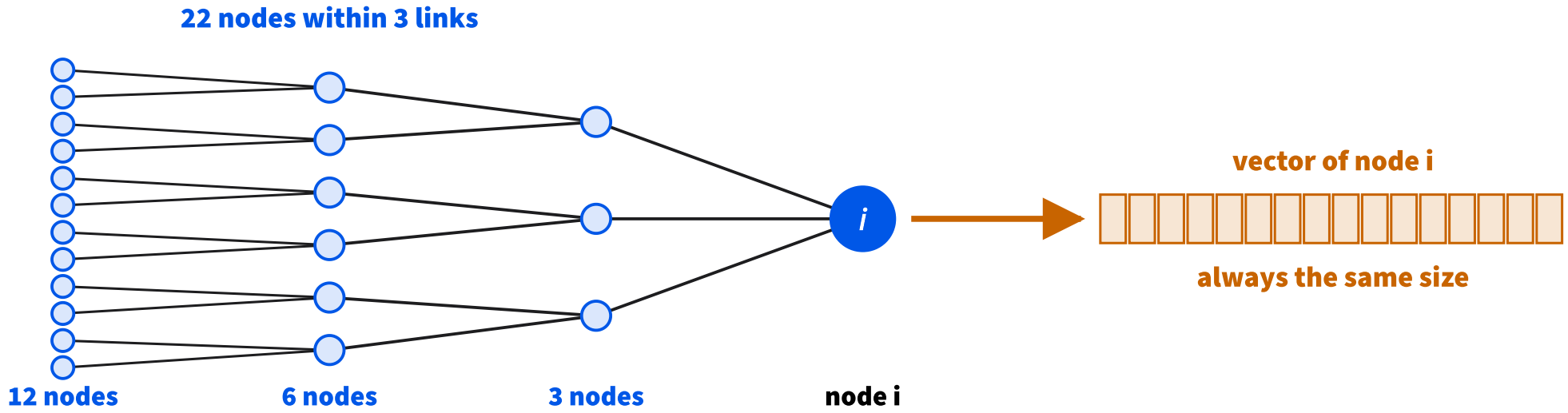
NEXT Does depth have a second cost?

Far information must squeeze into one fixed vector

PROBLEM After T layers, all a node knows about the nodes within T links sits in its **one vector** of fixed size: ours one score, the bigger network 16 numbers.

WE DO Count the nodes within 3 links on a tree with 3 neighbours per node.

WHY The count grows with T; the vector does not.



Observation (Alon and Yahav, 2021). Where far nodes matter, networks fail even to fit the training data.

SO **22 faces must fit into one score or 16 numbers, so far faces are crushed: over-squashing, the second price of depth (p. 149). A chain has only $2T + 1$ such nodes, so our chain barely suffers.**

NEXT How do networks ease both costs?

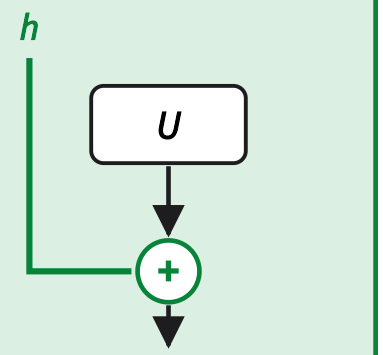
Five fixes keep far information from fading

PROBLEM Depth averages; a fixed vector squeezes.

WE DO Change the update or the graph.

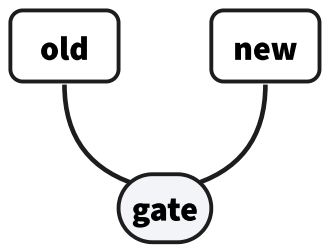
WHY Far information must arrive without fading.

keep the old vector



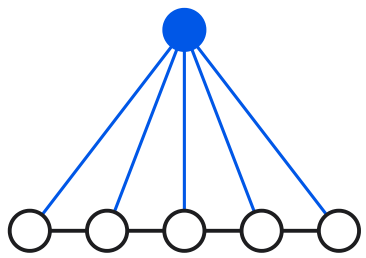
h ← h + U(h, a)

a learned gate



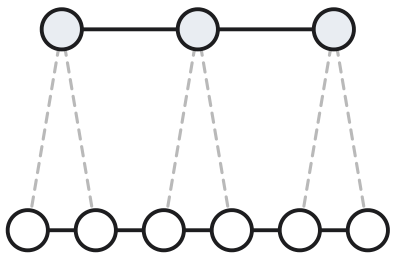
mix old and new

one extra node



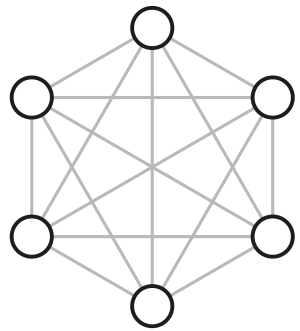
linked to every node

a coarse graph



Lecture 11

attention to all

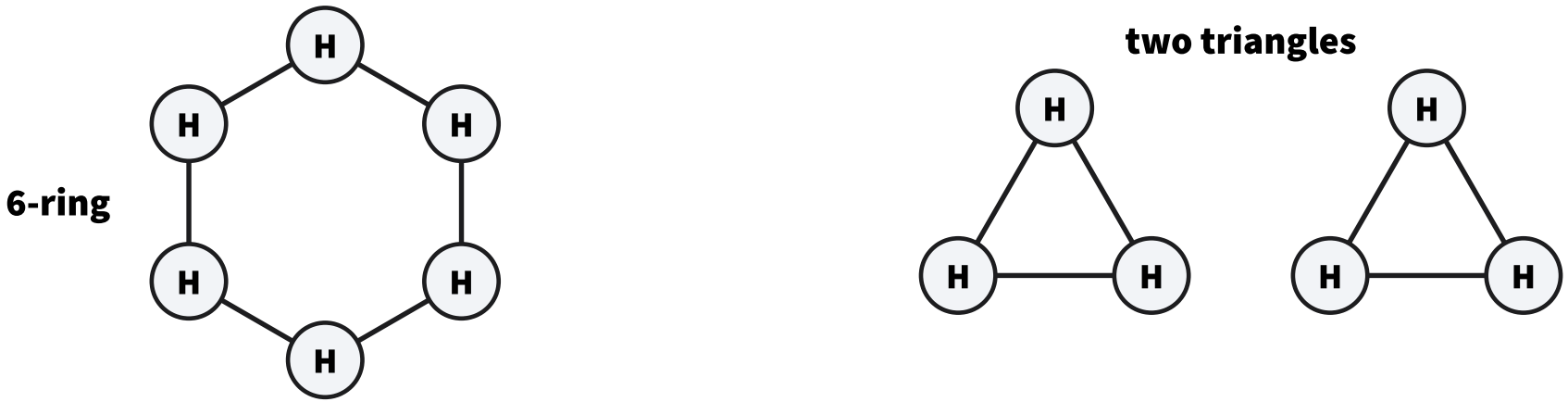


Lecture 12

SO **Our network re-adds its own observation $u \cdot x_i$ in every layer: a form of the first fix.**

NEXT Can enough reach tell all graphs apart?

Next: is enough reach enough to tell graphs apart?



All coins H. Can 10 layers separate the 6-ring from the two triangles?

yes | no | only with ring training

Answer. No: every node sees the same picture at every depth.

SO **Reach is not the only limit.**

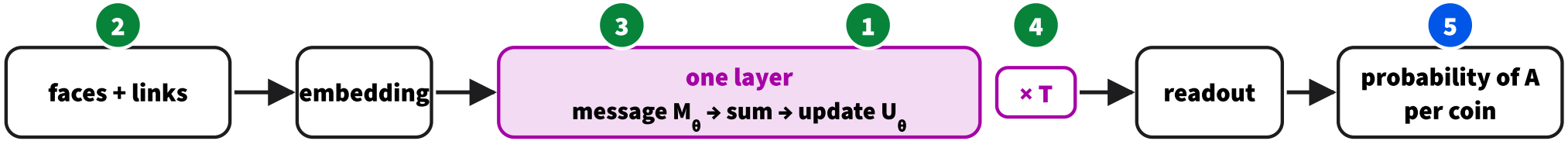
NEXT What can a message-passing network tell apart?

Part 5 asks: which graphs can it never tell apart?

PROBLEM With good parts and enough layers, can the network still be fooled?

WE DO Compare a ring of 6 coins with two triangles.

WHY Then we know when **not to trust** a trained network.



1	What is one layer, and how do we learn it?	✓ answered
2	Does one trained network fit any numbering and size?	✓ answered
3	Which message and sum should a layer use?	✓ answered
4	How many layers does a coin need?	✓ answered
5	**Which graphs can it never tell apart?**	**because then we know when not to trust it**

then 6: all five results on one small network · 7: one application, Sudoku, and the Transformer

SO **The limit is what each node sees, not the number of layers.**

NEXT Do a 6-ring and two triangles get different outputs?

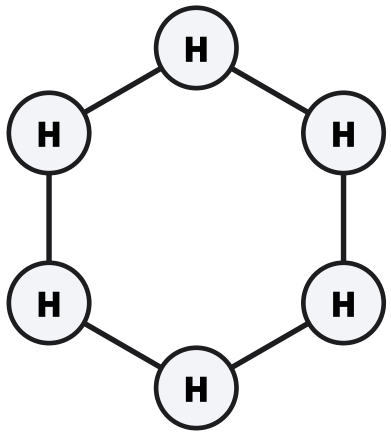
A 6-ring and two triangles get the same outputs

PROBLEM Loops change the exact probability.

WE DO All coins H; compare the exact probability (summing over all 64 cases), loopy belief propagation (Lecture 6) and our network.

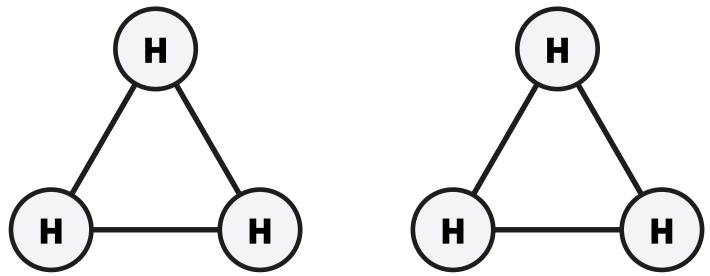
WHY Do the methods notice the difference?

6-ring



exact	0.904
loopy belief propagation	0.908
our network	0.881

two triangles



exact	0.852
loopy belief propagation	0.908
our network	0.881

SO **The exact probabilities differ; loopy belief propagation and the network give one output for both.**

NEXT What does a node of each graph actually see?

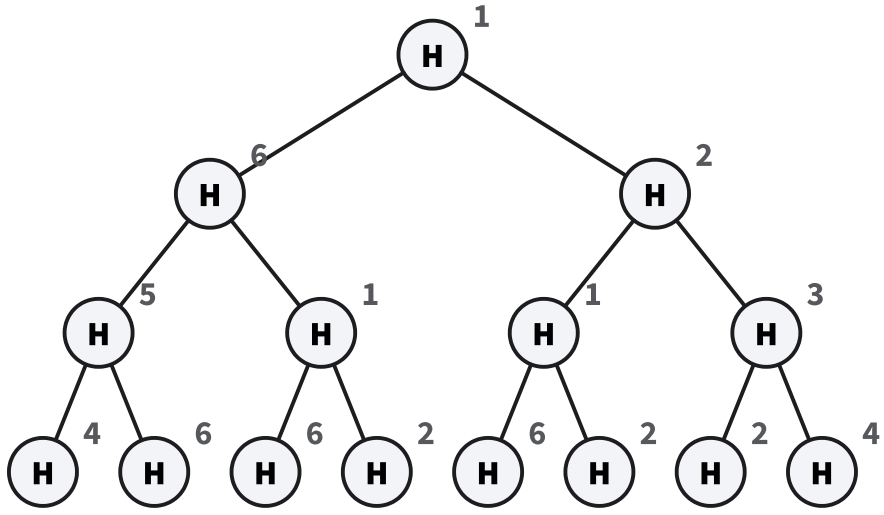
Every node of both graphs sees the same tree

PROBLEM Why one output for two graphs?

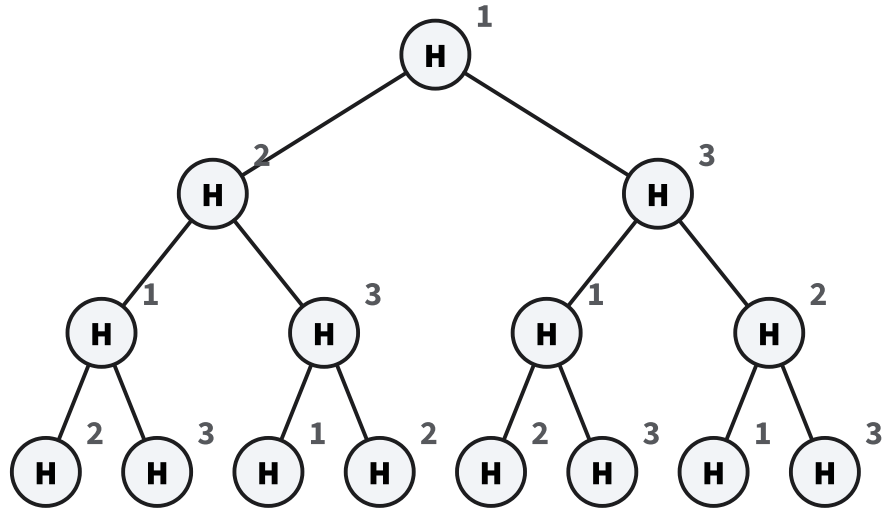
WE DO Unroll what node 1 hears.

WHY The network computes on this tree.

6-ring, from node 1



two triangles, from node 1



Grey numbers name nodes; the network sees only faces.

SO **Same shape, all H, at every depth. This is the computation tree.**

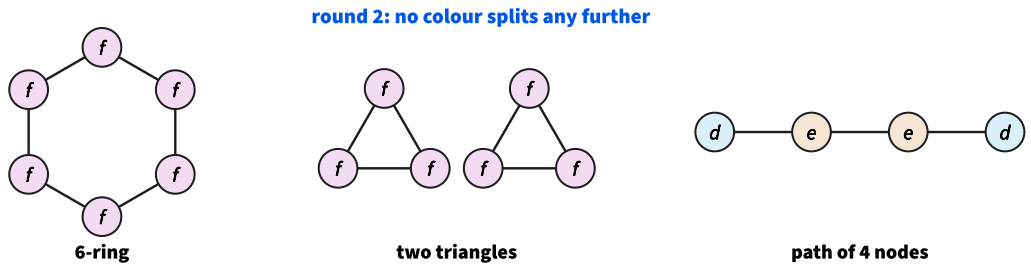
NEXT How can we compare such trees quickly?

Colour refinement compares computation trees quickly

PROBLEM Computation trees double each layer: too big to compare.

WE DO Round 0: colour = face. Each round, write down for every node its colour and how many neighbours of each colour it has; nodes with the **same note** get the same new colour, different notes different colours.

WHY Same colour after t rounds $\Leftrightarrow$ same tree of depth t.



round	node	its colour	its neighbours' colours	new colour
1	path, end node	f	one f	d
1	path, middle node	f	two f	e (differs from d: a different pair)
1	ring or triangle, any node	f	two f	e, the same for all 12 nodes

Definition. **Weisfeiler–Leman (WL) colour refinement**: $c_i^{(t+1)} = \mathrm{new}\big(c_i^{(t)},\ \{c_j^{(t)} : j \in N(i)\}\big)$; **new** is a lookup table (an injective map): each distinct pair gets its own colour name, e.g. the next unused letter (p. 150).

SO **Path ends split off (round 2 splits nothing more); ring and triangles keep one colour.**

NEXT Do equal colours force equal outputs?

Equal colours force equal network vectors

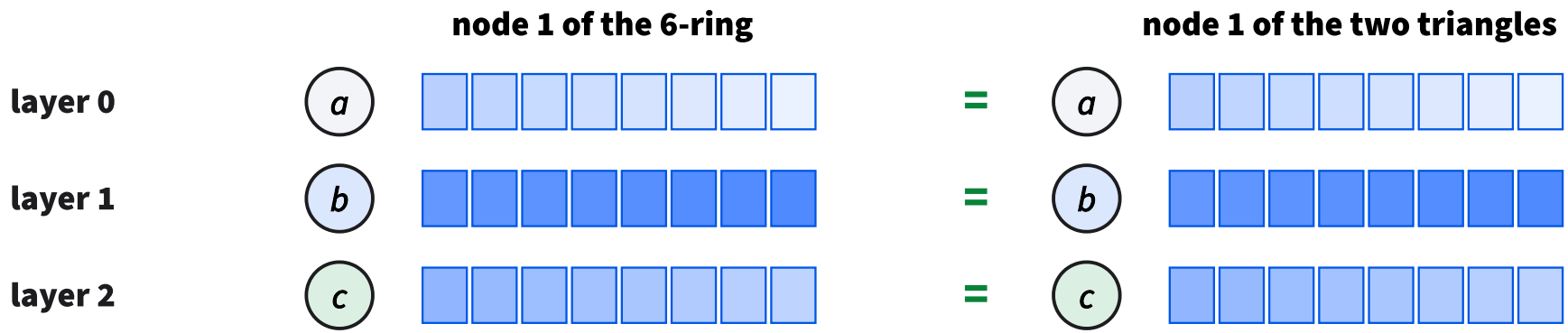
PROBLEM Is refinement a limit?

WE DO Compare colours and vectors by layer.

WHY Both read the same inputs.

Guarantee (Xu et al., 2019; Morris et al., 2019). Nodes with equal colours after t rounds get equal vectors after t layers, in every message-passing network (p. 151).

In words: colours bound what networks see.



SO **Both graphs: one colour, one output.**

NEXT Which networks reach this limit?

A sum keeps counts; a mean or a maximum loses them

PROBLEM Refinement's note counts neighbours of each colour. Which way of combining messages keeps those counts?

WE DO Node i has neighbours {1, 1}; another node has {1}. Combine by mean, max and sum.

WHY If the combination loses the count, two nodes with different notes get the same vector.



Guarantee (Xu et al., 2019). If a layer **sums** the neighbours' vectors (encoded so that the sum identifies the multiset) and its update is **one-to-one** (distinct inputs give distinct outputs, like *new* (p. 73)), then after t layers two nodes have equal vectors exactly when they have equal colours. Such a layer, $h_i' = \mathrm{MLP}\big((1+\epsilon)\,h_i + \sum_{j\in N(i)} h_j\big)$, is the **graph isomorphism network (GIN)** (p. 127).

SO **Only the sum separates them. Ours sums too, but its linear update is not one-to-one.**

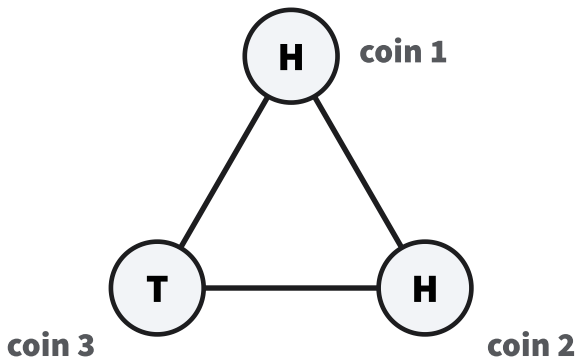
NEXT Why did loopy belief propagation fail too?

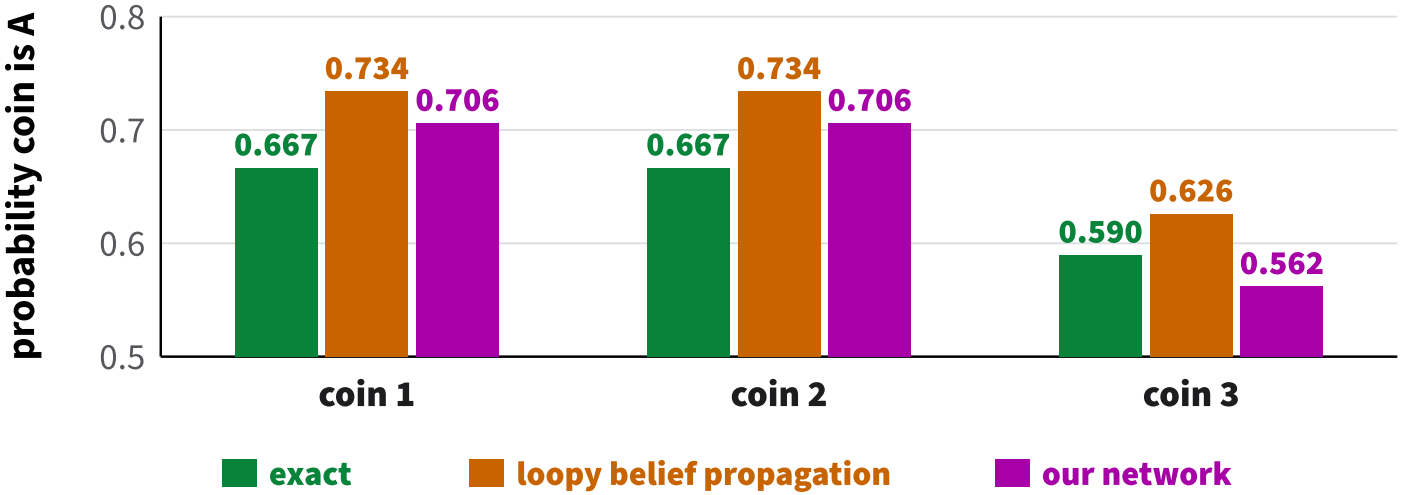
Loopy belief propagation also runs on the tree

PROBLEM Loopy belief propagation failed too.

WE DO Unroll its messages **(Lecture 6, p. 49)**; exact: sum over the 8 cases; our network: the rule of **(p. 81)**.

WHY Its computation tree repeats each coin's evidence.





SO **On the 3-coin ring of the midterm, loopy belief propagation overshoots; our network lands closer to the exact probability. Two different limits: on a given ring the network can be more accurate than belief propagation, yet neither can tell the ring from the triangles (p. 74).**

NEXT Which extra input reveals the loop?

One extra number per node can reveal a loop

PROBLEM Every node looks alike.

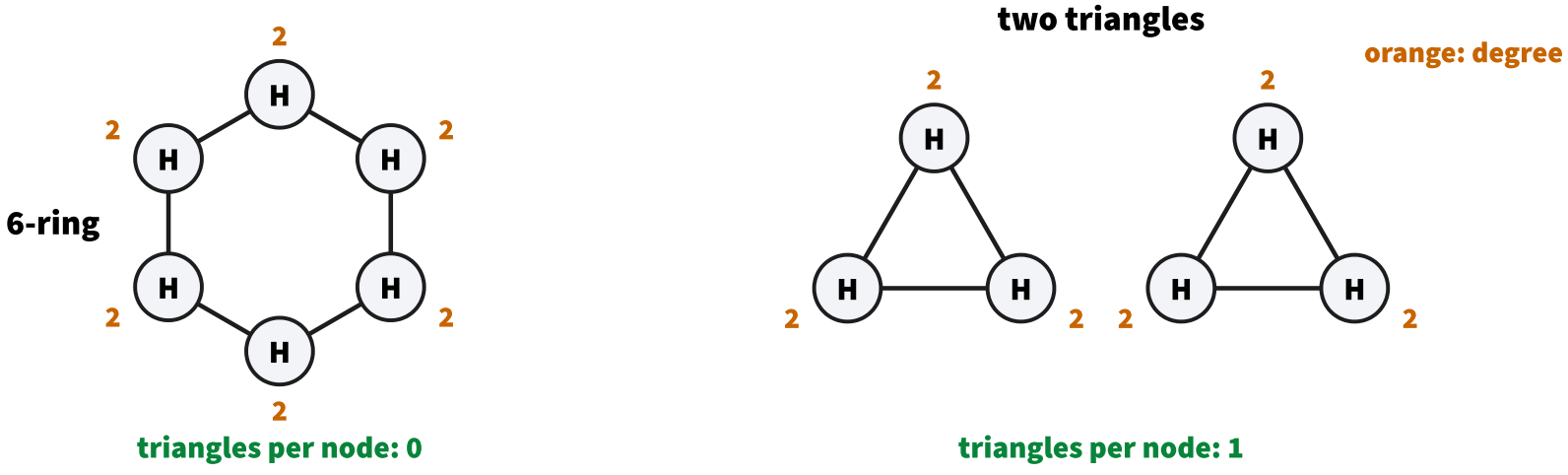
WE DO Add one input per node.

WHY New inputs split colours.

Add each node's number of links (degree). Does it separate the two graphs?

yes | no | only with many layers

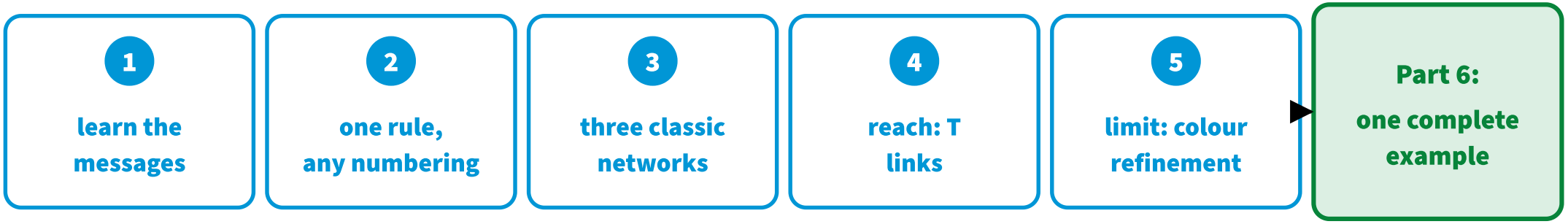
Answer. No: every degree is 2. Triangle counts work.



SO **Inputs that see cycles help.**

NEXT How do Parts 1 to 5 fit together?

Next: Parts 1 to 5 on one complete example



Parts 1 to 5: the coin chain, learned messages, and their limits

Part 4: a network with few layers cannot see far. Part 5: it cannot tell the 6-ring from two triangles. Which of the two does adding layers fix?

the reach | the colour limit | both

Answer. Only the first: each extra layer reaches one link further. The second never: ring and triangles have equal colours at every round, so equal vectors at every depth (p. 74).

SO **Part 6 tests all five results on one trained network.**

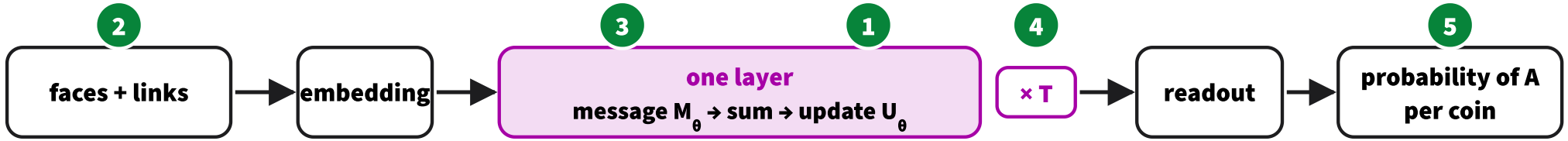
NEXT Do all five results hold on one trained network?

Part 6 runs all five results on one small network

PROBLEM Parts 1 to 5 gave five results in theory.

WE DO Train the two-parameter network, then check each result **by hand** or by test.

WHY Each result becomes a number you can reproduce.



1	What is one layer, and how do we learn it?	✓ answered
2	Does one trained network fit any numbering and size?	✓ answered
3	Which message and sum should a layer use?	✓ answered
4	How many layers does a coin need?	✓ answered
5	Which graphs can it never tell apart?	✓ answered

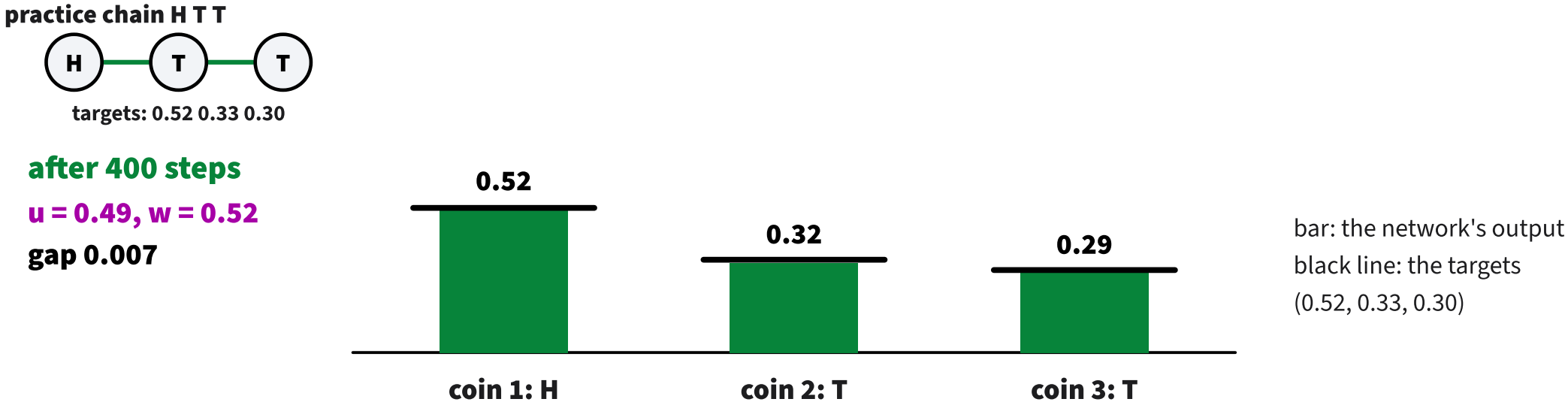
then 6: all five results on one small network · 7: one application, Sudoku, and the Transformer

SO **One network, $\theta = (u, w)$, through all five questions.**

NEXT What do the outputs on H T T do during training?

During training, the outputs on H T T walk to the targets

Start at $u = w = 0.1$. Each step: run the network on the practice chains, compute the gap, move u, w by the rule (p. 38). We pause at steps 0, 5, 20, 400 and show the outputs on one practice chain, H T T.



SO **The gap falls to 0.007. Training ends at $u = 0.49, w = 0.52$: both close to ½.**

NEXT What do u = w = ½ give on our chain?

Three layers on our chain, by hand

Round u and w to $\frac{1}{2}$. Each layer: $s_i \leftarrow \frac{1}{2} \cdot x_i + \frac{1}{2}\cdot$**(sum of the neighbours' s)**, all coins at once; then probability of A $= 1/(1+e^{-s_i})$.

coin	H 1	H 2	T 3	H 4	T 5
face x	+1	+1	−1	+1	−1
start s = ½ · x	0.5	0.5	−0.5	0.5	−0.5
after layer 1	0.75	0.5	0	0	−0.25
after layer 2	0.75	0.875	−0.25	0.375	−0.5
after layer 3	0.9375	0.75	0.125	0.125	−0.3125
probability of A	0.719	0.679	0.531	0.531	0.423
exact	0.715	0.688	0.524	0.561	0.423

coin 3: 1 / (1 + e^(−0.125)) = 0.531

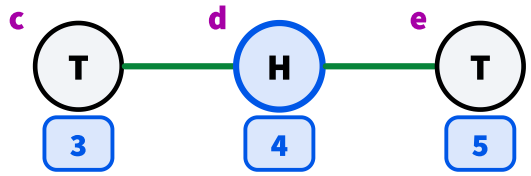
SO **Coin 3 hears all five faces after two layers. Every coin lands within 0.03 of the exact probability; on 400 test chains the average gap is 0.025.**

NEXT Does a swap of numbers change any score?

Coin d's update reads its linked coins, not their numbers

Follow coin d, by hand with $u = w = \frac{1}{2}$ (p. 81). It keeps number 4; its neighbour c now has number 1.

old numbers

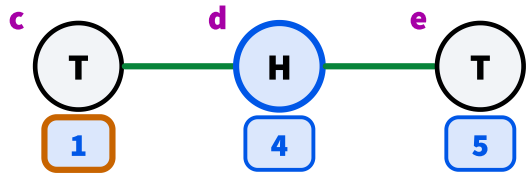


$s_4 \leftarrow ½ \cdot x_4 + ½ \cdot (s_3 + s_5)$

s_3 is coin c

layer 1: ½ · (+1) + ½ · (−0.5 − 0.5) = 0

new numbers



$s_4 \leftarrow ½ \cdot x_4 + ½ \cdot (s_1 + s_5)$

s_1 is coin c: its number changed

layer 1: ½ · (+1) + ½ · (−0.5 − 0.5) = 0

both: after 3 layers s = 0.125, probability 0.531; the same two coins, c and e, feed coin d

SO **Same two neighbours, same scores in, same score out. Only the name of a score changed.**

NEXT Does this hold for all five coins?

The example: after the swap, every coin keeps its score

Each column is **one coin**, in its place. We run the network with the old numbers and with the new ones; the **score** is its lean to A (p. 34).

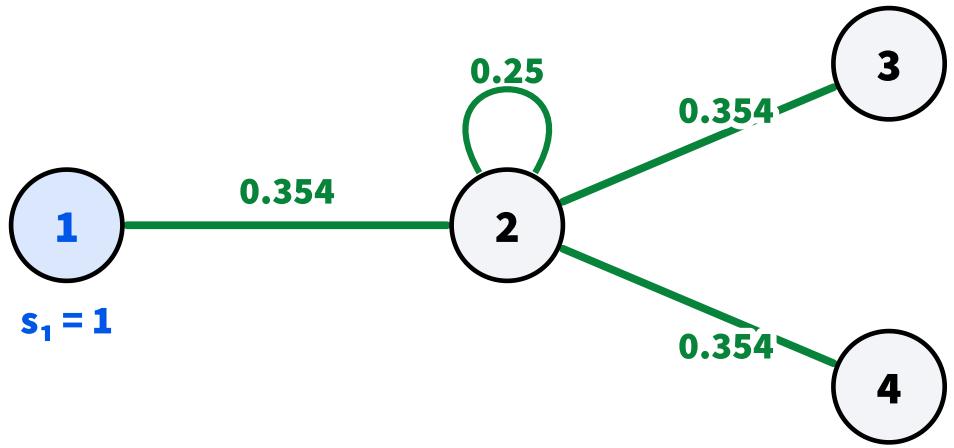
coin	a H	b H	c T	d H	e T	
number after the swap	3	2	1	4	5	
score, old numbers	0.9375	0.75	0.125	0.125	−0.3125	
score, new numbers	0.9375	0.75	0.125	0.125	−0.3125	identical
probability of A	0.719	0.679	0.531	0.531	0.423	
exact	0.715	0.688	0.524	0.561	0.423	

SO **Each coin gets the same score, so the same probability, in both runs: only its number changed. The outputs follow the coins: equivariant (p. 45).**

NEXT How do the three classic networks weigh friends on Lecture 7's tree?

Graph convolution by hand: weights from degrees alone

The three rules of Part 3 differ in how node 2 **weighs** its friends' scores, not in training. Graph convolution (p. 51) with $w = 1$, no relu: $s'_2 = \sum_j s_j / \sqrt{\tilde{d}_2 \tilde{d}_j}$ with $\tilde{d}_2 = 4$, $\tilde{d}_1 = \tilde{d}_3 = \tilde{d}_4 = 2$; only $s_1 = 1$.



weights on the links into node 2

node 2's new value:
0.354

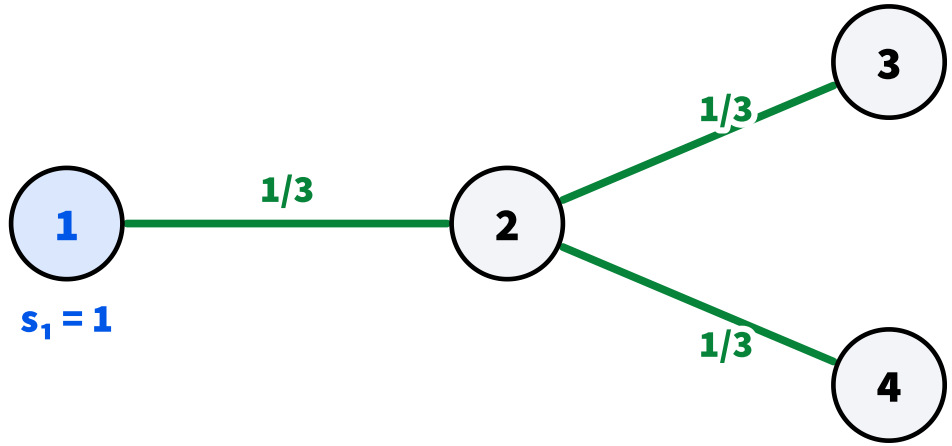
friend 1: 1 link + itself = 2; node 2: 3 links + itself = 4; weight 1 ÷ √(2 · 4) = 0.354

SO $s'_2 = 1/\sqrt{4 \cdot 2} = 0.354$**. Friends 3 and 4 would get the same weight: degrees cannot see the signal (p. 144).**

NEXT What does an average of sampled friends give?

GraphSAGE keeps node 2's own value beside the mean

Same start, $w_1 = w_2 = 1$ (p. 52); the sample is all three friends, each with weight one third.



each friend gets one third; node 2's own value is kept apart

weights on the links into node 2

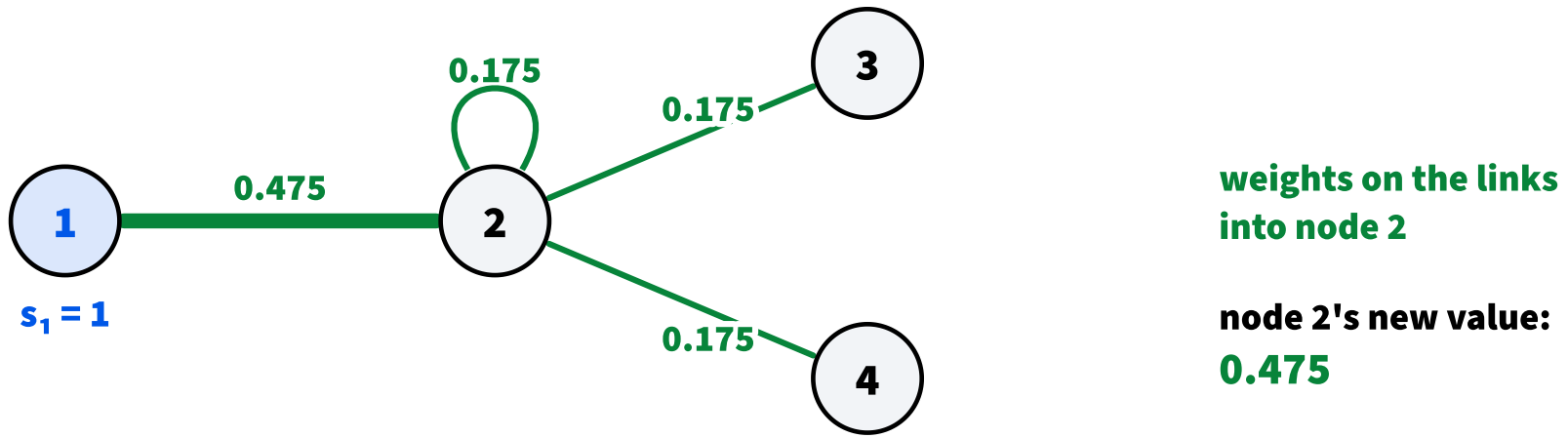
node 2's new value:
0 + 0.333 = 0.333

SO **The mean of 1, 0, 0 is 0.333; node 2's own value, 0, stays apart (p. 145).**

NEXT What weights does graph attention give here?

Graph attention gives friend 1 the largest weight

Graph attention (p. 53) with $a = 1, w = 1$, no relu. Scores: $e_{21} = s_2 + s_1 = 1$; $e_{23} = e_{24} = e_{22} = 0$. Weights: $\alpha_{21} = e^1/(e^1 + 3e^0) = 2.718/5.718 = 0.475$; the other three $1/5.718 = 0.175$ each.



friend 1 scores 1, the others 0; softmax turns scores into weights with total 1

SO $s_2' = 0.475 \cdot 1 + 0.175 \cdot (0 + 0 + 0) = 0.475$**: the weight follows the signal; the other two rules gave friend 1 only 0.354 or ⅓ (p. 145).**

NEXT How close does our network come to the floor?

Two parameters follow the floor for one layer, then fall behind

PROBLEM Is the floor reached?

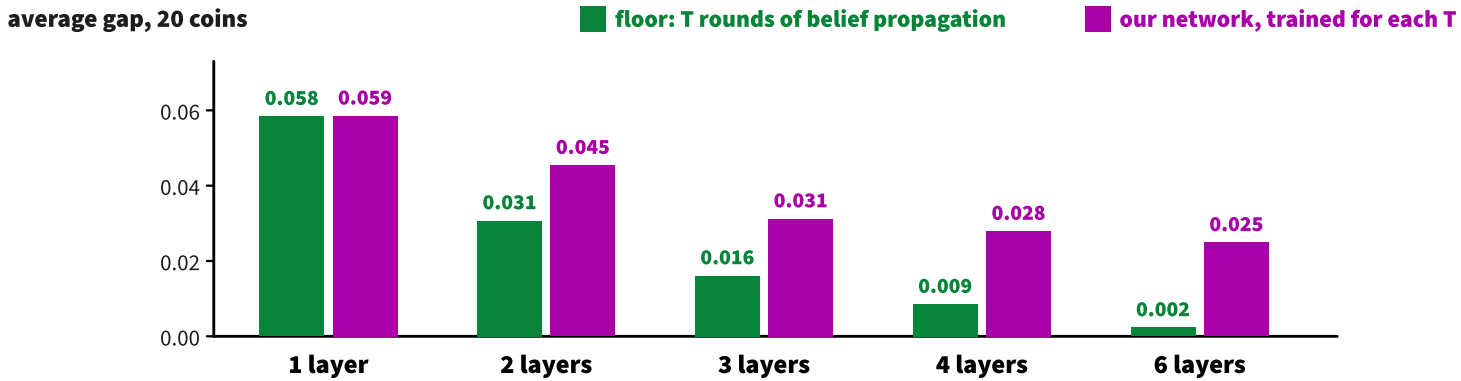
WE DO Train our network for 1, 2, 3, 4 and 6 layers.

WHY The floor is the best any network can do.

Can 2 trained layers beat 2 rounds of belief propagation?

yes | no | only on long chains

Answer. No, in the cross-entropy loss (p. 63); in our gap: 0.045 against 0.031.



SO **One layer: 0.059 against 0.058. More layers: two parameters are too few; the 1 649-parameter network stays within 0.004 (p. 122).**

NEXT Trained on 3 to 8 coins, what happens on 50?

The theorem predicts the 50-coin result; the test agrees

PROBLEM Trained on 3 to 8 coins; used on 50.

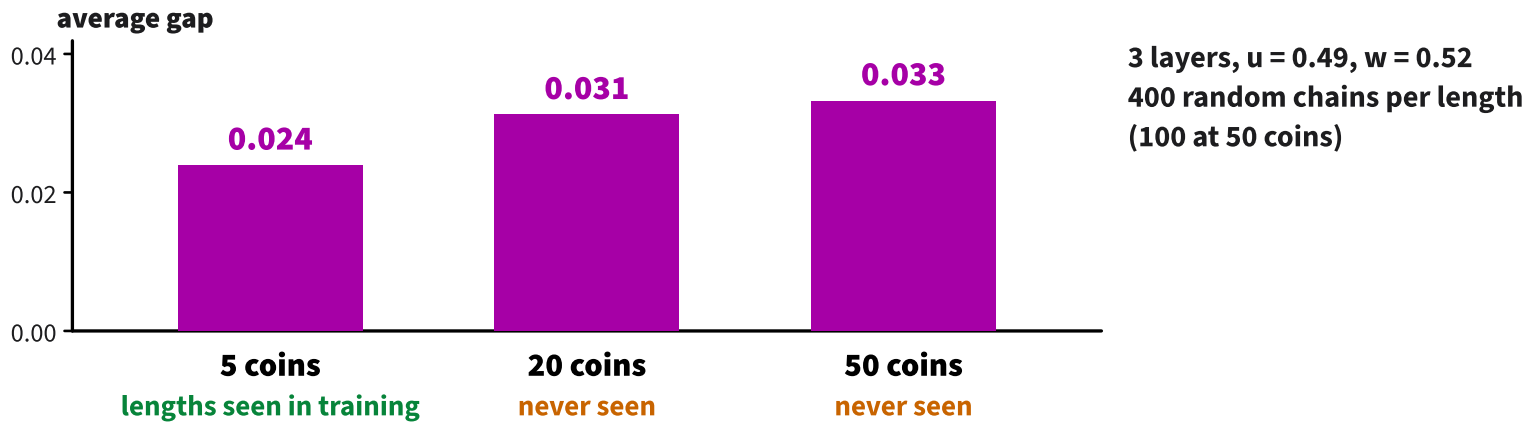
WE DO Predict from the theorem, then measure.

WHY A theorem says what to expect **before** testing.

Our 3-layer network sees only 3 links far (p. 62). On 50 coins, its average gap to the exact probabilities …

grows tenfold or more | stays near 0.03 | cannot be computed

Answer. Stays near 0.03. Three layers reach 3 links each way, so a coin sees 1 + 3 + 3 = 7 coins whatever the length (p. 62); on 20-coin chains the gap was 0.031 (p. 87). Measured on 100 random 50-coin chains: 0.033.



SO **Length does not matter, as the theorem says; how close training gets to the floor is measured, not proved.**

NEXT Can our network tell a 6-ring from two triangles?

By hand: every node of both graphs gets the same score

Our network, $u = w = \frac{1}{2}$ (p. 81). All faces are H, and in both graphs every node has exactly two neighbours.

every node: face +1 and two H neighbours, so s ← ½ + ½ · (2 · s)

start	layer 1	layer 2	layer 3
0.5	**1**	**1.5**	**2**
	½ + ½ · 2 · 0.5	½ + ½ · 2 · 1	½ + ½ · 2 · 1.5

6-ring and two triangles alike: probability 1 / (1 + e^(−2)) = 0.881; exact 0.904 and 0.852

SO **Each node sees the same numbers in every layer, so the graphs look identical to the network.**

NEXT What does the whole example show?

The example in one table: every result, measured

One network: 3 layers, $\theta = (u, w) \approx (\frac{1}{2}, \frac{1}{2})$ (p. 81). Each row is one part's result, tested.

part	what the theory says	what we measured
1 learn	two parameters can copy belief propagation	400 chains: average gap 0.025
2 symmetry	outputs follow the coins	swap a and c: every score identical
4 depth	length does not matter	20 coins: 0.031; 50 coins: 0.033
5 limits	6-ring and two triangles look alike	both 0.881; exact 0.904 and 0.852

Part 3 changes the parts, not this network; its numbers are on the friends' tree (p. 84). A bigger network, 1 649 parameters, reaches 0.006 (p. 109).

SO **Every theoretical result shows up in the numbers of one network with two parameters.**

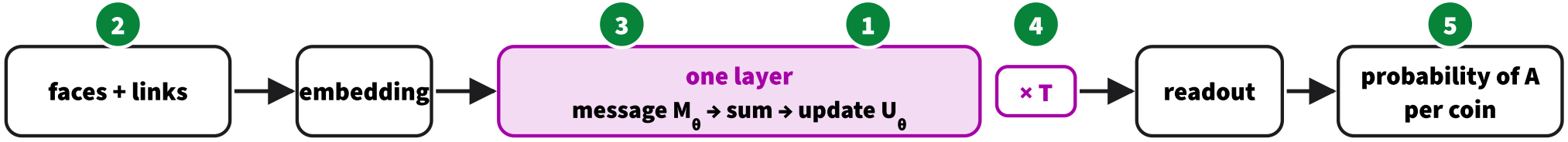
NEXT Can the network learn a puzzle whose rules it is never told?

Part 7: one application in full, then the Transformer

PROBLEM Can the network learn to solve a puzzle whose rules it is never told?

WE DO Sudoku as a graph, every part of today used once; then a language model as the same network on the complete graph.

WHY One application in full teaches more than six in passing (others: appendix); the Transformer shows where this leads.



1	What is one layer, and how do we learn it?	✓ answered
2	Does one trained network fit any numbering and size?	✓ answered
3	Which message and sum should a layer use?	✓ answered
4	How many layers does a coin need?	✓ answered
5	Which graphs can it never tell apart?	✓ answered

then 6: all five results on one small network · 7: one application, Sudoku, and the Transformer

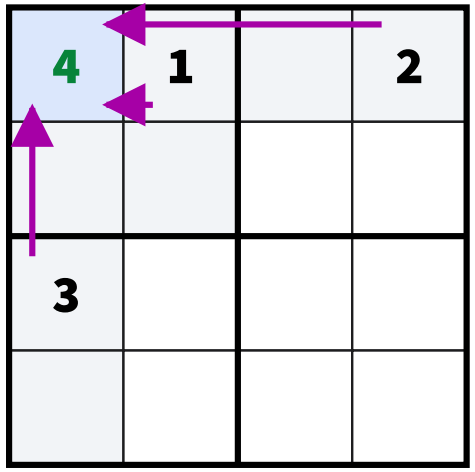
SO **Same start, layers and readout; only the graph changes.**

NEXT What are the rules, and how is a puzzle a graph?

Sudoku is a graph: cells linked by row, column and box

Rule: every row, column and box holds each digit **exactly once** (here 1 to 4). **Graph:** each cell is linked to its row, column and box.

COIN CHAIN		SUDOKU
coin	↔	cell
face we see	↔	given digit
probability of A or B	↔	probability of each digit



the blue cell's neighbours: its row, its column and its 2 × 2 box (grey)

from the 1 in its row: not 1

from the 2 in its row: not 2

from the 3 in its column: not 3

so the blue cell is 4

SO **4 × 4: 7 neighbours per cell; 9 × 9: 20. A filled cell tells its neighbours "not my digit".**

NEXT What does a cell hold, and what replaces each part of our network?

Every part of our network has a Sudoku version

A coin held one score, leaning to A or B. A cell holds **four probabilities**, one per digit; a given digit starts at certainty.

part	coin chain (p. 34)	Sudoku
start	**score $u \cdot x_i$ from the face**	**given digit: that digit 1, others 0; blank: ¼ each**
message M_θ	**$w \cdot s_j$, one number**	**a small network reads the neighbour's four probabilities; learns "rule out"**
update U_θ	**$u \cdot x_i +$ sum of messages**	**a small network combines the sum with the cell's own four probabilities**
readout	**probability of A $= 1/(1+e^{-s})$**	**four probabilities that add to 1**
targets	**belief propagation**	**solved puzzles**

SO **Same network; θ is bigger because a message must read four probabilities, not one score. The rules are never written in: they must be learned.**

NEXT What does one round do, and why are several needed?

$\mathbf{M_\theta}$ = message, ours $w \cdot s_j$ · $\mathbf{U_\theta}$ = update, ours $u \cdot x_i$ + sum · $\boldsymbol{\theta}$ = parameters, ours (u, w)

Round 2 uses what round 1 decided: reach grows by one cell

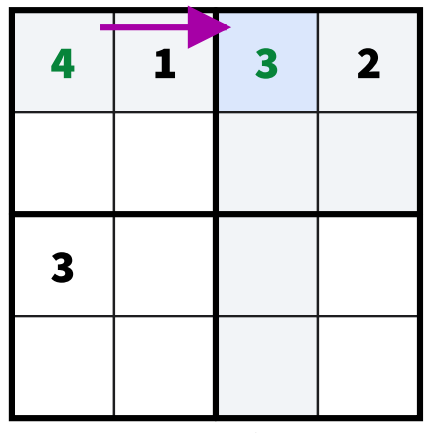
Round 1 settled the blue cell. The cell to its right heard only 1 and 2 in round 1; in round 2 it also hears the new 4.

after round 1

4	1	?	2
3			

the ? cell heard 1 and 2 in round 1: not enough

round 2



the 4 travelled one link: one round

from the new 4: not 4

from the 1: not 1

from the 2: not 2

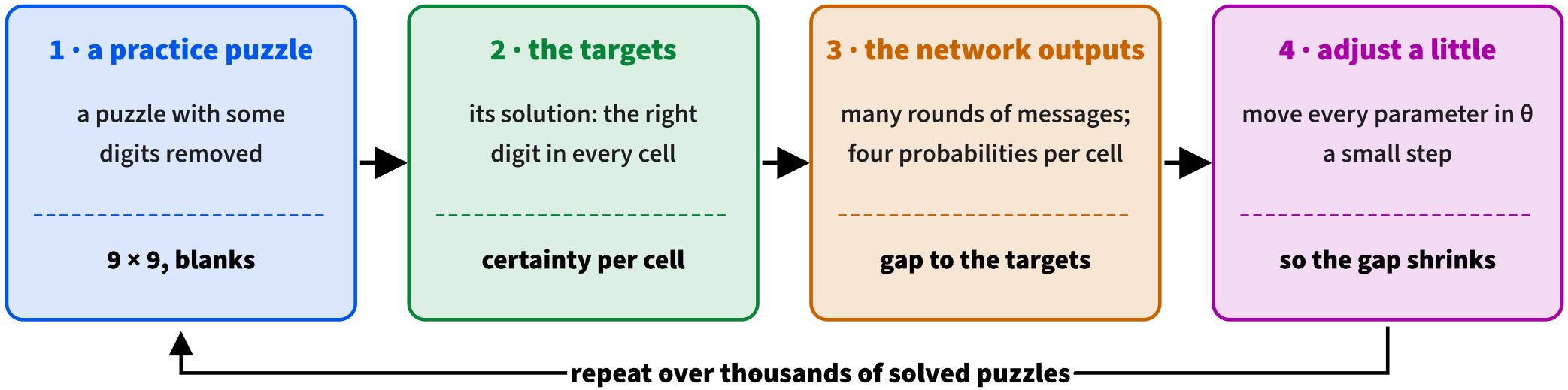
so the cell is 3

SO **A deduction travels one link per round (p. 62): hard puzzles need chains of deductions, so many rounds.**

NEXT How does the network learn the rules?

Training: solved puzzles are the targets

As in Part 1 (p. 37): a practice puzzle, its solution as the targets, the network's output, a small change to the parameters θ. Nobody tells the network the rules.

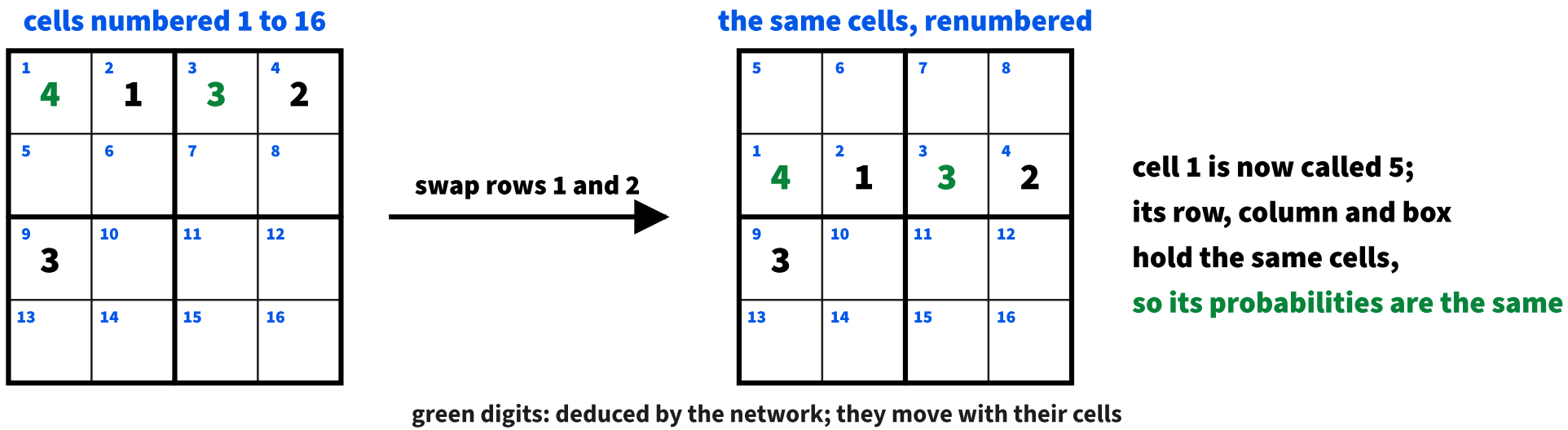


SO **The message network learns "a 1 next door rules out 1" because that shrinks the gap on thousands of solved puzzles.**

NEXT Why does one network solve every puzzle?

Swap two rows: the same graph, so the outputs swap along

Rows 1 and 2 share the same boxes, so swapping them only **renumbers** the cells (p. 42). Every cell keeps its neighbours.



SO **By the theorem of Part 2 (p. 45), each cell keeps its four probabilities: one network for every puzzle, and for 9 × 9 as for 4 × 4.**

NEXT How well does it work, and where does it stop?

Result: 96.6% of the hardest puzzles, and two limits we predicted

1. **Result.** Trained on solved puzzles, the network solves **96.6%** of the hardest 9 × 9 Sudokus in its test set [Sudoku].
2. **Limit 1, reach (Part 4).** A deduction travels one link per round; hard puzzles need long chains, so **many rounds**.
3. **Limit 2, what it sees (Part 5).** Two blank cells with the same computation tree (p. 74) get the **same four probabilities**: no guessing between them.

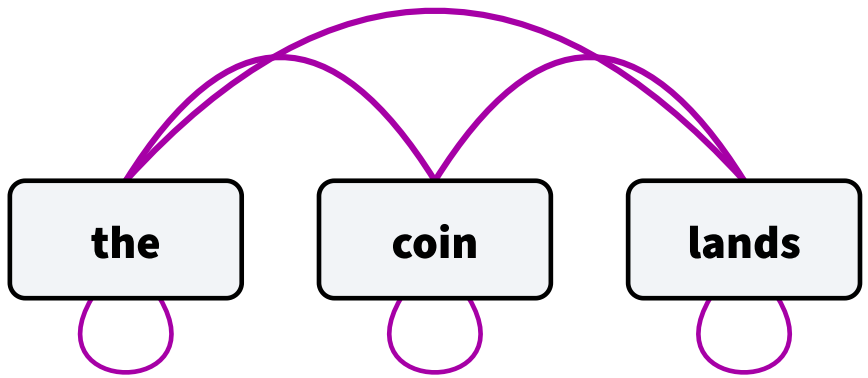
> **One graph, one learned message, every part of today at work.**

SO **Sudoku used a graph we chose by hand.**

NEXT What graph does a language model use?

A sentence is a graph: every word linked to every word

Sudoku linked a cell to its row, column and box. A language model does not know in advance which words matter to which, so it links **every pair**: the **complete graph**, each word also linked to itself.



3 links between words,
3 links of a word to itself:
3 · 3 = 9 attention weights

SO **n words give $n \cdot n$ links; which links matter must be decided per sentence.**

NEXT Which of the three message rules decides links per input?

Self-attention is graph attention on that graph

Graph attention (p. 53) scored each neighbour, turned scores into weights with softmax, and summed. Self-attention is the same three steps with vectors and three learned matrices.

step	graph attention (Part 3)	self-attention (Transformer)
score	$e_{ij} = a \cdot (s_i + s_j)$	$e_{ij} = (W_Q h_i) \cdot (W_K h_j)/\sqrt{d}$
weights	$\alpha_{ij} = \mathrm{softmax}_j(e_{ij})$	$\alpha_{ij} = \mathrm{softmax}_j(e_{ij})$, **over all n words**
message M_θ	$\alpha_{ij} \cdot w \cdot s_j$	$\alpha_{ij} \cdot W_V h_j$
update U_θ	**relu of the sum**	$h_i +$ **sum, then a small network per word**

Symbols: h_i, word i's vector of d numbers; W_Q, W_K, W_V: learned matrices (query, key, value), the θ of this layer. Several heads = several such layers side by side.

SO **A Transformer is a graph neural network on the complete graph: the same template, start, T layers, readout (p. 36). This is the standard view [Transformers as GNNs].**

NEXT What do today's five answers say about it?

Today's five answers, read on the Transformer

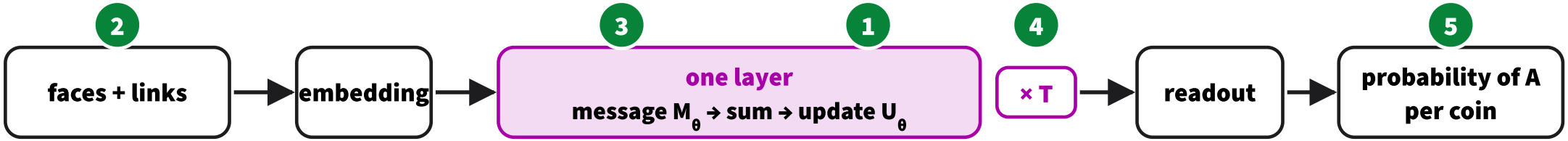
part	on the coin chain	on the complete graph of words
1 learn	$\theta = (u, w)$ **from solved chains**	W_Q, W_K, W_V **from text, predicting the next word**
2 symmetry	**outputs follow the coins** (p. 45)	**outputs follow the words: "dog bites man" = "man bites dog" unless the position is an input: positional encoding** (p. 47)
3 weights	**learned weights read the scores**	**attention weights read the vectors: the links are chosen per sentence**
4 depth	T **layers reach** T **links**	**every word is 1 link from every word: one layer reaches all; the cost is** n^2 **weights per layer**
5 limit	**equal computation trees, equal outputs**	**without positions, two words with the same neighbours are identical; positions break the tie**

SO **Every result of today applies; Lecture 12 builds the Transformer from here.**

NEXT What should we remember?

One network, five results

Each part answered one question about the **same learned network**.



1. **What is one layer, and how do we learn it?** → learned start, message, update: embedding, M_θ, U_θ
2. **Does one trained network fit any numbering and size?** → yes: a sum ignores order, so one θ fits every graph
3. **Which message and sum should a layer use?** → weigh by degree, sample neighbours, or learn the weights
4. **How many layers does a coin need?** → T layers see T links; too many make all nodes alike
5. **Which graphs can it never tell apart?** → graphs whose nodes see the same computation trees

then 6: all five results on one small network · 7: one application, Sudoku, and the Transformer

Derive the messages when the factors are known; otherwise, learn them from examples. A Transformer is this network on the complete graph.

Check yourself (1 of 3): Parts 1 and 2

1. Belief propagation starts from the face probabilities. In a graph neural network, which learned part replaces this start?

the embedding | the message M_θ | the readout

Answer. **The embedding:** the learned start; for us the score $u \cdot x_i$ from the face.

2. We swap the numbers of two coins of a chain. The graph neural network's output for a given coin ...

stays the same | changes | becomes one half

Answer. **Stays the same:** a sum ignores order, so each output moves with its coin.

Learned parts replace fixed formulas; numbering never matters.

Check yourself (2 of 3): Parts 3 and 4

3. Node 2 has three friends; only friend 1 carries a signal. Which network can give friend 1 the largest weight?

graph convolution | GraphSAGE | graph attention

Answer. **Graph attention:** its weights are computed from the scores; the others use only degrees or sample size.

4. We go from 4 to 6 layers on 20-coin chains. The floor, T rounds of belief propagation, goes …

up | down | nowhere

Answer. **Down:** each coin sees two more links, which can only help; 0.0085 to 0.0024.

Learned weights pick useful neighbours; more layers see further.

Check yourself (3 of 3): Part 4's cost and Part 5

5. We apply 40 averaging layers without parameters. The coins' scores become …

more different | more alike | unchanged

Answer. **More alike:** each average mixes neighbours; the spread fell from 2 to 0.083.

6. Can more layers separate a ring of six from two triangles, all faces H?

yes, at six layers | yes, with wider vectors | no, never

Answer. **No:** every coin's computation tree is the same at every depth.

Depth makes nodes alike; equal trees stay equal.

WHAT'S NEXT

Next: a toy weather model, then amortized inference

PROBLEM One network handles every chain at once.

WE DO Friday: a small weather model. Lecture 10: this idea, named.

WHY Training once serves **every new input**.



SO **A network that outputs a posterior for any data is amortized inference.**

NEXT When is such a network good enough?

References: messages, graph neural networks and their limits

[Belief propagation] F. R. Kschischang, B. J. Frey and H.-A. Loeliger. Factor graphs and the sum-product algorithm. *IEEE Transactions on Information Theory* 47(2):498–519, 2001.

[Sudoku] R. B. Palm, U. Paquet and O. Winther. Recurrent relational networks. *Advances in Neural Information Processing Systems* 31, 2018.

[Message-passing networks] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals and G. E. Dahl. Neural message passing for quantum chemistry. *International Conference on Machine Learning* 34, *Proceedings of Machine Learning Research* 70:1263–1272, 2017.

[Graph convolutional network] T. N. Kipf and M. Welling. Semi-supervised classification with graph convolutional networks. *International Conference on Learning Representations*, 2017.

[GraphSAGE] W. L. Hamilton, R. Ying and J. Leskovec. Inductive representation learning on large graphs. *Advances in Neural Information Processing Systems* 30, pp. 1024–1034, 2017.

[Graph attention network] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò and Y. Bengio. Graph attention networks. *International Conference on Learning Representations*, 2018.

[Weisfeiler–Leman test] B. Weisfeiler and A. Leman. The reduction of a graph to canonical form and the algebra which appears therein. *Nauchno-Technicheskaya Informatsia*, Series 2, 9:12–16, 1968.

[Expressive power] K. Xu, W. Hu, J. Leskovec and S. Jegelka. How powerful are graph neural networks? *International Conference on Learning Representations*, 2019. · C. Morris, M. Ritzert, M. Fey, W. L. Hamilton, J. E. Lenssen, G. Rattan and M. Grohe. Weisfeiler and Leman go neural: higher-order graph neural networks. *AAAI Conference on Artificial Intelligence* 33:4602–4609, 2019.

[Oversmoothing; over-squashing] Q. Li, Z. Han and X.-M. Wu. Deeper insights into graph convolutional networks for semi-supervised learning. *AAAI Conference on Artificial Intelligence* 32:3538–3545, 2018. · U. Alon and E. Yahav. On the bottleneck of graph neural networks and its practical implications. *International Conference on Learning Representations*, 2021.

References: graph networks in today's AI

[Drug discovery] J. M. Stokes, K. Yang, K. Swanson, W. Jin, A. Cubillos-Ruiz, N. M. Donghia, C. R. MacNair, S. French, L. A. Carfrae, Z. Bloom-Ackermann, et al. A deep learning approach to antibiotic discovery. *Cell* 180(4):688–702, 2020.

[Travel times] A. Derrow-Pinion, J. She, D. Wong, O. Lange, T. Hester, L. Perez, M. Nunkesser, S. Lee, X. Guo, et al. ETA prediction with graph neural networks in Google Maps. *ACM International Conference on Information and Knowledge Management* (CIKM), pp. 3767–3776, 2021.

[Recommendations] R. Ying, R. He, K. Chen, P. Eksombatchai, W. L. Hamilton and J. Leskovec. Graph convolutional neural networks for web-scale recommender systems. *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 974–983, 2018.

[Weather] R. Lam, A. Sanchez-Gonzalez, M. Willson, P. Wirnsberger, M. Fortunato, F. Alet, S. Ravuri, T. Ewalds, Z. Eaton-Rosen, W. Hu, et al. Learning skillful medium-range global weather forecasting. *Science* 382(6677):1416–1421, 2023.

[Protein structure] J. Jumper, R. Evans, A. Pritzel, T. Green, M. Figurnov, O. Ronneberger, K. Tunyasuvunakool, R. Bates, A. Žídek, A. Potapenko, et al. Highly accurate protein structure prediction with AlphaFold. *Nature* 596(7873):583–589, 2021.

[Attention] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser and I. Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems* 30, pp. 5998–6008, 2017.

[Transformers as GNNs] C. K. Joshi. Transformers are graph neural networks. *The Gradient*, September 2020. · M. M. Bronstein, J. Bruna, T. Cohen and P. Veličković. Geometric deep learning: grids, groups, graphs, geodesics, and gauges. arXiv:2104.13478, 2021, Section 5.4. · J. Leskovec. Graph Transformers. Stanford CS224W lecture 14, 2023.

[Training] D. P. Kingma and J. Ba. Adam: a method for stochastic optimization. *International Conference on Learning Representations*, 2015.

Thank you!

Friday 6 November: the lab builds a small weather model on a mesh.

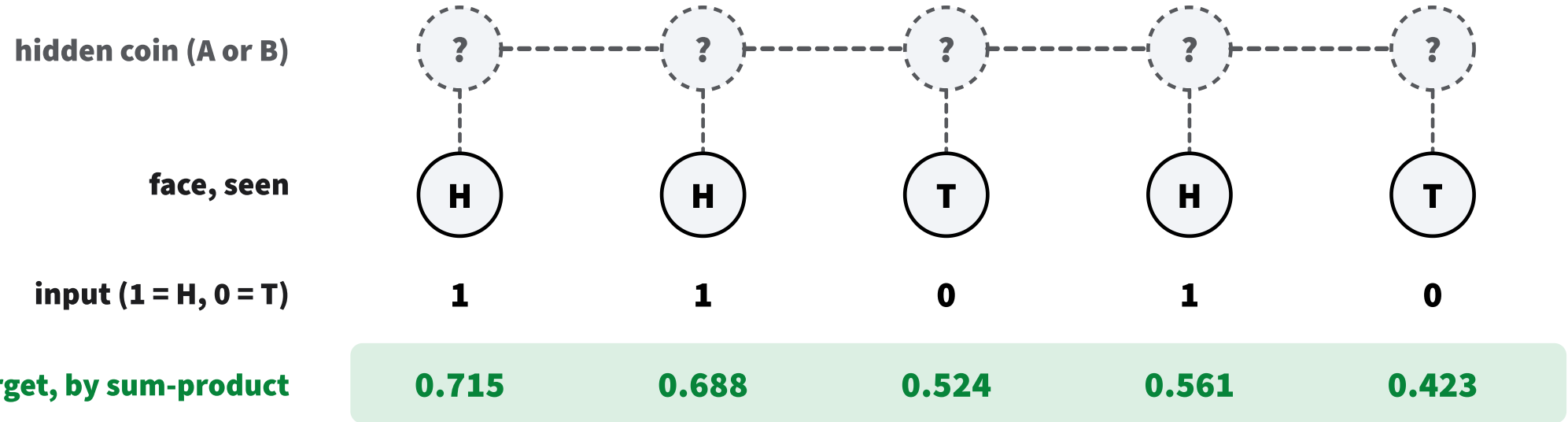
Appendix: the derivations behind each section.

Each example is a random chain with its exact answers

PROBLEM The network needs examples of the **right answer**.

WE DO Draw 3 to 8 coins, faces H or T with chance $\frac{1}{2}$.

WHY Sum-product gives **exact targets on chains (Lecture 2, p. 169)**.



SO **Input x_i: coin i's face. Target p_i: the exact chance that coin i is A.**

NEXT Which network reads such a chain?

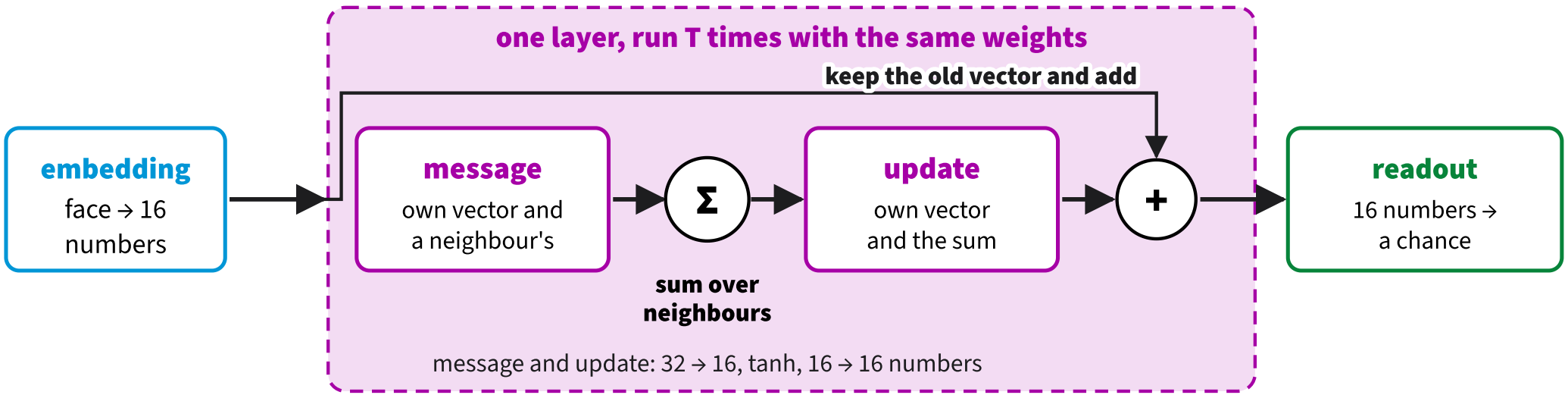
Our layer adds its update to the old vector

PROBLEM Which exact network do we train (p. 35)?

WE DO One message and one update rule for all layers.

WHY Adding keeps the old vector (p. 68).

> $h_i \leftarrow h_i + U_\theta(h_i, a_i)$: h_i, coin i's 16 numbers; a_i, the sum of neighbours' messages; U_θ, the update network.



SO **A readout turns them into a chance (p. 114).**

NEXT How many numbers must training choose?

Training must choose 1 649 numbers

PROBLEM How large is the network, and why 16 numbers per coin?

WE DO Count the weights and biases of each part (p. 114).

WHY A belief-propagation message is **one number** (p. 123); 16 leave room.

part	what it reads and writes	numbers
embedding	**the face → 16 numbers**	**32**
message rule	**two coins' vectors → 16 numbers**	**800**
update rule	**own vector and the sum → 16 numbers**	**800**
readout	**16 numbers → a chance**	**17**
total	**all four parts**	**1 649**

SO **1 649 numbers for any chain and any T.**

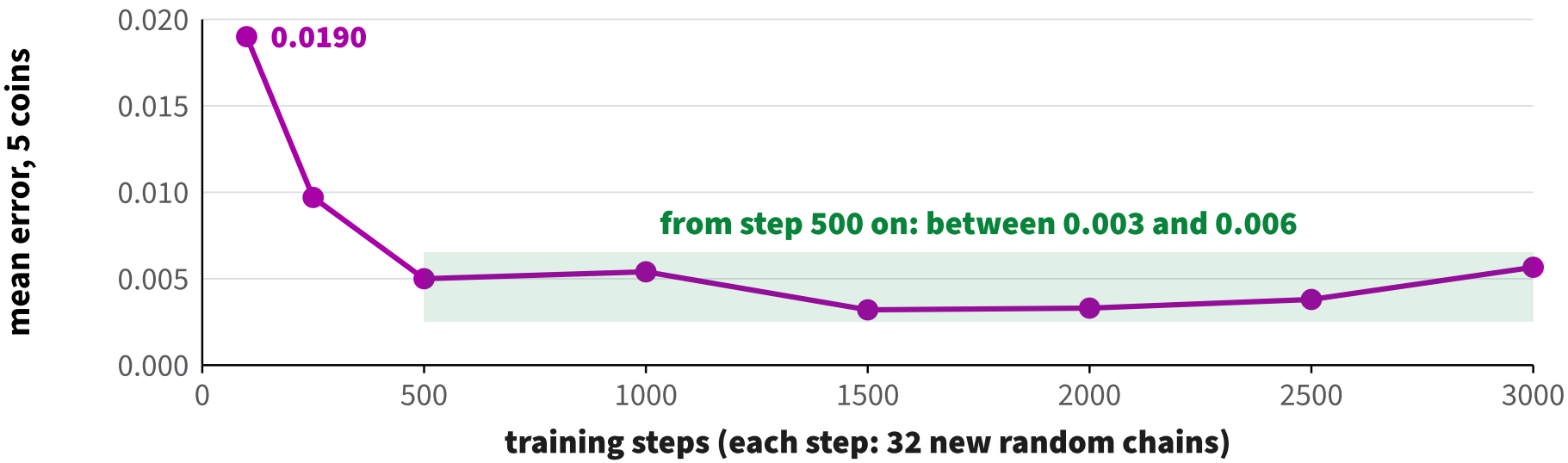
NEXT How many training steps are enough?

The test error stops falling after about 500 steps

PROBLEM How many **Adam** steps (gradient descent, one step size per weight)?

WE DO Cross-entropy (p. 37), 32 new chains per step, learning rate 0.003.

WHY Fresh chains: nothing to **memorise**.



SO **3000 steps, six times what is needed: 96 000 chains** (p. 121)**.**

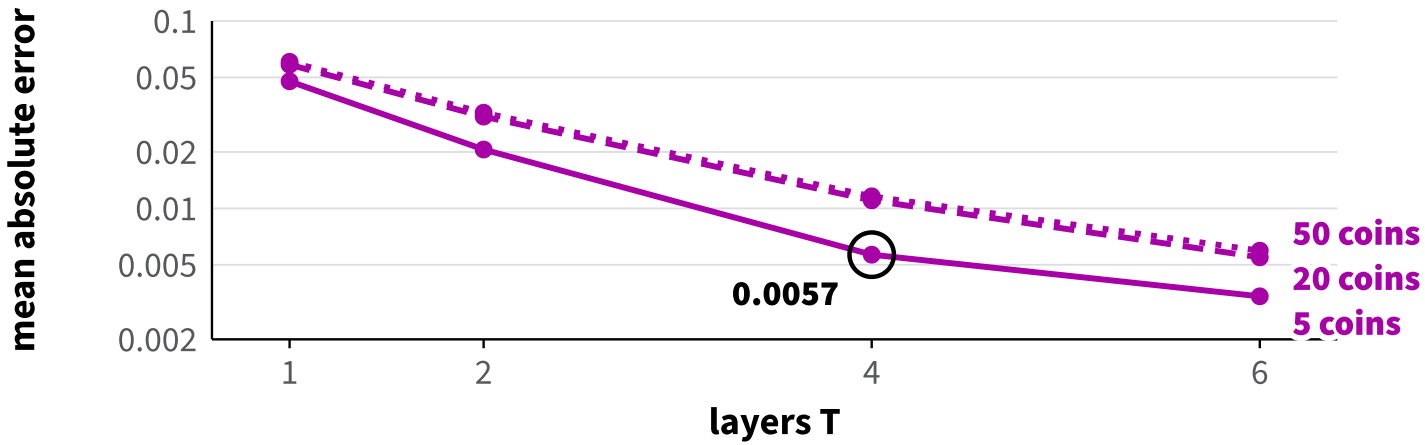
NEXT How do layers and chain length matter?

Layers set the error; chain length barely changes it

PROBLEM Does it work on longer chains?

WE DO Measure the error on 400 test chains (p. 122).

WHY Lengths test the **shared rule**.



4 layers, 5 coins: what causes the error 0.0057?

coins out of reach | imperfect training | rounding

Answer. Imperfect training: 4 links reach all 5 coins, so the floor is 0.

SO **Within reach, all error is training error.**

NEXT Which exact network was trained?

The network of the example, layer by layer

↳ $d = 16$. $\mathrm{Lin}_{k \to l}(v) = W \cdot v + c$: W has $k \cdot l$ weights, c has l biases. $[u, v]$: the two vectors stacked. One M_θ and one U_θ for all T layers.

$$h_i^{(0)} = \mathrm{Lin}_{1 \to 16}(x_i)$$

embedding: $16 + 16 = 32$

$$m_{j \to i} = \mathrm{Lin}_{16 \to 16}\big(\tanh(\mathrm{Lin}_{32 \to 16}[h_i, h_j])\big)$$

message: $528 + 272 = 800$

$$h_i \leftarrow h_i + \mathrm{Lin}_{16 \to 16}\big(\tanh(\mathrm{Lin}_{32 \to 16}[h_i, a_i])\big)$$

update, $a_i = \sum_{j \in N(i)} m_{j \to i}$**: 800**

$$\hat{y}_i = \sigma\big(w \cdot h_i^{(T)} + b\big)$$

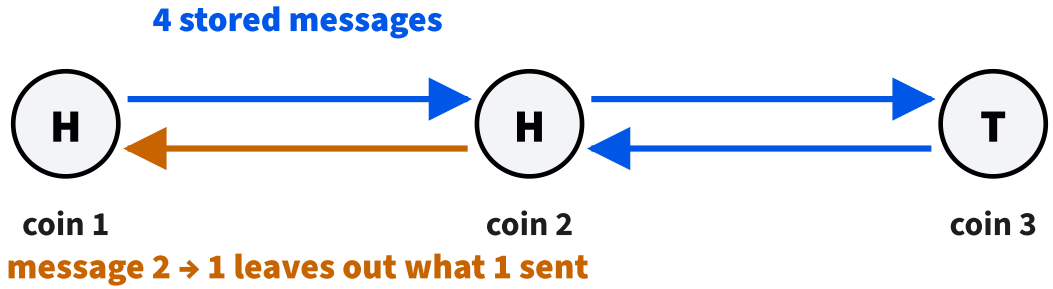
readout: $16 + 1 = 17$

$32 + 800 + 800 + 17 = 1\,649$ **numbers, whatever** T **and the graph.**

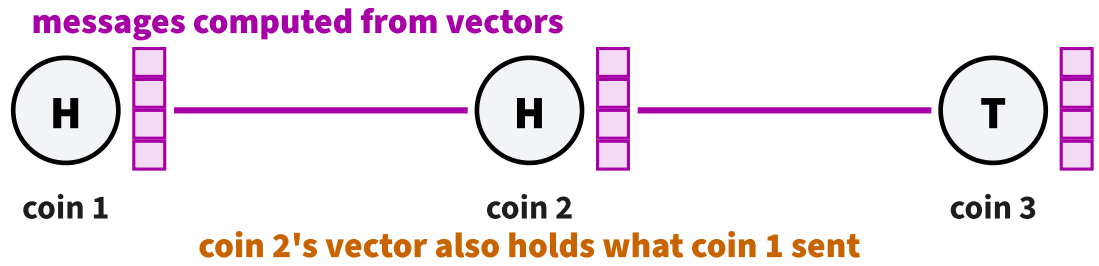
One list per coin must hold more than two chances

Belief propagation stores a message for each direction of a link; a graph neural network stores only **one list per coin**.

belief propagation
one message per direction



graph neural network
one vector per coin



SO **Coin 2's list also holds what coin 1 sent, so it needs room. Its length is our choice; ours is 16** (p. 119)**.**

NEXT How do we write one round of the network?

On our chain, the network misses each answer by under 0.01

We give the trained network the faces H H T H T and compare its chance of A for each coin with the exact one. Coin 1: 0.712 against 0.715, a gap of only 0.003.

coin	1: H	2: H	3: T	4: H	5: T
exact (belief propagation)	0.7155	0.6879	0.5244	0.5605	0.4225
network, 4 layers	0.7122	0.6803	0.5145	0.5531	0.4170
difference	0.0033	0.0076	0.0099	0.0074	0.0055

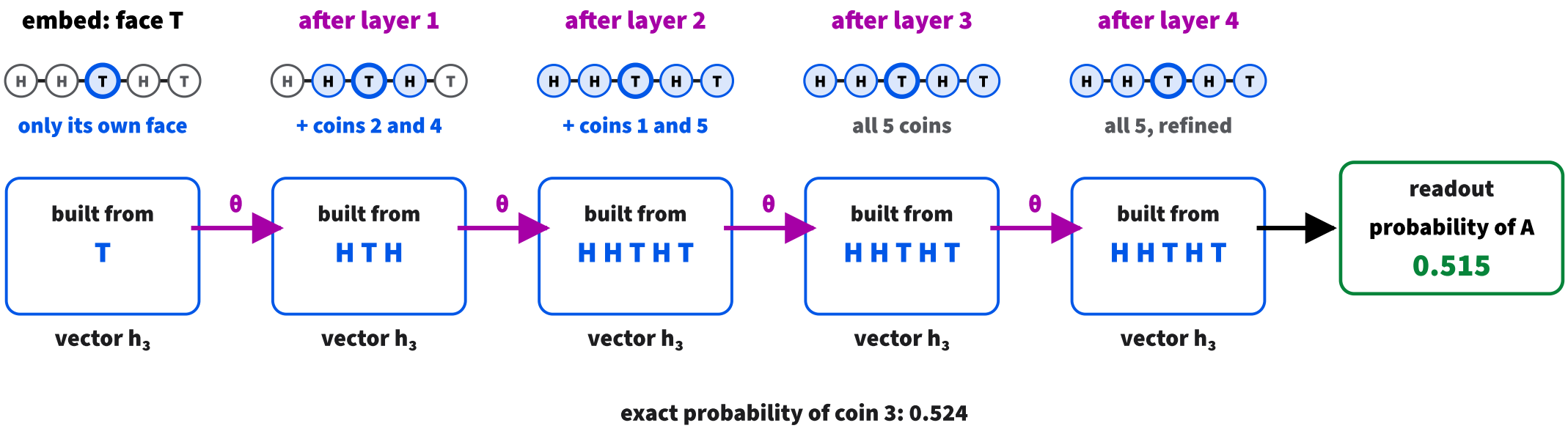
mean difference over the five coins: 0.0067

SO **The largest gap is 0.0099 (coin 3), although the network never saw the coin model itself.**

NEXT How does the network get coin 3's 0.515?

Inside the network: each layer, coin 3 hears one coin further

Follow coin 3's list h_3 through the network (p. 36). The embedding starts it from the face T; each layer adds what the two **neighbours** send (p. 35).



SO **After two layers the list holds all five faces, as in belief propagation (p. 31); layers 3 and 4 refine them. The readout gives 0.515.**

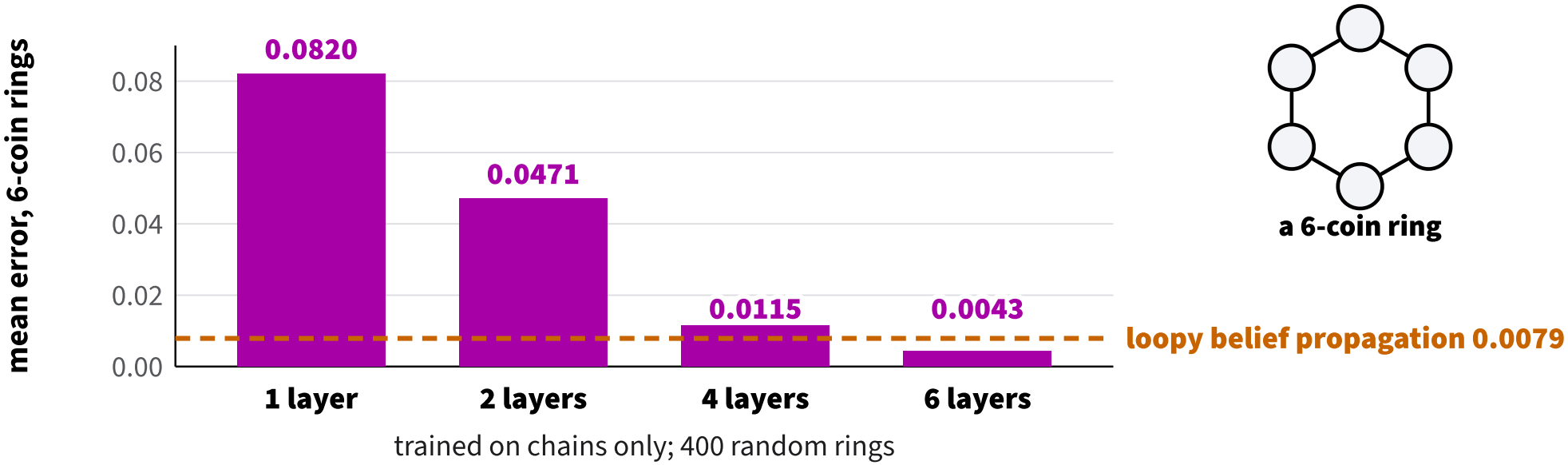
NEXT Where else do learned messages already help?

On 6-coin rings, six layers beat loopy belief propagation

PROBLEM Training saw no loops.

WE DO Test on 400 random 6-coin rings; exact answers by listing all 64 cases.

WHY Loopy messages **count evidence twice** (Lecture 6, p. 49).



SO **Six layers err by 0.0043, less than loopy belief propagation's 0.0079.**

NEXT Which extra input reveals the loop?

Why 16 numbers: we tried lists of 1 to 16

PROBLEM A coin must keep what it hears from both sides; one chance is little room.

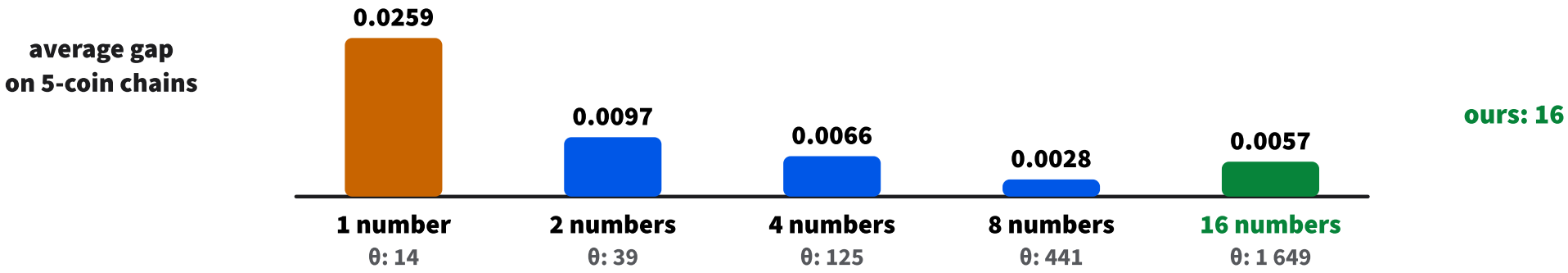
WE DO Give every coin a **list of numbers**, its vector; the embedding picks the first list from its face.

WHY The numbers are set by training (part of θ); only their count is our choice. We try 1 to 16.

H → (0.223, −0.536, −0.116, …) 16 numbers

T → (−0.001, −0.223, −0.088, …) 16 numbers

both lists are learned: they are part of θ



SO **One number gives the largest gap; 2 to 16 all stay below 0.01. The length is our choice, like the number of layers; we use 16.**

NEXT What does one layer do to coin 3's list?

Layer 1 on coin 3: two messages, one sum, one change

Coin 3 shows T; coins 2 and 4 show H, so they start with H's list. The first 3 of 16 numbers are shown.

coin 3 now: list of face T	(−0.001, −0.223, −0.088, …)	h_3
message from coin 2 (face H)	(−0.286, 0.090, 0.244, …)	$M_\theta(h_3, h_2)$
message from coin 4 (face H)	(−0.286, 0.090, 0.244, …)	$M_\theta(h_3, h_4)$: the same, both are H
sum of the two messages	(−0.572, 0.180, 0.488, …)	add number by number
change	(0.054, −0.175, −0.351, …)	$U_\theta(h_3, \text{sum})$
coin 3 after layer 1	(0.053, −0.398, −0.439, …)	= now + change

$$h_3' = h_3 + U_\theta\big(h_3,\ M_\theta(h_3, h_2) + M_\theta(h_3, h_4)\big)$$

SO **One layer = messages, their sum, a change added to the list (p. 35).**

NEXT How does one gradient step reach all 1 649 numbers?

Shared weights collect gradients from every layer

↳ Each step: 32 chains of 3 to 8 coins, faces H or T with chance $\frac{1}{2}$, targets p_i by sum-product; stacked as one graph of n coins. Adam, learning rate 0.003, 3000 steps.

$$\mathcal{L}(\theta) = \frac{1}{n}\sum_{i=1}^{n} -\left[p_i \cdot \ln \hat{y}_i + (1 - p_i) \cdot \ln(1 - \hat{y}_i)\right]$$

mean cross-entropy

$$\theta \text{ used in layers } 1, \ldots, T: \quad \theta^{(1)} = \cdots = \theta^{(T)} = \theta$$

unroll: one copy per layer

$$\frac{\partial \mathcal{L}}{\partial \theta} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}}{\partial \theta^{(t)}}$$

chain rule over the copies

$$\frac{\partial \mathcal{L}}{\partial h^{(t-1)}} = \frac{\partial \mathcal{L}}{\partial h^{(t)}} \cdot \frac{\partial h^{(t)}}{\partial h^{(t-1)}}$$

backward, layer T down to 1

Backpropagation through the unrolled layers; each layer adds its share.

Test errors of the trained networks, in one table

↳ Mean absolute error $|\hat{y}_i - p_i|$ over coins and test graphs: 400 random graphs, 100 at 50 coins. Networks trained on chains of 3 to 8 coins only. Floor: T rounds of belief propagation (p. 63).

layers T	5 coins	20 coins	50 coins	6-coin rings	floor, 20 coins
1	0.0477	0.0585	0.0606	0.0820	0.0577
2	0.0206	0.0309	0.0324	0.0471	0.0307
4	0.0057	0.0110	0.0116	0.0115	0.0085
6	0.0034	0.0055	0.0059	0.0043	0.0024

Loopy belief propagation on the same 6-coin rings: 0.0079.

A belief-propagation message is one number per link

↳ Values A, B. $\Phi(z) = \varphi_j(z) \cdot \prod_{k \in N(j)\setminus\{i\}} \mu_{k\to j}(z)$ (p. 138); $\psi = \kappa$ if equal, else $1-\kappa$. Log-ratios: $\Delta_{j\to i} = \ln \frac{\mu_{j\to i}(A)}{\mu_{j\to i}(B)}$, $\Delta_j = \ln \frac{\varphi_j(A)}{\varphi_j(B)}$.

$$e^{\Delta_{j\to i}} = \frac{\kappa\cdot\Phi(A) + (1-\kappa)\cdot\Phi(B)}{(1-\kappa)\cdot\Phi(A) + \kappa\cdot\Phi(B)}$$

sum over coin j

$$= \frac{\kappa\cdot e^{r} + 1 - \kappa}{(1-\kappa)\cdot e^{r} + \kappa}, \quad r = \Delta_j + \sum_{k\in N(j)\setminus\{i\}} \Delta_{k\to j}$$

divide by $\Phi(B)$

$$\tanh\tfrac{\Delta_{j\to i}}{2} = (2\kappa - 1)\cdot\tanh\tfrac{r}{2}$$

$\tanh\frac{x}{2} = \frac{e^x-1}{e^x+1}$

$$p(z_i = A \mid x) = \sigma\Big(\Delta_i + \sum_{j\in N(i)} \Delta_{j\to i}\Big)$$

belief; σ, the sigmoid

A fixed function on each link, a sum at each node, a sigmoid: the network's template.

Proof: on a tree, no T-layer network beats T rounds of BP

↳ **Setting.** A tree with the coin model; $W = N_T(i)$, the coins within T links of i; x_W their faces, x all faces. Target $p_i = p(z_i = \mathrm{A} \mid x)$ **(p. 37)**. For an output $\hat{y}_i \in (0,1)$ the cross-entropy loss is $\mathcal{L}_i = -p_i \ln \hat{y}_i - (1-p_i)\ln(1-\hat{y}_i)$; the claim is about its expectation over random chains.

(1) $\hat{y}_i = g(x_W)$ for some function g

receptive-field theorem **(p. 62)**

(2) $\mathbb{E}[\mathcal{L}_i \mid x_W] = -\bar{p}\ln g - (1-\bar{p})\ln(1-g), \quad \bar{p} = \mathbb{E}[p_i \mid x_W]$

$\mathcal{L}_i$ is linear in p_i; g is fixed given x_W

$$\frac{d}{dg} = -\frac{\bar{p}}{g} + \frac{1-\bar{p}}{1-g} = 0 \;\Rightarrow\; g = \bar{p}; \qquad \frac{d^2}{dg^2} > 0$$

unique minimum (Gibbs' inequality)

$$\bar{p} = \mathbb{E}\big[p(z_i = \mathrm{A} \mid x) \,\big|\, x_W\big] = p(z_i = \mathrm{A} \mid x_W) =: y_i$$

tower rule

(3) $y_i = T$ rounds of belief propagation at i

tree: next page; this is where loops break the proof

Hence $\mathbb{E}[\mathcal{L}_i] = \mathbb{E}\big[\mathbb{E}[\mathcal{L}_i \mid x_W]\big] \geq$ the same with $g = y_i$. The gap we measure, $|\hat{y}_i - y_i|$, is a different quantity: the floor is proven for cross-entropy only.

layers T	1	2	3	4	6
our two-parameter network, 20 coins (gap)	0.0585	0.0454	0.0312	0.0279	0.0249
floor: T rounds of belief propagation (gap)	0.0584	0.0307	0.0161	0.0085	0.0023

On a chain, T rounds of messages see T links exactly

↳ Chain coin model; window W: coins within T links of coin i. Messages start flat (all ones). Far coins sum out to flat messages: the coin chain's stationary distribution is uniform.

$$p(z, x_W) = \tfrac{1}{2} \cdot \prod_{\text{links}} \psi(z_j, z_{j+1}) \cdot \prod_{j \in W} \varphi_j(z_j)$$

faces outside W unused

$$\sum_{z_j} \psi(z_{j-1}, z_j) = \kappa + (1 - \kappa) = 1$$

sum out the last unseen coin

$$\sum_{z_1} \tfrac{1}{2} \cdot \psi(z_1, z_2) = \tfrac{1}{2}$$

left end: still a fair start

$$p(z_W, x_W) = \tfrac{1}{2} \cdot \prod_{\text{links in } W} \psi \cdot \prod_{j \in W} \varphi_j$$

the chain on W alone

$$\text{round } T \text{ at } i = \text{exact sum-product on } W$$

a flat message = a summed-out coin

Check: $T = 2$, coin 1 of H H T H T gives 0.704, the exact answer for H H T alone.

Repeated averaging keeps only the top eigenvector

↳ $\hat{A}$ symmetric: orthonormal eigenvectors u_k, eigenvalues $1 = \lambda_1 > |\lambda_k|$ for $k \geq 2$ on a connected graph with self-loops (9-coin chain: $\lambda_2 = 0.953$, smallest -0.306); $u_1 \propto \tilde{D}^{1/2}\mathbf{1}$, since $\hat{A}\,\tilde{D}^{1/2}\mathbf{1} = \tilde{D}^{-1/2}(A+I)\mathbf{1} = \tilde{D}^{1/2}\mathbf{1}$.

$$h^{(t)} = \hat{A}^t h = \sum_k \gamma_k \cdot \lambda_k^t \cdot u_k, \quad \gamma_k = u_k \cdot h$$

expand h in eigenvectors

$$= \gamma_1 \cdot u_1 + \sum_{k \geq 2} \gamma_k \cdot \lambda_k^t \cdot u_k$$

$\lambda_1^t = 1$

$$\left| \frac{(\hat{A}^t h)_i}{\sqrt{\tilde{d}_i}} - c \right| \leq |\lambda_2|^t \cdot \|h\|$$

c the same for all i; $\tilde{d}_i \geq 1$

step t	0	1	2	5	10	20	40
degree-corrected spread, 9-coin chain	1.284	0.835	0.631	0.406	0.223	0.125	0.048
plain mean (main slide); its matrix is similar to $\hat{A}$, same λ_2	2.000	1.333	1.000	0.658	0.374	0.216	0.083

After degree correction all nodes approach one value: the spread falls like $|\lambda_2|^t$.

A sum of well-chosen features tells every multiset apart

↳ Features from a countable set $\mathcal{X}$; multisets X of fewer than N elements. Goal: f with $X \mapsto \sum_{x \in X} f(x)$ one-to-one, so a sum can act as the colour test's hash.

$$Z : \mathcal{X} \to \{1, 2, 3, \dots\} \text{ one-to-one}$$

number the features

$$f(x) = N^{-Z(x)}$$

one digit place per feature

$$\sum_{x \in X} f(x) = \sum_{z} \text{count}_X(z) \cdot N^{-z}$$

digit z = count of feature z

$$\text{count}_X(z) < N \Rightarrow \text{base-}N \text{ digits are unique}$$

different multisets, different sums

Guarantee (Xu, Hu, Leskovec, Jegelka 2019). For features from a countable set, the graph isomorphism network (GIN), sum plus a one-to-one update, is as strong as the Weisfeiler–Leman (WL) test.

Mean and max fail: $\{1, 1\}$ and $\{1\}$ share both; only the sum, 2 against 1, differs.

The normalised matrix has eigenvalues between −1 and 1

↳ $\hat{A} = \tilde{D}^{-1/2}(A + I)\tilde{D}^{-1/2}$; $\tilde{D}$ holds the degrees counted with self-loops. $R = \tilde{D}^{-1}(A + I)$ averages each node over itself and its neighbours.

$$\hat{A} = \tilde{D}^{1/2} \cdot R \cdot \tilde{D}^{-1/2}$$

similar matrices: same eigenvalues

$$R_{ij} \geq 0, \qquad \sum_j R_{ij} = 1$$

each row is an average

$$Rv = \lambda v,\ |v_i| = \max_j |v_j| : \quad |\lambda| \cdot |v_i| \leq \sum_j R_{ij} \cdot |v_j| \leq |v_i|$$

an average never exceeds its largest entry

$$\Rightarrow\ |\lambda| \leq 1;\ \ \hat{A}\ \text{symmetric}\ \Rightarrow\ \lambda \in [-1, 1]$$

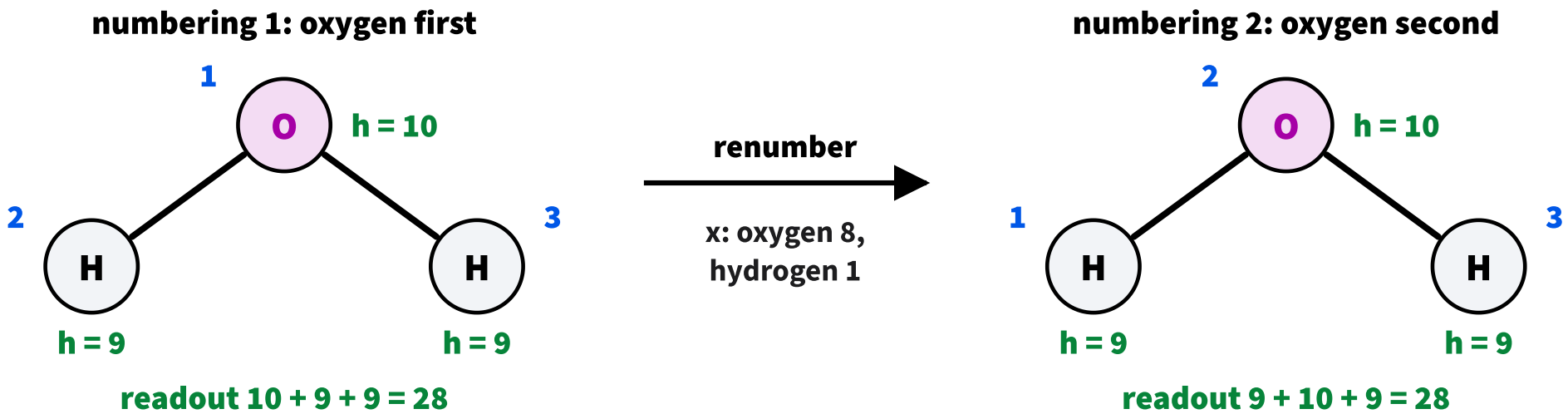
real eigenvalues, size at most 1

Repeated multiplication by $\hat{A}$ never blows up.

A molecule's property ignores how its atoms are numbered

A physics calculation of one molecule takes **hours**; chemists screen millions with a **message-passing network**.

COIN CHAIN		MOLECULE
coin, link	→	atom, bond
face x_i	→	atom type (atomic number)
one answer per coin	→	one property per molecule



SO **Water, one layer $h_i = x_i + \sum_{j \in N(i)} x_j$, then a sum: 28 in both numberings.**

NEXT How do atoms send messages?

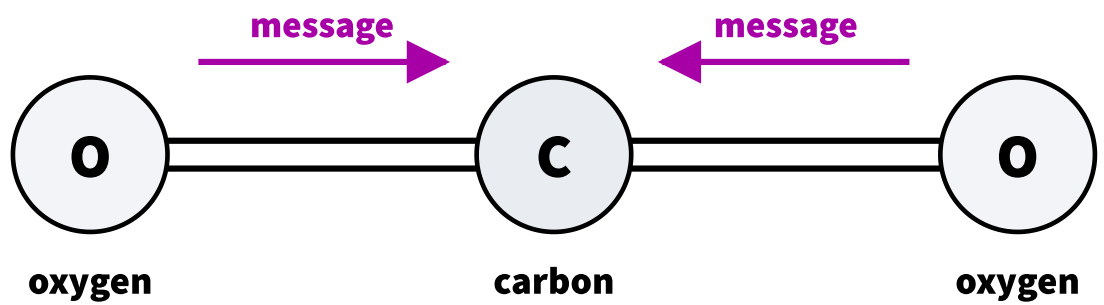
A molecule is a graph: atoms send messages along bonds

PROBLEM Predict molecule properties without slow simulation.

WE DO Atoms are nodes, bonds are links; sum atom vectors.

WHY Properties depend on **which atoms are bonded**.

COIN CHAIN		MOLECULE
coin and its face	↔	atom and its element
link	↔	chemical bond



carbon's sum = message from the left oxygen + message from the right oxygen

SO **One network for molecules of any size; used to screen molecules for new antibiotics.**

NEXT Can a road map be read the same way?

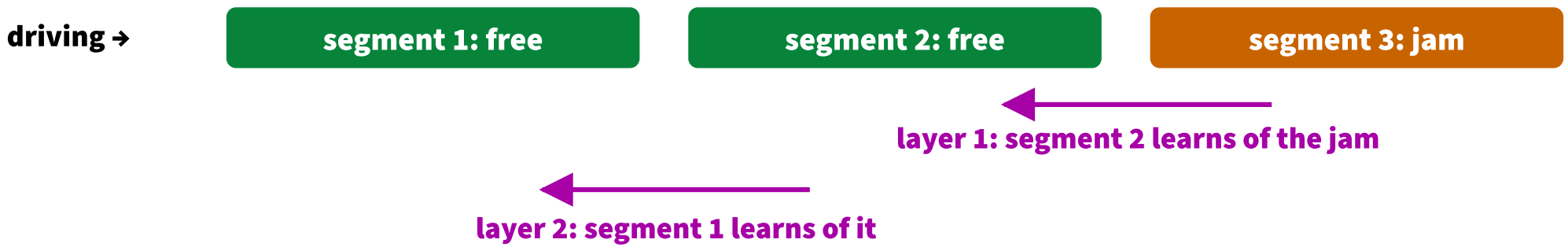
Road segments pass traffic news to their neighbours

PROBLEM Predict a route's travel time from current traffic.

WE DO Road segments are nodes; joining segments are linked.

WHY A jam ahead slows you later: news must **travel**.

COIN CHAIN		ROAD MAP
coin and its face	↔	segment and its speed
T layers, T links	↔	news from T segments away



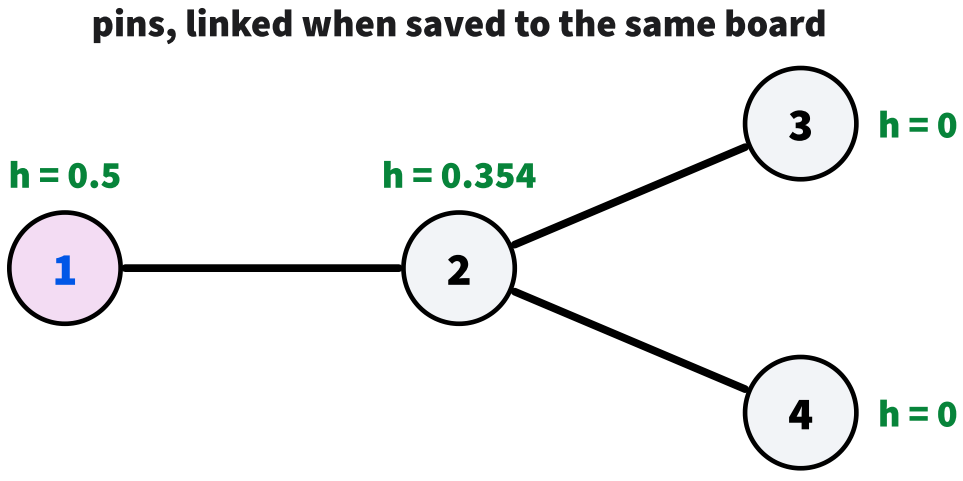
SO **Google Maps uses such a network for arrival times.**

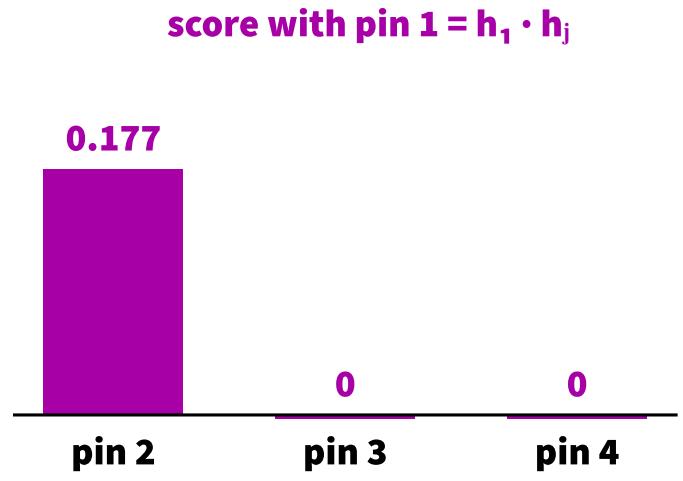
NEXT Can a graph suggest pictures to a user?

Pinterest recommends pins with sampled neighbourhoods

Suggest related pins among **billions** of saved images, with **GraphSAGE-style layers**.

COIN CHAIN		RECOMMENDATION
coin	→	pin (a saved image)
link	→	saved to the same board
answer per coin	→	score for a pair of pins





SO **With Step 1's values (p. 84), pin 2 scores highest with pin 1: recommend pin 2.**

NEXT How can a network see far on a mesh?

Weather networks add long links to see far

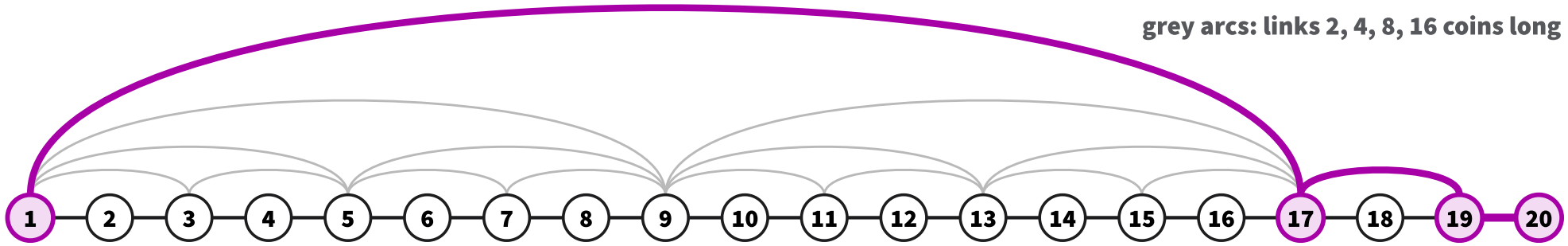
PROBLEM Weather depends on far air.

WE DO Add long mesh links.

WHY Few rounds cross the globe.

OUR COIN CHAIN		WEATHER NETWORK
coin	↔	**mesh point on the globe**
neighbour link	↔	**finest-mesh link**
links 2, 4, 8, 16 coins long	↔	**coarser-mesh links**

coin 1 → 17 → 19 → 20: 3 rounds



SO **Coin 1 to coin 20: 19 rounds, or 3. Full story: the lab.**

NEXT How well does a real weather model do?

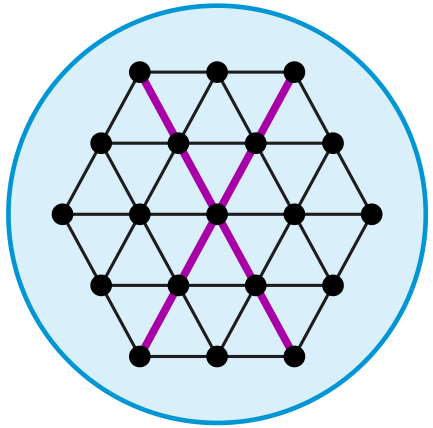
A weather model passes messages on a mesh over the globe

PROBLEM Forecast global weather 10 days ahead in under a minute.

WE DO Mesh points pass messages, with long links (p. 133).

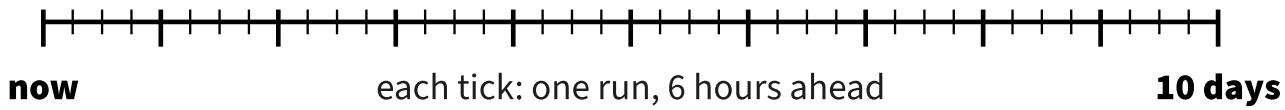
WHY Winds carry weather from **neighbouring points**.

COIN CHAIN		WEATHER MODEL
coin and its face	↔	point: temperature, wind, …
40 runs	↔	40 runs of 6 hours: 10 days



short links: nearby points

long links: far points in one layer



SO **It beat the leading operational forecast on 90% of 1380 verification targets. Friday's lab: a toy version.**

NEXT Where must networks see rings?

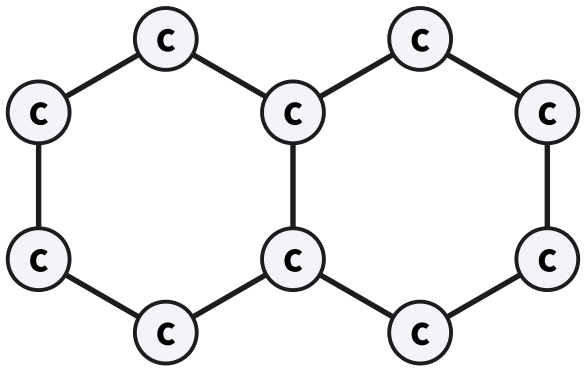
Chemistry needs rings: walk inputs reveal them

PROBLEM Refinement colours both molecules alike.

WE DO Add each atom's chance to walk back.

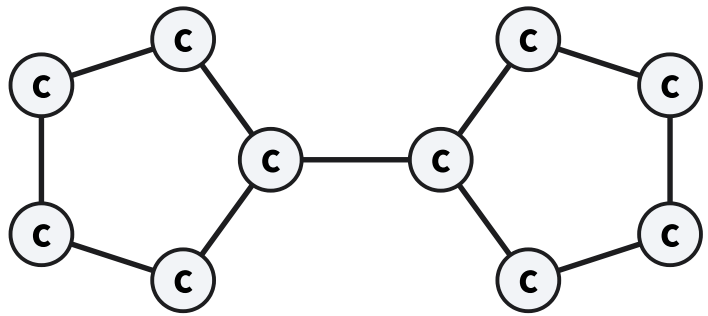
WHY Only odd rings allow odd returns.

decalin: two 6-rings share a bond



back after 5 steps: never

bicyclopentyl: two 5-rings joined



back after 5 steps: possible

OUR GRAPHS		MOLECULES
back in 3 steps: ring never; triangle 2 ways, ⅛ each	↔	back in 5 steps: never, or sometimes
extra input per node	↔	positional encoding (Lecture 12)

SO **One number per node separates what fooled every message-passing network.**

NEXT What if every pair matters?

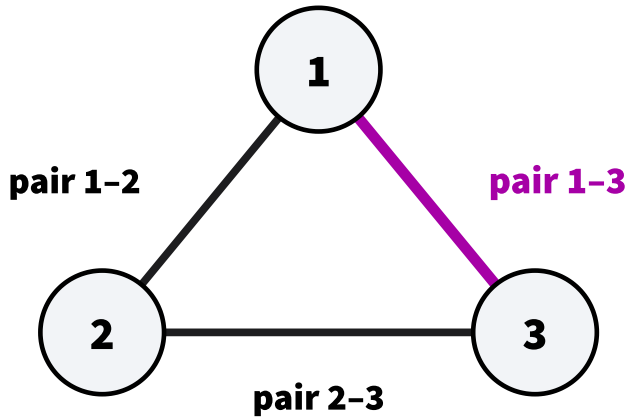
A protein model updates every pair of residues

PROBLEM Predict a protein's three-dimensional shape from its amino acids (**residues**).

WE DO A vector per **pair**; update pair 1–3 from 1–2 and 2–3.

WHY Distances must **fit** in every triangle.

COIN CHAIN		PROTEIN MODEL
a vector per coin	↔	a vector per pair of residues
message along a link	↔	triangle updates and attention



pair 1–3 is updated from pairs 1–2 and 2–3,
for every third residue (here residue 2)

distance 1–3 ≤ distance 1–2 + distance 2–3

SO **Pairs update pairs: near-experimental accuracy on most test proteins.**

NEXT What if all words are linked?

Six more systems, and the part each relies on

For each system: the part of today it relies on, and how.

system	part	how it uses it
molecules	**2** symmetry	atom numbering does not matter; any size
Pinterest	**3** parts	GraphSAGE: sample a few neighbours
Transformer	**3** parts	graph attention on all pairs of words
travel times	**4** depth	T layers carry news T segments
weather	**4** depth	long links: far points in few layers
proteins	**5** limits	a vector per pair, to see triangles

SO **The limits shape the designs: to see far, systems add long links, not many layers.**

NEXT What should we remember?

One round of belief propagation: message, sum, update

PROBLEM On any graph, which parts of a round could a network replace?

WE DO Write the message in logarithms, $\ell_{j\to i} = \ln \mu_{j\to i}$; name each part.

WHY Logarithms turn the product of messages into a **sum**.

$$\mu_{j\to i}(z_i) = \sum_{z_j} \psi(z_i, z_j) \cdot \varphi_j(z_j) \cdot \prod_{k\in N(j)\setminus\{i\}} \mu_{k\to j}(z_j)$$

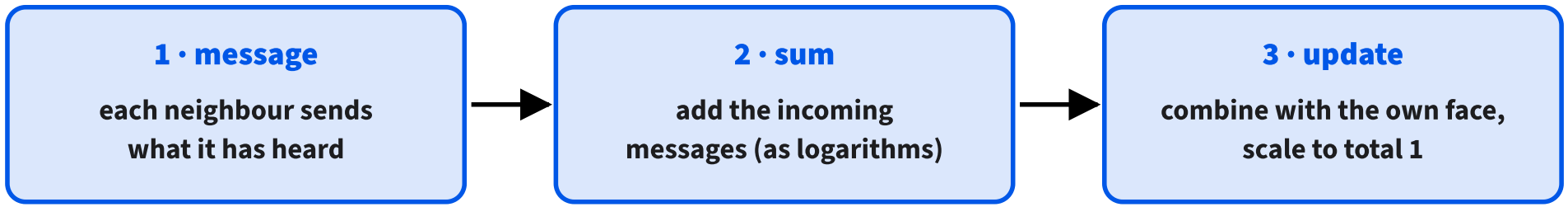
sum-product message
(Lecture 2, p. 169)

$$\ell_{j\to i}(z_i) = \ln \sum_{z_j} \psi(z_i, z_j) \cdot \varphi_j(z_j) \cdot \exp\Big(\sum_{k\in N(j)\setminus\{i\}} \ell_{k\to j}(z_j) \Big)$$

message from what j heard

$$b_i(z_i) \;\propto\; \varphi_i(z_i) \cdot \exp\Big(\sum_{j\in N(i)} \ell_{j\to i}(z_i) \Big)$$

sum the logs, then **update**



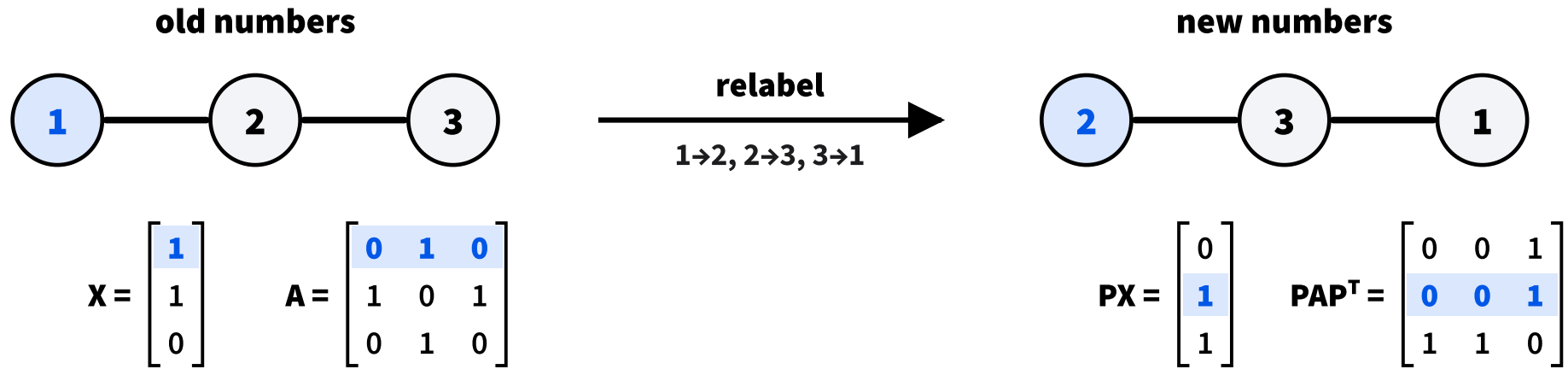
b_i: the belief of node i, its chance of each value; $\propto$: equal after scaling to total 1.

SO **A round is three parts: a message, a sum, an update, all fixed by ψ and φ.**

NEXT What if a network fills in these parts?

Relabelling in matrix form: rows and links move together

↳ The renumbering of (p. 42), written with matrices. Features X: row i is x_i; adjacency A: $A_{ij} = 1$ if i and j are linked.



Definition. A **relabelling** π gives old node i the new number $\pi(i)$. Its **permutation matrix** P has $P_{\pi(i),i} = 1$, other entries 0. Features X become PX; adjacency A becomes $PAP^\top$.

In matrix form. $(PX)_{\pi(i)} = x_i$ and $(PAP^\top)_{\pi(i)\pi(j)} = A_{ij}$. f is **invariant** if $f(PX, PAP^\top) = f(X, A)$; F is **equivariant** if $F(PX, PAP^\top) = P \cdot F(X, A)$, for every P. Theorem (p. 45): $\mathrm{Layer}(PH, PAP^\top) = P \cdot \mathrm{Layer}(H, A)$.

Each node keeps its feature and its links; only its number changes.

Proof: a relabelled node keeps its neighbours and its sum

↳ Relabelled inputs $\tilde{H} = PH$, $\tilde{A} = PAP^\top$ (p. 139); old node i is new node $\pi(i)$.

$$\tilde{h}_{\pi(i)} = h_i, \qquad \tilde{N}\big(\pi(i)\big) = \{\pi(j) : j \in N(i)\}$$

definition of PH and $PAP^\top$

$$\tilde{a}_{\pi(i)} = \sum_{l \in \tilde{N}(\pi(i))} M_\theta(\tilde{h}_{\pi(i)}, \tilde{h}_l) = \sum_{j \in N(i)} M_\theta(\tilde{h}_{\pi(i)}, \tilde{h}_{\pi(j)})$$

write $l = \pi(j)$; Lemma (p. 44)

$$= \sum_{j \in N(i)} M_\theta(h_i, h_j) = a_i$$

first line

$$\tilde{h}'_{\pi(i)} = U_\theta\big(\tilde{h}_{\pi(i)}, \tilde{a}_{\pi(i)}\big) = U_\theta(h_i, a_i) = h'_i$$

same U_θ at every node

In words: **a new number changes the name of each neighbour, not who it is.**

SO **Row $\pi(i)$ of the new output is row i of the old, so $\tilde{H}' = P \cdot H'$. The layer never reads a node's number.**

NEXT What follows for many layers and whole-graph answers?

From a plain sum to the normalised matrix in three steps

PROBLEM Why $1/\sqrt{\tilde{d}_i \cdot \tilde{d}_j}$ and not a plain sum?

WE DO Repair one flaw of the sum per step.

WHY Each step keeps one property: scale, own vector, symmetry.

$$\sum_{j \in N(i)} h_j$$

plain sum: **grows with** $\deg i$

$$\frac{1}{\deg i} \cdot \sum_{j \in N(i)} h_j$$

divide: an average

$$\frac{1}{\tilde{d}_i} \cdot \sum_{j \in N(i) \cup \{i\}} h_j, \quad \tilde{d}_i = \deg i + 1$$

self-loop $A + I$: own vector kept

$$\sum_{j \in N(i) \cup \{i\}} \frac{1}{\sqrt{\tilde{d}_i \cdot \tilde{d}_j}} \cdot h_j = (\hat{A} \cdot H)_i$$

split $\tilde{d}_i$ between both ends

SO **$\hat{A}$ is symmetric, and busy neighbours count less. Its eigenvalues lie in $[-1, 1]$, so repeating it never blows up** **(p. 128).**

NEXT What if a node has millions of neighbours?

Graph attention in full: score, weight, new vector

↳ The graph attention network (GAT) of **(p. 53)**. W: learned matrix; a: learned vector; $[u\,;v]$: u stacked on v.

$$e_{ij} = \mathrm{lrelu}\big(a^\top \cdot [\,W \cdot h_i\,;\,W \cdot h_j\,]\big)$$

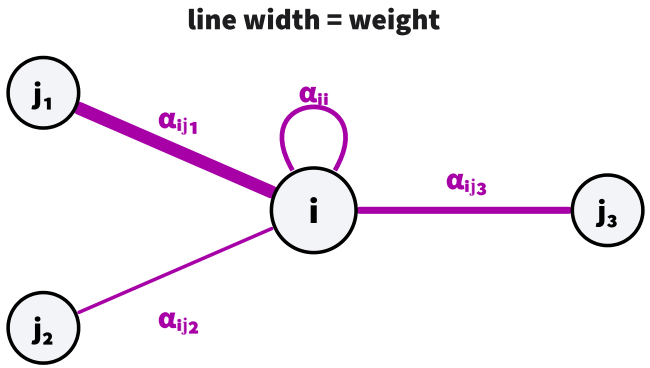
score from both vectors

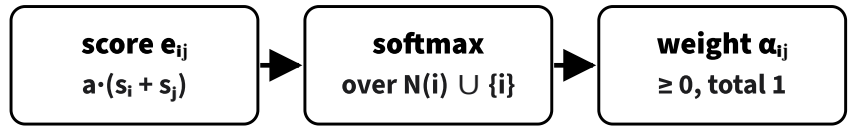
$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{l \in N(i)\cup\{i\}} \exp(e_{il})}$$

softmax: weights, total 1

$$h'_i = \mathrm{relu}\Big(\sum_{j \in N(i)\cup\{i\}} \alpha_{ij} \cdot W \cdot h_j\Big)$$

weighted sum





$\mathrm{lrelu}(u) = u$ if $u > 0$, else $0.2 \cdot u$ (leaky rectified linear unit); $\mathrm{relu}(u) = \max(u, 0)$ (rectified linear unit).

As messages: send $\exp(e_{ij}) \cdot W \cdot h_j$ and $\exp(e_{ij})$, sum both, divide in the update.

The three networks as formulas in one template

↳ Template **(p. 35)**: message $m_{j\to i}$, aggregate a_i, update h'_i. $\tilde{d}_i$: links of i plus one; $S(i)$: sampled neighbours.

network	message $m_{j\to i}$	aggregate	update h'_i
template	$M_\theta(h_i, h_j)$	**sum over** $N(i)$	$U_\theta(h_i, a_i)$
graph convolutional	$\frac{1}{\sqrt{\tilde{d}_i \cdot \tilde{d}_j}} \cdot W \cdot h_j$	**sum over** $N(i) \cup \{i\}$	$\mathrm{relu}(a_i)$
GraphSAGE	h_j	**mean over** $S(i)$	$\mathrm{relu}(W \cdot [h_i; a_i])$
graph attention	$\alpha_{ij} \cdot W \cdot h_j$	**sum over** $N(i) \cup \{i\}$	$\mathrm{relu}(a_i)$

link task loss: $-\ln \sigma(s_{ij}) - \ln\big(1 - \sigma(s_{ik})\big),\ s_{ij} = h_i^\top h_j$, i–j a link, i–k a random non-link

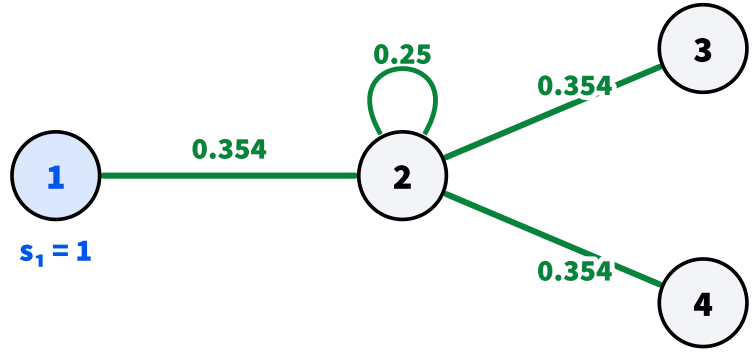
σ: the sigmoid **(p. 36)**; node and graph tasks use the cross-entropy of **(p. 37)**.

Graph convolution on the friends' tree, term by term

↳ One layer, $W = 1$, no relu; start $h = (1, 0, 0, 0)$ (p. 84).

term j	links of 2, plus 1: $\tilde{d}_2$	links of j, plus 1: $\tilde{d}_j$	weight $\frac{1}{\sqrt{\tilde{d}_2 \cdot \tilde{d}_j}}$	value h_j
friend 1	**4**	**2**	$1/\sqrt{8} = 0.354$	**1**
itself (2)	**4**	**4**	$1/4$	**0**
friends 3, 4	**4**	**2**	$1/\sqrt{8}$ **each**	**0**

$h_2' =$ sum of weight · value $= \frac{1}{\sqrt{8}} \cdot 1 + \frac{1}{4} \cdot 0 + \frac{1}{\sqrt{8}} \cdot 0 + \frac{1}{\sqrt{8}} \cdot 0 = 0.354$



weights on the links into node 2

node 2's new value: **0.354**

friend 1: 1 link + itself = 2; node 2: 3 links + itself = 4; weight 1 ÷ √(2 · 4) = 0.354

Node 1 by the same rule: $\frac{1}{2} \cdot 1 + \frac{1}{\sqrt{8}} \cdot 0 = 0.5$**; nodes 3 and 4: 0.**

GraphSAGE and graph attention on the friends' tree

↳ Same start $h = (1, 0, 0, 0)$, $W = 1$, no relu **(p. 85)**. GraphSAGE samples $S(2) = \{1, 3, 4\}$.

GraphSAGE: part	values used	how combined	result
own vector h_2	**0**	**kept apart**	**0**
mean over $S(2)$	$h_1, h_3, h_4 = 1, 0, 0$	$(1 + 0 + 0)/3$	**0.333**

Graph attention **(p. 86)**, illustrative $a = (1, 1)$: score $e_{2j} = \mathrm{lrelu}(h_2 + h_j)$.

term j	score e_{2j}	$\exp(e_{2j})$	weight $\alpha_{2j} = \exp(e_{2j})/\mathrm{total}$	value h_j
friend 1	**1**	**2.718**	$2.718/5.718 = 0.475$	**1**
itself, friends 3, 4	**0**	**1 each**	$1/5.718 = 0.175$ **each**	**0**
total		$2.718 + 3 \cdot 1 = 5.718$	**1**	

GraphSAGE: $h'_2 = [\,0\,;\,0.333\,]$**. Attention:** $h'_2 = 0.475 \cdot 1 + 0.175 \cdot (0 + 0 + 0) = 0.475$**.**

Proof: one layer reads only the neighbours

PROBLEM Why exactly one link per layer?

WE DO Induction on the layer t.

WHY A layer reads neighbours only.

$$h_i^{(0)} = \text{embedding of } x_i$$

start: own input only

$$h_i^{(t)} = U_\theta\Big(h_i^{(t-1)}, \sum_{j \in N(i)} M_\theta\big(h_i^{(t-1)}, h_j^{(t-1)}\big)\Big)$$

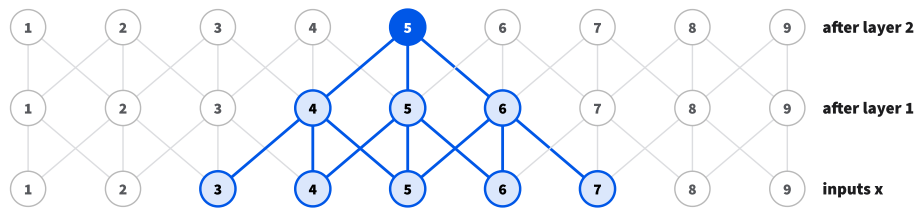
one layer (Part 1)

$h_j^{(t-1)}$ uses only x_k with $\text{dist}(j,k) \le t-1$

induction hypothesis, $j = i$ or $j \in N(i)$

$$\text{dist}(i,k) \le \text{dist}(i,j) + \text{dist}(j,k) \le 1 + (t-1) = t$$

triangle inequality



SO **By induction up to $t = T$: $h_i^{(T)}$ uses only the inputs in $N_T(i)$.**

NEXT Why does a far coin's pull fade link by link?

Proof: each link multiplies a coin's lean by 2κ − 1

↳ A coin's **lean** δ: how far its probability of A lies above $\frac{1}{2}$ (for ± 1 coins this is their correlation, up to a constant). The next coin equals it with chance $\kappa = \frac{4}{5}$, else it is the other coin **(p. 64)**.

$$p(z_j = A) = \tfrac{1}{2} + \delta, \qquad p(z_j = B) = \tfrac{1}{2} - \delta$$

coin j leans to A by δ

$$p(z_{j+1} = A) = \kappa \cdot \left(\tfrac{1}{2} + \delta\right) + (1 - \kappa) \cdot \left(\tfrac{1}{2} - \delta\right)$$

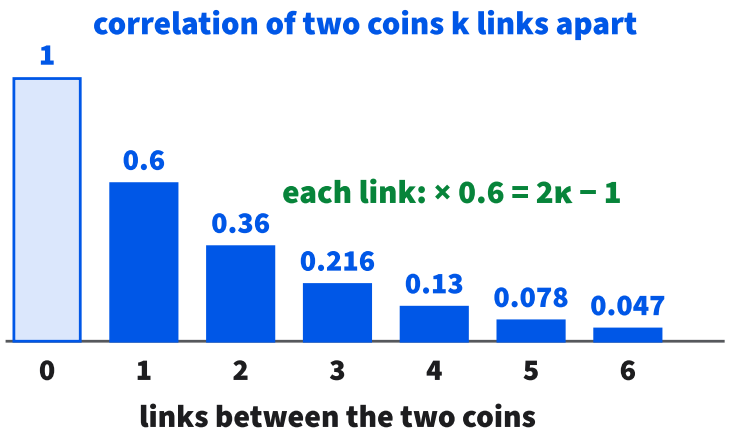
kept, or switched

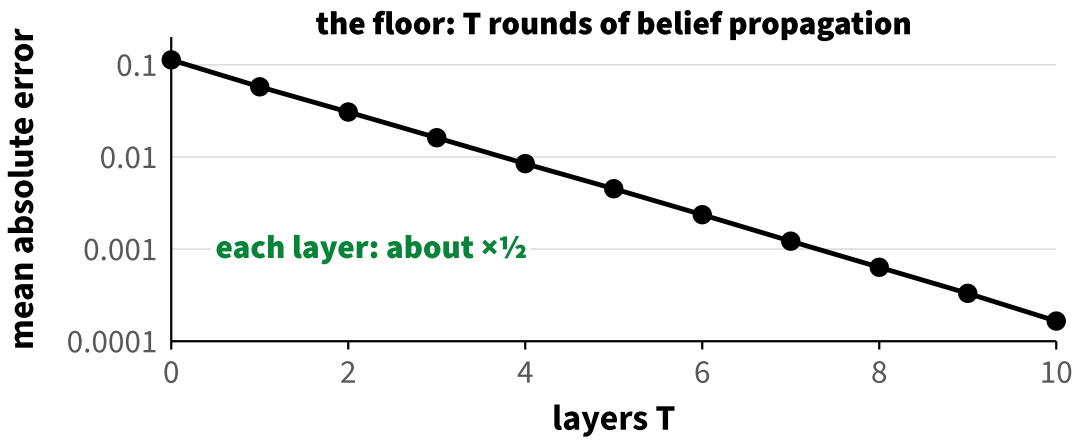
$$= \tfrac{1}{2} + (2 \cdot \kappa - 1) \cdot \delta = \tfrac{1}{2} + 0.6 \cdot \delta$$

collect terms; $\kappa = \frac{4}{5}$

$$k \text{ links}: \quad \tfrac{1}{2} + 0.6^k \cdot \delta$$

repeat link by link





Six links away, a coin keeps less than 5% of its lean.

Proof: each round multiplies each part by its eigenvalue

PROBLEM Why does λ_2 set the rate?

WE DO Expand h in eigenvectors; apply $\hat{A}$ t times (p. 126).

WHY Same algebra as Lecture 4.

$$h = \sum_{k=1}^{n} \gamma_k \cdot u_k$$

$$\hat{A}^t h = \sum_{k=1}^{n} \gamma_k \cdot \lambda_k^t \cdot u_k$$

$$\hat{A}\,\tilde{D}^{1/2}\mathbf{1} = \tilde{D}^{-1/2}(A+I)\,\mathbf{1} = \tilde{D}^{1/2}\mathbf{1}$$

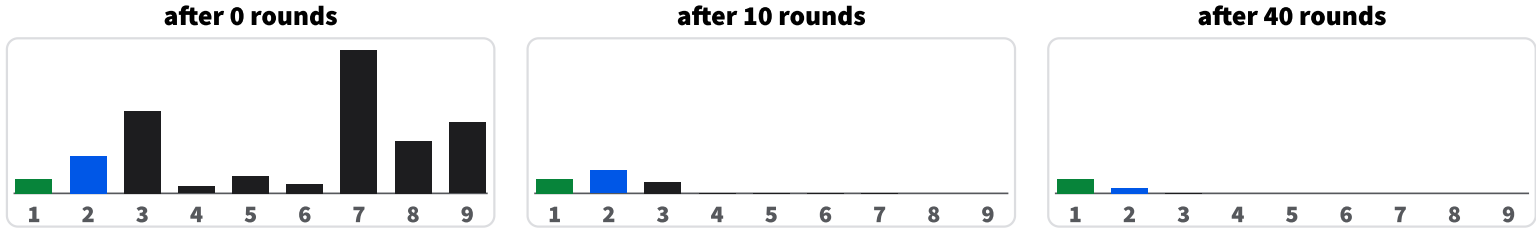
$$\left\|\hat{A}^t h - \gamma_1 \cdot u_1\right\| \le |\lambda_2|^t \cdot \|h\|$$

$\hat{A}$ symmetric: orthonormal eigenvectors

$\hat{A}\,u_k = \lambda_k \cdot u_k$, each round

$\lambda_1 = 1, u_1 \propto \tilde{D}^{1/2}\mathbf{1}$

connected: other $|\lambda_k| \le |\lambda_2| < 1$



bar k: size of eigenvector part k; green: k = 1, λ = 1, kept; blue: k = 2, λ = 0.953

SO $\tilde{D}^{-1/2}\hat{A}\,\tilde{D}^{1/2} = \tilde{D}^{-1}(A+I)$ **is a random walk: oversmoothing is Markov-chain mixing (Lecture 4, p. 62).**

NEXT How fast does the receptive field grow?

On a tree the receptive field doubles with each link

↳ A tree in which every node has 3 links; $N_T(i)$: the nodes within T links of node i (p. 67).

$$\text{distance } 1: \ 3 \text{ nodes}$$

node i's own 3 links

$$\text{distance } t: \ 3 \cdot 2^{t-1} \text{ nodes}$$

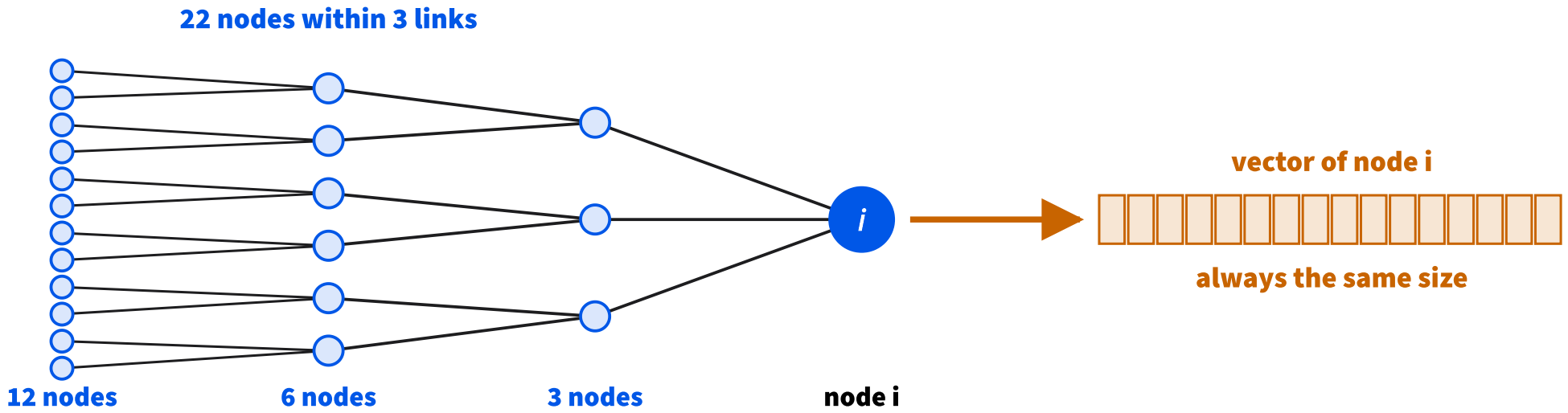
each node has 2 outward links

$$|N_T(i)| = 1 + 3 \cdot \left(1 + 2 + \cdots + 2^{T-1}\right) = 1 + 3 \cdot (2^T - 1)$$

geometric sum

$$T = 3: \ 1 + 3 \cdot 7 = 22; \qquad \text{chain}: \ 2 \cdot T + 1 = 7$$

tree against chain



The field grows exponentially; the vector stays at 16 numbers.

Colour refinement as a formula

↳ $c_i^{(t)}$: node i's colour after round t. $\{\{\cdot\}\}$: a multiset, a set that keeps repeats. Hash: one new colour for each distinct pair (p. 73).

$$c_i^{(0)} = x_i$$

start: the colour is the input

$$c_i^{(t)} = \text{hash}\Big(c_i^{(t-1)},\ \{\{c_j^{(t-1)} : j \in N(i)\}\}\Big)$$

own colour, neighbours' colours

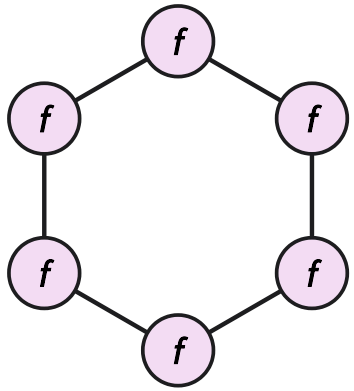
$$\text{path of 4, round 1}: \ (a, \{\{a\}\}) \mapsto b, \ \ (a, \{\{a, a\}\}) \mapsto c$$

ends and middles split

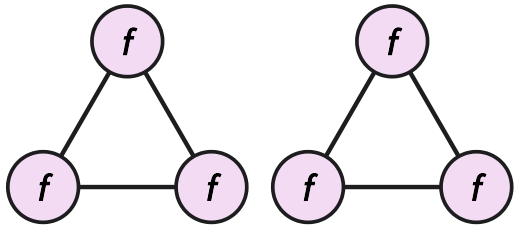
stop when no colour splits

at most n rounds

round 2: no colour splits any further



6-ring



two triangles



path of 4 nodes

If two graphs end with different colour counts, they are surely different.

Proof: equal colours pass down to equal vectors

PROBLEM Why equal colours, equal vectors?

WE DO Induction on the round t.

WHY Hash and layer read the same things.

$$c_i^{(0)} = c_j^{(0)} \Rightarrow x_i = x_j \Rightarrow h_i^{(0)} = h_j^{(0)}$$

start: colour = input

$$c_i^{(t)} = c_j^{(t)} \Rightarrow c_i^{(t-1)} = c_j^{(t-1)} \text{ and } \{\{c_k^{(t-1)}\}\}_{k \in N(i)} = \{\{c_k^{(t-1)}\}\}_{k \in N(j)}$$

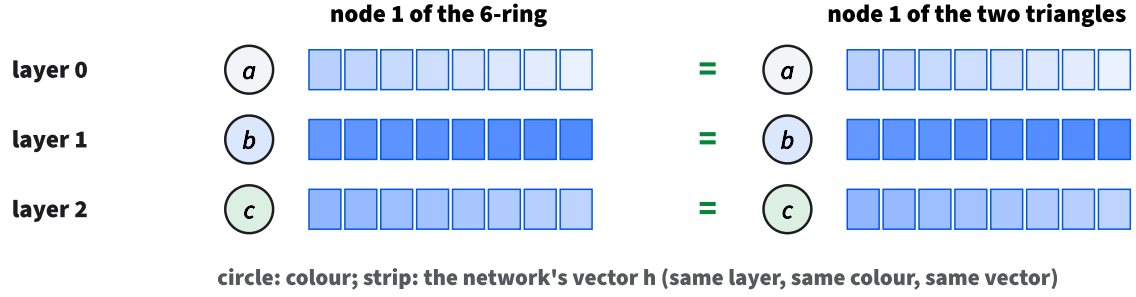
one colour per pair

$$\Rightarrow h_i^{(t-1)} = h_j^{(t-1)} \text{ and } \{\{h_k^{(t-1)}\}\}_{k \in N(i)} = \{\{h_k^{(t-1)}\}\}_{k \in N(j)}$$

induction hypothesis

$$\Rightarrow a_i^{(t)} = a_j^{(t)} \Rightarrow h_i^{(t)} = U_\theta\left(h_i^{(t-1)}, a_i^{(t)}\right) = h_j^{(t)}$$

a sum sees only the multiset



SO **No message-passing network separates more than refinement.**

NEXT Which networks reach this limit? (p. 75)